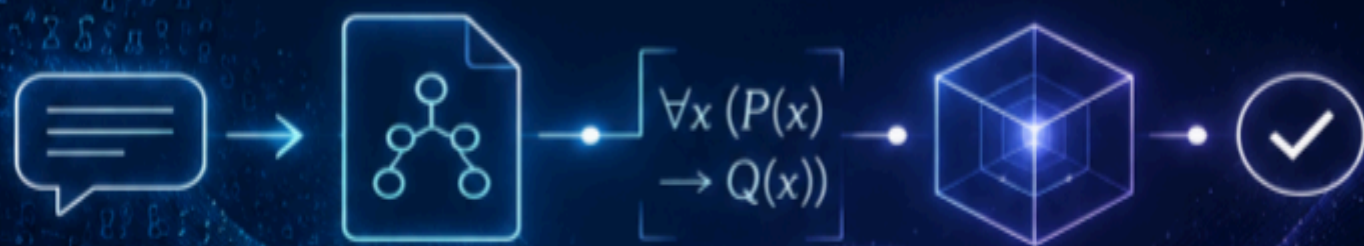


LIMBAJ NATURAL EXECUTABIL

Un studiu critic al semanticii formale, al
interpretării abstracte, al execuției simbolice
și al arhitecturilor LLM fundamentate simbolic



LIMBAJ NATURAL EXECUTABIL

Un studiu critic al semanticii formale, al interpretării abstracte, al execuției simbolice și al arhitecturilor LLM fundamentate simbolic

Către un runtime neuro-simbolic pentru documente lungi, reguli și sisteme agentice



Copyright © [2026] Axiologic ResearchZ

Toate drepturile rezervate.

KDP drepturi de publicare deținute de Outfinity SRL (Iași, România).

Disponibil gratuit pe site-ul Axiologic (www.axiologic.net).

Nota autorului

Această lucrare a fost creată sub umbrela Axiologic Research ca parte a două programe de cercetare interconectate, unul dedicat inteligenței artificiale și celălalt către Outfinitism, ambele conduse de S În Albășcă. Pentru a explora detaliile despre modul în care aceste două programe se intersectează și se construiesc unul pe celălalt, invităm cititorii să viziteze pagina Outfinity Venture Studio [de la www.outfinity.ch](http://www.outfinity.ch). În timp ce modelele de limbaj de inteligență artificială au fost folosite pentru a ajuta la generarea de text, filosofia fundamentală, structura tematică și abordarea analitică derivă exclusiv din deceniile de studiu dedicat de către autor a fenomenelor sociale, a tehnologiilor sociale și a tehnologiilor dure autentice, rezultate din diverse proiecte europene și de cercetare autofinanțate. Recent, în cadrul proiectului de cercetare Ahile, am studiat profund AI literatura generată de literatură și verificarea sa automată folosind abordări neuro simbolice; toate cărțile gratuite puse la dispoziția publicului fac parte din suita de lucrări pe care le folosim pentru a ne testa cele mai recente tehnologii.

Cuprins

Rezumat.....	5
Prefață: domeniu, metodă și poziție critică.....	8
1. De la limbaj fluent la semantică executabilă.....	11
1.1 Ce presupune execuția și unde trebuie trasată limita încrederii.....	11
1.2 Decalajul formalizării: de ce un solver corect poate produce totuși răspunsul greșit.....	15
2. Genealogia intelectuală a limbajului executabil.....	18
2.1 Limbaje controlate, semantică formală, discurs și logică naturală.....	18
2.2 Analiză semantică, interogări executabile, inducție de programe și sisteme neuro-simbolice timpurii.....	21
3. Interpretare abstractă și execuție simbolică pentru limbaj.....	24
3.1 Interpretarea abstractă ca semantică a incertitudinii delimitate.....	24
3.2 Execuție simbolică, rezolvarea constrângerilor, verificarea modelelor și rafinare semantică.....	28
4. Ce a realizat de fapt era LLM.....	32
4.1 Autoformalizare, raționament asistat de programe, logică, specificații temporale și planificare.....	32
4.2 Analiză de programe asistată de LLM, explorare simbolică, transformatori abstracți și NLP certificat.....	36
5. De ce eșuează sistemele de limbaj executabil.....	40
5.1 Fundamentare, identitate, formalizări spurii, ontologii și cunoaștere despre lume.....	40
5.2 Pragmatică, nonmonotonicitate, scară, securitate și eșecul evaluării.....	44
6. Familii arhitecturale pentru limbaj natural executabil.....	48
6.1 Proiecte concurente: reprezentări universale, portofolii de formalisme, compilare controlată și sisteme query-first.....	48
6.2 Un pipeline de referință de la text la simboluri, execuție și răspunsuri constrânse în limbaj natural.....	53
7. LongTextJS, CircuitJS și nllAgent: un studiu de caz critic.....	56
7.1 Ce implementează arhitectura publică și de ce contează granițele sale.....	57
7.2 Evaluare independentă: puncte forte, probleme serioase actuale și evoluția necesară..	60
8. Construirea și validarea unui runtime neuro-simbolic.....	64
8.1 Strategie de implementare: un sistem de limbaj executabil minimal, dar onest științific.	64
8.2 Întrebări de cercetare, benchmarkuri și un program de publicare.....	71
Concluzie: de la formă probabilistică la consecință simbolică.....	77
Referințe.....	80

Rezumat

Modelele lingvistice mari au făcut ca limbajul natural să fie o interfață eficientă cu calculul, totuși nu au făcut limbajul natural exenuabil în sensul puternic în care un program, o interogare a bazei de date, un sistem de tranziție sau o teorie logică este executabilă. Un model poate genera o explicație convingătoare în timp ce schimbă domeniul de aplicare al unui cuantificator, omițând o excepție, inventând o premisă sau aplicând aceeași regulă în mod diferit în două locuri. Instrumentele externe îmbunătățesc fiabilitatea numai atunci când modelul a tradus corect cererea. Un demonstrator de teoreme se poate dovedi perfect formalizarea greșită; un interpret Python poate executa un program care răspunde accidental la exemplu în timp ce denaturează întrebarea; un executor simbolic poate expune o cale reală prin software, chiar dacă regula în limba naturală folosită pentru a construi starea căii a fost tradusă greșit. Prin urmare, problema dificilă nu este doar să adăugați un rezolvator după un LLM. Este de a construi un lanț auditiv de la limbajul sursă la simboluri împământate, de la simboluri împământate la obiecte formale executabile și de la rezultate simbolice înapoi la limbajul natural, fără a permite modelului de limbă finală să modifice ceea ce a fost stabilit.

Această carte chestionează tradițiile relevante ca fiind o problemă de cercetare conectată. Bazele anterioare includ limbaje naturale controlate, model-teoreic și discurs semantic, logică naturală, parsare semantică, programare logică, sisteme text-la-interogare, sinteză de programe de exemple și limbaj, motoare de reguli, tabele de decizie, logici temporale, planificare automată și sisteme neuro-simbolice. Fundațiile programului-analiză includ interpretarea abstractă, execuția simbolică și conciliidă, analiza fluxului de date, verificarea modelului, SAT/SMTZ/CSPa rezolvarea, Datalog și evaluarea punctului de fixare, automatele și verificarea producătoare de dovezi. Literatura recentă LLM adaugă raționament dedicat programului, determinare de rezolvare declarativă, autoformalizare, traducere de primă comandă, traducere temporală-logică temporală, planificare formală, auto-reparare ghidată de rezolvare, analiza programului asistată de LLM, explorarea simbolică ghidată LLM și sinteza transformatoarelor abstracte verificate. În aceste corpuri de muncă, cele mai puternice rezultate apar în mod constant atunci când formalismul țintă este suficient de îngust pentru a avea o semantică explicită, iar producția acceptată este verificată de o componentă independentă de LLM.

Sondajul acordă o atenție deosebită distincției dintre două utilizări ale interpretării abstracte. Primul, deja stabilit în NLP, verifică un model neuronal peste o familie definită oficial de perturbări textuale. Domeniile abstracte de mijloc și relaționale pot certifica faptul că predicția unui clasificator nu se schimbă sub fiecare substituție permisă de un set adversaric specificat. Aceasta este o garanție formală autentică, dar se referă la stabilitatea modelului în cadrul unei relații de perturbare construită; nu certifică faptul că parafările umane arbitrare păstrează sensul și nici că răspunsul original al modelului nu este adevărat. A doua utilizare este mai speculativă și mai relevantă pentru limbajul natural executabil: tratarea setului de interpretări admisibile ale unui document ca un semantic de colectare și de calcul conservatoare trebuie, nu poate, nu poate și să necunoscute asupra aceluși set. Acest lucru necesită domenii explicite pentru identitate, timp, modalitate, cantitate, statut sursă, forță normativă și acoperire. De asemenea, necesită o garanție condiționată onestă: analiza este solidă numai în raport cu interpretarea stabilită, tonologia, politica de împământare și normele de transfer care îi sunt furnizate.

Execuția simbolică este complementară. Interpretarea abstractă poate arăta ieftin că o încălcare este imposibilă sub abstractizarea actuală sau că rămâne posibilă o încălcare.

Execuția simbolică poate încerca apoi să construiască o interpretare și o cale concretă care să o realizeze. Dacă nu există o astfel de cale, martorul eșuat poate dezvălui care abstracție sau alegerea semantică a fost prea grosieră. Acest lucru sugerează un model de cercetare numit rafinament semantic ghidat de contraexalipsă: un nucleu de contraexaltație, nesatisfăcut sau o dovadă eșuată este cartografiat înapoi la pasajele exacte ale sursei și angajamentele semantice care l-au generat; LLMZ sau un om este rugat să revizuiască doar aceste angajamente; analiza simbolică este reluată. LLM este folosit pentru a căuta spațiul formalizărilor, în timp ce sâmburele simbolice determină ce urmează din fiecare formalizare acceptată.

Sistemele recente prezintă fragmente importante ale acestei viziuni. PAL deleagă calculul procedural pentru Python. SatLM generează constrângeri declarative pentru un motor de satisfacătoare. Logic-LM și LINC traduc sarcinile de raționament în limba naturală în logica formală și se bazează pe rezolvatoare simbolice sau pe dovezile de la acestea pentru deducere. NL2TL și nl2spec traduc cerințele în specificațiile temporale, nl2spec susținând în mod explicit corectarea interactivă la nivel de subformulă. Lucrările de planificare formală au arătat câștiguri mari atunci când LLMs construiesc instanțe de planificare care sunt apoi rezolvate și verificate de instrumente de sunet. SymGPT traduce regulile în limbaj natural ERC într-o gramatică restricționată și utilizează execuția simbolică pentru a găsi încălcări concrete ale contractului inteligent, raportând mii de încălcări ale regulilor și căi explicite de atac. În cealaltă direcție, AutoBug descompune analiza programului pe căi simbolice LLM, dar rămâne aproximativ; SAIL folosește LLMs pentru a căuta transformatoare abstracte, în timp ce verificările semantice și sintactice dure păstrează soliditatea globală. Aceste rezultate sunt dovezi că LLMs sunt propuneri eficiente și mecanisme de căutare în jurul nucleelor formale, nu dovezi că generarea probabilistică în sine a devenit raționament formal.

O revizuire sistematică recentă a neuro-simbolului NLP susține că arhitecturile federative, în care modulele neuronale și simbolice rămân autonome și comunică prin interfețe structurate, arată adesea câștiguri mai mari decât arhitecturile care doar injectează prejudecăți simbolice în rețelele neuronale. Autorii săi avertizează, de asemenea, că avantajul aparent nu este încă concludent, deoarece sistemele federative și injective sunt de obicei evaluate pe diferite tipuri de sarcini. Această carte adoptă aceeași poziție disciplinată. Cazul arhitectural pentru separare este puternic, deoarece clarifică limita de încredere, permite reluarea independentă și menține componenta simbolică inspectabilă; afirmația empirică că o familie depășește universal o alta rămâne nedovedită [Chatzikyriakidis and Lappin 2026].

Cartea oferă, de asemenea, un studiu de caz critic al arhitecturii publice NaturalLanguageLinterAgen, sau nllAgent, și a componentelor sale LongTextJS și CircuitJSZ. Implementarea separă un program de document imuabil de un program de teorie reutilizabil. Acesta păstrează ancorele de sursă exacte, tipurile de observație versiuni, identitățile producătorului, acoperirea, lacunele și statutul epistemic. Revizuirea circuitului derivă observațiile cerute efectiv de o regulă, publicarea verifică dacă există producători compatibili compatibili, compatibilitatea timpului de rulare testează dacă documentul de beton a dat observații și acoperire suficiente, iar un verficator independent controlează dacă este publicată o constatare a candidaților. Învățarea are loc într-un spațiu de lucru de unică folosință și nu poate publica automat. Acestea sunt decizii de guvernanta neobișnuit de serioase pentru un sistem de analiză a documentelor asistată de LLM. Depozitul public înregistrează, de asemenea, limitări nerezolvate, mai degrabă decât ascunderea lor. Registrul său actual de probleme grave identifică incrementalitatea programată limitată, propagarea incompletă a alternativelor de interpretare prin fiecare cale de execuție, analiza compatibilității conservatoare

pentru comportamentul procedural opac, planificarea minimă a interogării și indexarea temporală, generarea controlată în mod deliberat îngustă și sprijinul mutațiilor care rămâne specific scenariului. Acestea sunt limitări credibile ale cercetării: ele se referă la substratul semantic și de execuție, nu doar lipsa lustruirii dintre utilizator-interfață.

Nicio evaluare independentă a colegilor sau o evaluare tehnică a terților a nllAgent nu a fost localizată în căutările efectuate pentru această ediție. În consecință, evaluarea studiului de caz este în mod explicit propria revizuire arhitecturală a autorului, fundamentată în depozitul public, documentația, domeniul de aplicare executabil și problemele grave autodeclarate. Judecata este favorabilă cu privire la separarea probelor, teoriei, verificării și publicării, dar precaută cu privire la caracterul complet semantic. Cele mai puternice revendicări actuale de punere în aplicare se referă la controale structurale și controlate în limba engleză. Rezolvarea generală a identității, semantica bogată a evenimentelor, cunoștințele din lumea actuală, transferul larg de domeniu și revendicările de calitate susținute statistic rămân lucrări de cercetare, mai degrabă decât capacitatea stabilită.

Propunerea centrală a cărții este o perioadă de funcționare multi-formalism executabilă-natural-language. Un LLM propune pentru prima dată un set mic de structuri semantice candidate, fiecare legat de întinderi exacte ale sursei și tastate pe o tulburare selectată. Un sâmbure de fundament simbolic admite numai obiecte ale căror identitate, proveniență, domeniu de aplicare, timp, modalitate și dovezi satisfac contracte explicite. Un router selectează una sau mai multe ținte formale în funcție de întrebarea: evaluarea relațională sau Datalog pentru interogările de închidere și documente; tabele de decizie pentru politici; SAT/ZX0004QXZ/CSPz pentru constrângeri; automat pentru modelele ordonate; verificarea modelului pentru comportamentul temporal; execuția simbolică pentru martorii căii; și asistente doveditoare sau prover pentru precurse pentru revendicări deductive. Execuția formală produce un rezultat canonic care conține fapte acceptate, candidați respinși, incertitudine, martori, obiecte de probă, limite de acoperire și ipoteze. Un ultim LLM poate explica acest rezultat, dar fiecare propoziție din explicație trebuie să fie licențiată de structura canonică și trasabilă la probe sau derivare verificată.

Prin urmare, limbajul natural executabil nu trebuie promovat ca executarea prozei nerestricționate. Acesta ar trebui să fie definit ca un proces controlat de compilare, împământare, execuție și realizare față de unul sau mai mulți candidați semantici expliți. Garanția este condiționată, dar utilă: având în vedere revizuirea sursei declarate, ontologia, împământarea, setul de interpretare, semantica formală și limitele de execuție, rezultatul raportat urmează și poate fi reluat. Principala problemă de cercetare deschisă nu este rezolvatorul. Este construcția, comparația, rafinarea și validarea formalizărilor pe care operează rezolvatorul.

Prefață: domeniu, metodă și poziție critică

Ambiția limbajului natural executabil este ușor de exprimat și ușor de supraestimat. Un utilizator scrie o regulă, o politică, o ipoteză științifică, o procedură sau o întrebare într-un limbaj obișnuit; un sistem îl înțelege; conținutul urmează; rezultatul este returnat ca limbaj obișnuit. Modelele de limbaj contemporan fac ca începutul și sfârșitul acestei conduite să pară rezolvate. Ei pot citi proza nerestricționată, pot genera cod și formule, invocă instrumente și pot explica fluent rezultatele. Iluzia periculoasă este că mijlocul a dispărut. De fapt, mijlocul conține aproape fiecare problemă dificilă: să decidă care înseamnă contează, să identifice obiectele de-a lungul pasajelor, să distingem dovezi explicite de ipoteze de fond, să păstreze modalitățile și excepțiile, să selecteze un formalism adecvat, dovedind că artefactul generat este bine format și împiedicând explicația finală să spună mai mult decât execuția stabilită.

Un studiu de sinteză util nu poate fi un catalog de sisteme mici prezentate în subsecțiuni scurte izolate. Câmpul este fragmentat tocmai pentru că aceleași cuvinte sunt folosite pentru operațiuni diferite. „Execuția simbolică” poate însemna explorarea căii în stil King asupra intrărilor simbolice ale programului, executarea unei forme logice generate pe o bază de cunoștințe sau pur și simplu utilizarea unui interpret extern. „Verificarea formală” se poate referi la corectitudinea unui rezultat de rezolvare, corectitudinea unui program compilat, fidelitatea traducerii în limba naturală sau robustețea unei rețele neuronale sub o familie de perturbare predefinită. „Grounding” poate însemna conectarea cuvintelor la coloanele de baze de date, entitățile pentru a documenta întinderile, propunerile la cunoașterea lumii sau instrucțiuni robot la obiecte perceptive. Un tratament serios trebuie să arate modul în care aceste straturi se referă și unde se opresc garanțiile.

Prin urmare, metoda sondajului urmează mecanisme, mai degrabă decât categorii de marketing. Pentru fiecare linie de lucru, analiza pune cinci întrebări. Ce artefacte este produs din limbaj? Ce semantică are acel artefact independent de LLM? Cum se împământează artefactul în sursă sau în mediul înconjurător? Ce componentă determină de fapt rezultatul? Ce este și nu este garantat atunci când sistemul reușește? Lucrările sunt tratate ca dovezi pentru sarcinile lor raportate, nu ca o dovadă a unei arhitecturi generale. Rezultatele numerice puternice sunt incluse atunci când clarifică mecanismul, dar nu sunt transferate în domenii cu structură semantică diferită. Orizontul sondajului este 2 august 2026 și include literatură evaluată de colegi, proceduri oficiale, lucrări de cercetare primară și depozitul public nllAgent și documentație.

Termenul de „recenzie” este folosit cu atenție. Există acum mai multe sondaje largi, dar fiecare acoperă doar o parte a problemei end-to-end. Sondajul controlat-natural-în limba Kuhn organizează limbi în funcție de precizie, expresivitate, naturalitate și simplitate [Kuhn 2014]. Hamilton și colegii săi evaluează dacă neuro-simbolul NLP a oferit de fapt raționament, transfer, intercalabilitate, eficiență a datelor și generalizarea în afara distribuției, în timp ce sondajele mai recente clasifică sistemele neuro-simbolificate bazate pe sarcini și bazate pe date și cunoștințe. Sondajele privind execuția simbolică și analiza programului asistată de LLM descriu limitele ingineriei ale explorării căii și rolul tot mai mare al componentelor [Baldoni et al. 2018; Wang et al. 2025a] învățate. O revizuire dedicată studiază LLMs ca formalizatori pentru planificarea [Tantakoun et al. 2025] automată, iar sondajul de autoformalizare se concentrează pe traducerea matematicii informale în forme [Weng et al. 2025] de verificare a mașinii. O foaie de parcurs bidirecțională pentru cerințele examinează atât utilizarea LLMs pentru a face metodele

formale accesibile, cât și utilizarea metodelor formale pentru a constrânge cerințele [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b] produse [Ferrari and Spoletini 2025]. Un studiu de sinteză concentrat pe treizeci și cinci de lucrări cartografiază LLM formalizarea și specificațiile software asistate de software, inclusiv munca care vizează Dafny, C și Java; domeniul său de aplicare confirmă faptul că suportul de traducere crește mai repede decât asigurarea [Beg et al. 2025] semantică end-to-end. Revizuirea sistematică a Frontierelor din 2026 dezvoltă o nouă taxonomie pentru neuro-simbolul NLP și susține o mai mare atenție asupra sistemelor [Chatzikyriakidis and Lappin 2026] federative.

Nicio revizuire a sondajului nu unifică limbajul controlat, împământarea semantică, interpretarea abstractă, execuția simbolică, selecția formalismului, rezultatele purtării dovezilor și regenerarea constrânsă ca o singură arhitectură de încredere. Această concluzie este mai degrabă o sinteză a literaturii decât o afirmație de inexistență exhaustivă: terminologia diferă între domenii și lucrările relevante sunt distribuite în lingvistică de calcul, limbaje de programare, baze de date, metode formale, inginerie software, planificare și robotică. În cazul în care nu există o revizuire independentă, în special pentru nllAgent, textul își etichetează judecățile ca fiind o analiză originală, mai degrabă decât să implice aprobarea externă.

Poziția cărții este avizată, dar conservatoare. Respinge ideea că un model mare ar trebui să fie de încredere ca factorul negativ final doar pentru că proza sa seamănă cu o dovadă. De asemenea, respinge ideea opusă că doar un semantic complet scris de mână poate susține executarea formală utilă. În domeniul deschis, un LLM este adesea cel mai bun mecanism disponibil pentru propunerea de evenimente, relații, scheme formale și cartografierea candidaților. Întrebarea arhitecturală este cum să constrângeți această putere. Cel mai apăsător răspuns este un sistem federat cu artefacte intermediare explicite, interfețe tastate și versiuni, execuție simbolică independentă, stări conservatoare necunoscute și dovezi rejucare.

Această poziție are o consecință importantă. Scopul sistemului nu este întotdeauna de a produce o formalizare definitivă. Uneori, artefactul corect este un set de alternative. O pronume poate face referire la doi posibili actori; o frază legală poate admite domenii de aplicare concurente; o cerință poate fi incompletă; două surse pot să nu fie de acord; un document poate să nu spună dacă se aplică o excepție. Un timp simbolic care alege în mod arbitrar o interpretare este mai puțin riguros decât un LLM care afirmă cu sinceritate ambiguitatea. Executabilitatea devine valoroasă atunci când poate calcula ceea ce urmează sub fiecare citire admisibilă, ceea ce urmează sub cel puțin unul, care concluzii depind de o alegere contestată și ce dovezi suplimentare ar rezolva alegerea.

Prin urmare, arhitectura propusă tratează incertitudinea din punct de vedere structural. Scorurile de încredere pot clasifica candidații, dar nu transformă o interpretare propusă într-un fapt certificat. Statutul unui element înregistrează modul în care a devenit cunoscut: extras direct dintr-un parsterminist, propus de un model, asumat pentru un scenariu, derivat de o regulă, verificat de un verificador sau confirmat de o persoană. Acoperirea înregistrează dacă un domeniu a fost complet examinat, eșantionat, recuperat sau nu prelucrat. O concluzie negativă este permisă numai atunci când căutarea relevantă este închisă în cadrul unei politici declarate. Aceasta este diferența dintre un sistem simbolic fiabil și un clasificator fluent cu o ieșire în formă de JSON.

Discuția rămâne mai degrabă accesibilă decât rezistentă. Interpretarea abstractă, execuția simbolică, verificarea modelului și semantica logică sunt explicate prin esența lor operațională și obligațiile pe care le creează. Notarea formală apare doar în cazul în care clarifică o distincție

care proză s-ar putea estompa. Obiectivul nu este de a prezenta matematica decorativă. Este de a oferi designerilor de sistem, recenzorilor și cercetătorilor suficientă precizie conceptuală pentru a decide unde este valabilă o garanție propusă, unde este condiționată și unde este doar aspirațională.

Teza rezultată poate fi afirmată în mod clar. LLMs ar trebui să descopere forme formale plauzibile; componentele deterministe și în mod oficial constrânse ar trebui să controleze împământarea, compilarea, execuția și acceptarea; LLMs poate realiza apoi rezultatul canonic ca limbă, dar numai în temeiul unui contract care împiedică adăugirile neacționate. Aceasta nu este o soluție completă la sens. Este o cale credibilă de la o transformare a limbajului probabilistic la raționamentul limitat, executabil și auditabil.

1. De la limbaj fluent la semantică executabilă

1.1 Ce presupune execuția și unde trebuie trasată limita încrederii

Cuvântul „execută” poartă un angajament mai puternic decât „interpretarea”, „răspunsului” sau „rațiunea”. Un program este executabil deoarece operațiunile sale au un efect definit asupra unui stat. O interogare de bază este executabilă deoarece operatorii relaționali determină ce tuples sunt selectați, afiliați, grupați sau excluși. O problemă de planificare este executabilă deoarece acțiunile au precondiții și efecte, iar un planificator caută o secvență care atinge un obiectiv. O teorie logică este executabilă într-un sens mai larg, deoarece o procedură de inferență determină dacă urmează o concluzie, dacă există un model sau dacă constrângerile sunt satisfăcătoare. În fiecare caz, operațiunea decisivă este externă retoricii utilizate pentru a descrie sarcina. Un rezultat poate fi reprodus dintr-un artefact și semantica.

Un răspuns LLM este diferit. Modelul cartografiază un context unei distribuții asupra continuărilor. Poate internaliza modelele care seamănă cu algoritmi și poate rezolva corect multe sarcini, dar regula aplicată la un anumit pas nu este inspectabilă separat sau garantată a fi utilizată în mod consecvent. Procedura de gândire poate îmbunătăți performanța, dar un lanț de propoziții nu este un obiect proof doar pentru că este organizat ca pași. Înconsecința de sine enunțează mai multe continuări, nu mai multe execuții definite în mod oficial. Utilizarea sculei poate face o valoare exactă, lăsând în același timp interpretarea care a produs apelul instrumentului neverificat. Distincția crucială este între un model care propune un artefact executabil și un model a cărui generație proprie este tratată ca execuție.

Prin urmare, limbajul natural exenuabil începe cu materializarea. Sistemul trebuie să creeze un obiect cu o gramatică declarată, un sistem de tip sistem, semantica și o hartă sursă. Obiectul respectiv ar putea fi o interogare relațională, o teorie logică, o mașină de stat, un set de constrângeri temporale, un tabel de decizie, un Datalog program, un automat finit sau un grafic tip cu operatorii înregistrați. Alegerea particulară depinde de sarcină. Ceea ce contează este că comportamentul obiectului nu mai este determinat de distribuția următoare a modelului de limbă. Poate fi respins, executat, testat, comparat și reluat independent.

Această cerință este mai puternică decât producția structurată obișnuită. Un model poate emite JSON validă din punct de vedere sintactic, care conține entități inventate sau câmpuri neconsistente reciproc. O schemă validează forma, nu adevărul. În mod similar, un model poate genera SQL legal care se alătură coloanelor greșite, logica validă de prim ordin care inversează un cuantificator sau o formulă temporală care exprimă relația de prioritate opusă. Prin urmare, o limită utilă a încrederii nu este pur și simplu „după JSON generația”. Se află după împământarea sursei, verificarea tipului, compatibilitatea semantică și compilarea formală. Fiecare simbol care afectează răspunsul trebuie să aibă o origine admisibilă și un rol definit.

Arhitectura separă în mod natural patru autorități. Autoritatea sursă stabilește ce text, date sau observații au fost furnizate efectiv. Autoritatea semantică determină ce tipuri, relațiile, modalitățile și politicile de identitate sunt permise. Autoritatea de reguli determină ce constrângeri sau transformări constituie teoria care este executată. Autoritatea de executare stabilește dacă un rezultat al candidatului urmează în această teorie. Un LLM îi poate ajuta pe toți patru, dar nu ar trebui să le îmbine în tăcere. Când același model inventează o premisă, selectează o ontologie, aplică o regulă și scrie verdictul, nu există o separare semnificativă între dovezi și concluzie.

Autoritatea sursă începe cu oțete stabile și se întind. Un sistem de analiză a documentelor ar trebui să păstreze revizuirea exactă, structura și localizarea fiecărui fragment citat. Acesta ar trebui să distingă un citat de un rezumat și un rezumat de o inferență. Atunci când un model propune ca o propoziție să exprime o obligație, observația trebuie să indice propoziția și să păstreze modelul, promptul, schema și alternativele care au produs clasificarea. Dacă citatul nu poate fi găsit în sursă, observația este respinsă mai degrabă decât acceptată ca o reconstrucție plauzibilă. Acest lucru sună ca o inginerie de proveniență banală, dar este o condiție prealabilă pentru fiecare garanție ulterioară.

Autoritatea semantică este de obicei distribuită într-o ontologie superioară și pachete de domenii. O ontologie universală este atractivă pentru că promite interoperabilitate, dar tinde fie să devină prea abstractă pentru a decide întrebări utile sau prea mari și controversate pentru a guverna. Un design mai bun definește un mic strat comun pentru întinderi de surse, mențiuni, entități, evenimente, timp, lumi, stare, proveniență, alternative, acoperire și lacunele. Pachetele de domenii definesc apoi concepte precum intervenția clinică, obligația de reglementare, evenimentul obiectului narativ, apelul software API sau tranzacția financiară. Fiecare pachet trebuie să precizeze nu numai numele de teren, ci și angajamentele și încercările semantice. Un tip numit „aprobare” nu este semnificativ decât dacă sistemul specifică dacă denotă un document, un act, un stat, o decizie de autoritate sau o combinație.

Autoritatea de reguli ar trebui să fie redresată separat de interpretarea documentului. O politică, o teorie a luciderii sau un model științific se schimbă la o rată diferită de sursa pe care o analizează. Dacă regulile sunt încorporate invizibil în solicitări, devine dificil să știm dacă două runde au aplicat aceeași teorie. Artefactele de reguli psicite permit publicarea la reper, diff-urile semantice și rollback-ul. Ele permit, de asemenea, mai multe teorii asupra aceleiași reprezentări a documentelor. Un manuscris poate fi verificat pentru consistența terminologiei, coerența cauzală și sprijinul de citare fără a-l recomplimenta într-un prompt diferit opac pentru fiecare regulă.

Autoritatea de execuție ar trebui să fie la fel de mică și mecanică ca practică. Pentru interogări relaționale, poate fi un evaluator determinist. Pentru constrângeri, un SMT sau CSP rezolvator. Pentru comportamentul temporal, un verificator de model. Pentru proprietățile programului specifice căii, un executor simbolic. Pentru obligații de probă, un probator de motive plus un mic verificator de dovezi. Pentru un limbaj de circuit personalizat, poate fi un interpret restricționat cu operatori înregistrați și verificatori independenți. Componenta de acceptare nu trebuie să întrebe un LLM dacă rezultatul „arată corect”. Ar trebui să verifice un martor împotriva sursei și teoriei.

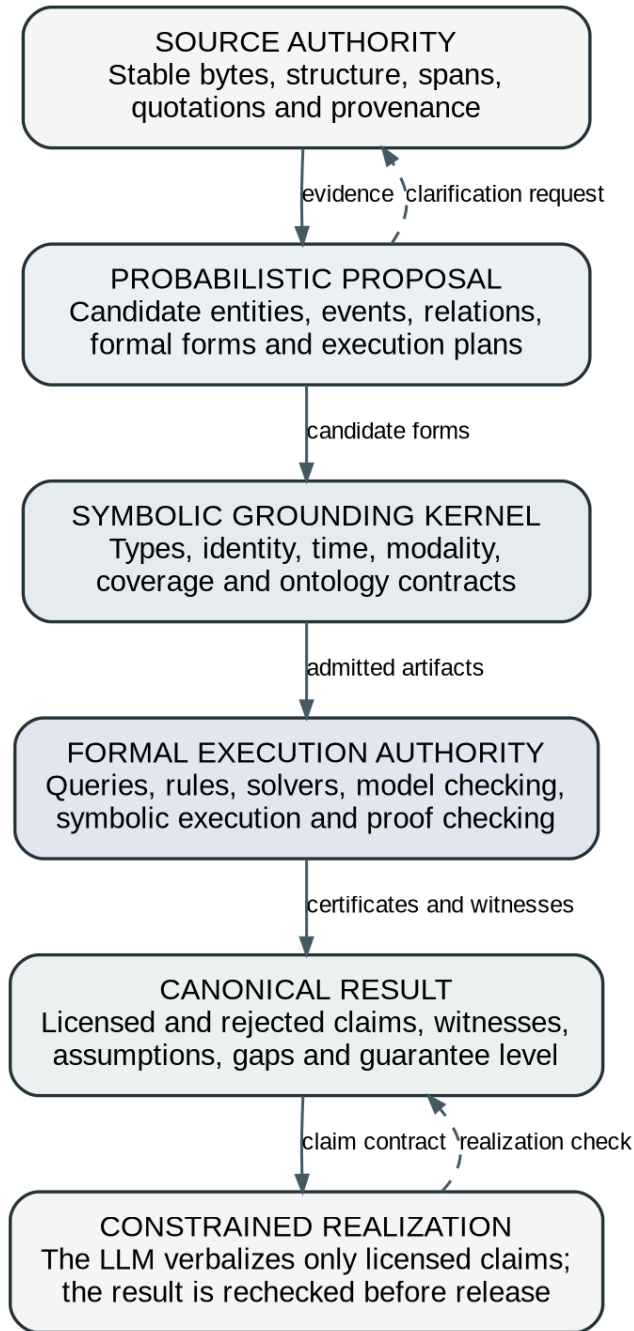


Figura 1. Încrederea limitei pentru un limbaj natural executabil. LLM propune formulare semantice, în timp ce punerea la cale la sol, executarea formală, acceptarea și licențierea finală a cererilor rămân în mod independent verificabile.

O modalitate utilă de clasificare a sistemelor este prin amplasarea oracolului final. Într-un sistem pur neuronal, LLM este oracolul. Într-un sistem augmentat de instrumente, un instrument calculează o valoare, dar LLM poate rămâne oracolul pentru a stabili dacă apelul instrumentului reprezintă întrebarea. Într-un sistem de rezolvare, rezolvatorul este oracolul pentru consecințele unei formalizări generate. Într-un sistem de dovezi, rezolvatorul și nucleul de împământare determină împreună rezultatul acceptat. Într-un sistem de acționare a dovezilor, chiar și ieșirea rezolvatorului poate fi verificată de un nucleu de încredere mai mic. Fiecare pas îngustează spațiul în care eroarea probabilistică poate schimba în tăcere verdictul.

Acest lucru nu implică faptul că fiecare aplicație are nevoie de asigurare teoretică-provert-nivel. Un asistent de scris poate utiliza circuite deterministe ușoare, în timp ce

un sistem de reglementare sau de siguranță poate necesita artefacte doveditoare formale. Principiul important este că garanția ar trebui să se potrivească cu mecanismul. O verificare determină a regexului poate certifica prezența unui model literal, nu absența unui concept. Un extractor de evenimente asistat de model poate susține un avertisment de continuitate propus, nu o relatare mecanică a identității narative. Un rezolvator poate stabili nesatisfacibilitatea constrângerilor, nu caracterul complet al traducerii în limba naturală. Rigor constă în mare parte în refuzul de a transfera încrederea peste aceste limite.

Prin urmare, cea mai puternică arhitectură este federativă. Componenta neurală nu este dizolvată într-o rețea simbolică, iar componenta simbolică nu este redusă la un regularizator diferențiat. Acestea rămân sisteme separate, cu o interfață tipă. LLM este responsabil pentru amplexarea, generarea de ipoteze, normalizarea semantică și recuperarea dintr-un limbaj neobișnuit. Nucleul formal este responsabil pentru admisibilitate, execuție, invariante, martori și reproductibilitate. Revizuirea din 2026 realizată de Chatzikyriakizid și Lappin numește sisteme conexe federative și observă că acestea arată adesea câștiguri puternice pe sarcini în care percepția poate fi separată de inferență. Autorii subliniază, de asemenea, că comparațiile rămân confuze de diferențele de sarcină. Argumentul arhitectural pentru federație este totuși independent de superioritatea de referință: modulele separate fac ca afirmațiile să fie audibile și permit unei componente să respingă o altă ZXC00000QXZ.

Ieșirea finală în limba naturală necesită propria limită. Odată ce un rezolvator returnează un rezultat, este tentant să furnizați urma unui LLM și să solicitați un răspuns lustruit. Fără constrângeri, modelul poate adăuga explicații de fond, poate să deducă motive sau să simplifice incertitudinea în certitudine. Prin urmare, un rezultat canonic ar trebui să enumere afirmațiile care pot fi verbalizate, statutul, dovezile, ipotezele și domeniul de aplicare al acestora. Randamentul poate alege formularea și organizarea, dar trebuie verificată împotriva structurii canonice. O metodă practică este realizarea la nivel de revendicare: modelul generează propoziții asociate identificatorilor de rezultate; un verificator asigură că fiecare propoziție este atribuită sau parafrazează o cerere permisă; materialul nesusținut este eliminat sau etichetat ca comentariu. În setările de înaltă asigurare, limba controlată prin șablit poate fi preferabilă prozei nerestricționate.

Limita de încredere propusă în această carte poate fi rezumată fără cod. LLMs poate propune formulare semantice, cartografieri ontologice, alegeri formaliste, reparații și explicații. Ei nu stabilesc dovezi, închid o căutare în lumea deschisă, certifică identitatea, nu decid satisfacția sau nu autorizează o cerere finală. Aceste funcții aparțin unor contracte simbolice explicite. Această separare păstrează valoarea unică a modelelor lingvistice, împiedicând în același timp ca fluenta lor să fie confundată cu dovada.

1.2 Decalajul formalizării: de ce un solver corect poate produce totuși răspunsul greșit

Slăbiciunea centrală a majorității sistemelor de raționament neuro-simbolic nu este motorul de inferență. SAT rezolvatoare, SMT rezolvatoare, motoare de baze de date, probe de teorii, verificali de modele și executori simbolici sunt adesea mult mai fiabili decât modelul lingvistic care le precede. Slăbiciunea constă în decalajul de formalizare: diferența dintre ceea ce înseamnă sursa în context și obiectul formal depus pentru executare. Un sistem poate fi perfect solid, de la intrarea formală la ieșirea formală, fiind în același timp nesănătos de la limbajul natural pentru a răspunde.

Luați în considerare o propoziție de politică simplă: „Un recenzor poate aproba un comunicat numai după finalizarea raportului de securitate, cu excepția cazului în care comandantul incidentului autorizează o excepție de urgență.” Un formalizator trebuie să păstreze cel puțin actorul, acțiunea, obiectul, condiția temporală, starea de finalizare, excepția, autoritatea și domeniul de aplicare al „numai după”. Trebuie să distingă permisiunea de obligație. Trebuie să decidă dacă excepția elimină doar condiția temporală sau autorizează aprobarea în general. Acesta trebuie să conecteze „raportul de securitate” la artefactul corect și „comandantul incidentului” la o identitate sau un rol. Dacă oricare dintre aceste alegeri este greșită, un rezolvator poate aplica în mod corect politica greșită.

Domeniul de aplicare cuantificator este o sursă clasică de eroare. "Fiecare cercetător a revizuit o lucrare" poate însemna că fiecare cercetător a revizuit cel puțin o lucrare, nu neapărat aceeași. „O lucrare a fost revizuită de fiecare cercetător” are o structură diferită. Negarea creează probleme similare: „nu fiecare probă trecută” nu înseamnă că nu a trecut nicio probă. Condiția, comparațiile, modalitatea, relațiile temporale, anafora și presupoziția creează distincții pe care traducerea la nivel de suprafață le poate șterge. Tradițiile semantice formale au dezvoltat mecanisme sofisticate pentru aceste fenomene, dar limbajul open-domeni adaugă cunoștințe mondiale, convenții de gen și obiective implicite pe care nicio reprezentare nu le surprinde complet.

Decalajul apare, de asemenea, în parsing semantic din dedoaie. Un sistem antrenat numai pe o întrebare și răspunsul final poate găsi un program care se întâmplă să producă răspunsul corect asupra mediului de antrenament. Astfel de programe false sunt o problemă cunoscută în păparea semantică slab supravegheată [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. Execuția nu distinge programul dorit de unul accidental dacă ambele produc aceeași dedare. Sistemele Text-to-SQL se confruntă cu același risc: o interogare poate returna rândurile potrivite din cauza valorilor actuale ale datelor în timp ce conțin o îmbinare sau filtrul greșit. Decodificarea ghidată de execuție elimină programele care se prăbușesc sau contrazic constrângerile intermediare, dar nu poate dovedi fidelitatea [Wang et al. 2018] semantică.

Decalajul de formalizare este cel mai bine descompus în obligații separate. Împământarea lexicală întreabă care predicate sau câmpuri corespund cuvintelor și frazelor. Împământarea menționării trimiterii întreabă care mențiuni denotă aceeași entitate. Terenurile structurale se referă la ce argumente, domenii și dependențe aparțin împreună. Împământarea temporară conectează frazele la instantanee, intervale, calendare și comenzi. Temeiirea modală distinge faptele, posibilitățile, obligațiile, permisiunile, intențiile, pretențiile raportate și contrafactualizările. Temelia ontologică determină dacă un concept aparține modelului de domeniu selectat. Împământarea evidentă leagă fiecare premisă de o întindere sursă, senzor,

rând de baze de date sau de presupunere declarată. Fiecare obligație poate eșua în mod independent.

Sistemul trebuie să decidă, de asemenea, cât de mult sens să formalizeze. O reprezentare semantică totală a prozei nerestricționate nu este nici practică, nici necesară pentru majoritatea sarcinilor. Regula care este executată creează cerere. Un circuit de terminologie poate avea nevoie doar de intervale exacte și de termeni normalizați. O regulă temporală de conformitate poate avea nevoie de evenimente, actori, date și excepții. O analiză cauzală poate necesita pretenții, intervenții, rezultate și incertitudine. Formarea determinată de cerere reduce costurile și limitează numărul de angajamente nesusținute. De asemenea, creează o problemă explicită de acoperire: dacă o regulă necesită un concept pe care producătorii disponibili nu îl pot materializa, alergarea trebuie să se oprească sau să se întoarcă necunoscută, mai degrabă decât să aproximeze în tăcere.

Ambiguitatea trebuie reprezentată ca alternative, nu s-au prăbușit imediat. Să presupunem că „Alex i-a spus lui Jordan că ar trebui să plece” apare într-o procedură. Pronumele se pot referi la Alex, Jordan sau la un grup, în funcție de convențiile de context și limbă. Un model poate clasifica candidații, dar o încredere ridicată nu este o dovadă. Dacă verdictul regulii din aval este același în temeiul tuturor legăturilor plauzibile, ambiguitatea este irelevantă și executarea poate continua. În cazul în care verdictul se schimbă, rezultatul ar trebui să expună dependența și să solicite clarificări sau context suplimentar. Acesta este un loc în care interpretarea abstractă devine utilă din punct de vedere conceptual: poate calcula concluziile asupra unui set de interpretări fără a enumera fiecare combinație în detaliu.

O cartografiere completă este adesea imposibilă, deoarece limbajul natural este open-world. Un document poate omite fapte care sunt adevărate, își pot asuma cunoștințe comune sau se poate baza pe practica instituțională. Raționamentul din lumea închisă tratează faptele neînregistrate ca fiind false; raționamentul open-world îi tratează ca fiind necunoscute. Ambele politici sunt adecvate în contexte diferite. Un inventar al bazei de date poate fi închis pentru un tabel și timp specificate. O colecție de e-mailuri preluate nu este, în general, o dovadă completă că nu a fost trimisă nicio notificare. Prin urmare, stratul de împănământ trebuie să atașeze acoperire la observații și căutări. Concluziile negative necesită un token de acoperire care prevede domeniul, revizuirea sursei, domeniul de aplicare, metoda, excluderile și politica de exhaustivitate.

Cunoștințele externe introduc alte complicații. Un LLM general conține informații extinse, dar neschimbate și potențial învechite. Un sistem simbolic nu poate trata memoria modelului ca fiind un fapt atemporal. Cunoștințele lumii ar trebui să intre prin pachetele sursă, datate, specifice jurisdicției și legate de conținut. Un pachet poate afirma că o regulă legală a fost eficientă într-o anumită jurisdicție într-un anumit interval sau că o constantă științifică a fost extrasă dintr-o sursă numită. Conflictele dintre haite ar trebui să rămână vizibile. Lumile fictive sau ipotetice trebuie să poată fi selectabile fără contaminare cu faptele din lumea actuală. Documentația nllAgent tratează corect astfel de pachete ca o graniță, mai degrabă decât să pretindă că memoria unui model de fundație oferă autoritate formală.

Fidelitatea formalizării poate fi testată, deși nu este universal dovedită. O metodă este verificarea bidirecțională: traduceți obiectul formal înapoi în limbaj controlat și comparați-l cu sursa. O alta este testarea contrastivă: generați modificări textuale minime care ar trebui să schimbe un element formal și să păstreze restul. O a treia este o comparație extinsă: cere formalizări concurente pentru a genera exemple distinctive sau rezultate de interogare. O a

patra este stabilitatea parafrazei: formulările semantic echivalente ar trebui să compileze artefacte echivalente sau dovedite înrudite. O a cincea este o explicație contraexplicabilă: atunci când rezolvatorul produce un martor, cartografiati fiecare dependență pentru a se extinde sursele și întrebați dacă scenariul este admisibil din punct de vedere lingvistic. O a șasea este o revizuire umană axată pe setul mic de alegeri care afectează rezultatul, mai degrabă decât întregul program formal.

Sistemele interactive, cum ar fi nl2spec, ilustrează valoarea reparării locale. În loc să ceară utilizatorului să înțeleagă sau să rescrie o întreagă formulă temporală, interfața asociază subformulele cu fragmente sursă, permițând corectarea [Cosler et al. 2023]-ului unei componente greșite. Feed-ul de rezolvare poate juca un rol similar. Logic-LM utilizează erori sintaxe și de execuție pentru a determina reformularea [Pan et al. 2023]. Sistemele mai avansate pot utiliza nuclee sau contraexemple nesatisfăcute, mai degrabă decât mesaje de eroare generice. Intuiția principală este că feedback-ul formal ar trebui să îngusteze întrebarea lingvistică.

Decalajul de formalizare plasează, de asemenea, limite în ceea ce privește revendicările de „sunere” de la un capăt la altul. Un sistem poate dovedi o teoremă a formei: dacă fiecare observație acceptată reprezintă cu exactitate sursa sa, iar ontologia și regulile sunt corecte, atunci verdictul urmează. Acest lucru este valoros. Acesta nu ar trebui să fie rescris ca „verdictul este dovedit din document” decât dacă obligațiile de observare și de punere în aplicare au fost, de asemenea, îndeplinite. Garanția ar trebui să numească ipotezele și necunoscutele sale. Un plic de sunet este mai onest decât o etichetă binară de încredere.

Prin urmare, această carte tratează formalizarea ca pe un proces de compilație cu mai multe artefacte intermediare, mai degrabă decât cu un singur prompt. Sursa este segmentată și ancorată. Observațiile candidaților sunt produse în cadrul schemelor. Sunt reprezentate alternative de identitate și de timp. Compatibilitatea cu ontologia este verificată. Se selectează un formalism țintă. Compilatorul produce un obiect executabil și o hartă sursă. Executorul produce un martor, o dovadă, un model sau un eșec. Rezultatul este reluat și redat. În fiecare etapă, o eroare are o categorie specifică și o posibilă reparație. Arhitectura nu elimină incertitudinea semantică; împiedică ascunderea incertitudinii în interiorul producției fluviale.

Urmează cea mai importantă concluzie de cercetare. Sistemele LLMZ susținute de Solver ar trebui evaluate separat pe baza formalizării și a execuției formale. Un scor mare de răspuns final poate ascunde programe false. Un scor scăzut poate fi cauzat de o bună formalizare cu un rezolvator inadecvat sau printr-o formalizare proastă cu un rezolvator perfect. Reperele au nevoie de aur sau structuri formale admisibile, de aliniere de dovezi și de perechile minime semantice, nu numai răspunsuri. Până când această separare va deveni standard, multe afirmații despre „raționarea formală cu LLMs” vor rămâne dificil de interpretat.

2. Genealogia intelectuală a limbajului executabil

2.1 Limbaje controlate, semantică formală, discurs și logică naturală

Ideea că limbajul care poate fi citit de om poate susține inferența mecanică precede modelele lingvistice mari cu zeci de ani. Lucrările anterioare au abordat problema din direcția opusă: mai degrabă decât să ceară unui model statistic să interpreteze proza nerestricționată, cercetătorii au proiectat limbi sau teorii semantice care au limitat ambiguitatea suficient pentru a sprijini traducerea în logică. Aceste tradiții sunt esențiale pentru că ele dezvăluie ce trebuie să păstreze un sistem de limbă executabilă chiar și atunci când un LLM efectuează analiza front-end.

Limbile naturale controlate sunt cel mai clar precedent de inginerie. Ei restricționează vocabularul, gramatica sau interpretarea, păstrând în același timp o suprafață care seamănă cu limbajul obișnuit. Attempto Controlled English este cel mai cunoscut exemplu. Propozițiile sunt traduse în structuri de reprezentare a discursului și apoi în logica formală, permițând interogări și deducție, rămânând în același timp lizibili la non-logicieni [Fuchs and Schwitter 1996; Fuchs et al. 2008]. Limbile controlate au fost utilizate pentru reprezentarea cunoștințelor, specificațiile tehnice, wiki-urile semantice, regulile de afaceri și cerințele. Sondajul lui Kuhn arată că familia este diversă: unele limbi sunt concepute mai degrabă pentru claritate umană decât pentru precizia formală, în timp ce altele fac comerț cu naturalitatea pentru semantica [Kuhn 2014] deterministă.

Lecția de durată nu este că limba nerestricționată ar trebui înlocuită cu un dialect controlat. Este faptul că un sistem executabil are nevoie de un strat de normalizare a cărei semantică este mai strictă decât proza obișnuită. Un LLM poate servi ca traducător dintr-o limbă deschisă într-o formă semantică controlată, dar forma controlată trebuie să aibă o gramatică și o interpretare independentă. Fără această limită, „limbajul natural controlat” devine doar un stil preferat de scriere și nu oferă nicio garanție de calcul.

Limele controlate expun, de asemenea, importanța feedback-ului autorului. Dacă o propoziție este în afara fragmentului controlat, un sistem tradițional o respinge sau produce o ambiguitate explicită. Interfețele moderne LLM tind să facă contrariul: selectează o citire și continuă o lectură plauzibilă. Respingerea este adesea tratată ca un defect de utilizare, dar într-un sistem de înaltă asigurare este o formă de integritate epistemică. O interfață de ultimă generație în limba executorie ar trebui să combine flexibilitatea parafrazei LLM cu onestitatea unui compilator în limba controlată. Poate propune mai multe lecturi normalizate, poate explica distincțiile și poate întreba numai când distincția schimbă rezultatul.

Semantica formală oferă o descendență teoretică mai profundă. Semantica modelului-teoretic reprezintă un sens în termeni de condiții în care expresiile sunt adevărate într-un model. Compoziționalitatea conectează interpretarea unei expresii complexe la semnificațiile părților sale și la combinația lor sintactică. Semantica în stil Montague, calculul lambda tips, cuantificatoarele generalizate și cadrele aferente oferă relatări precise despre domeniul de aplicare, structura argumentului și operatorii logici. Acestea demonstrează că sensul în limba naturală nu este reductibilă la o listă plată de triple. Cuantificatorii, negarea, verbele interminatoare, modalitatea și contextul pot schimba comportamentul logic al frazelor aparent simple.

Pentru limbajul natural executabil, semantica formală contribuie cu două cerințe. În primul rând, reprezentarea intermediară trebuie să păstreze structura operatorului, mai degrabă decât numai entitățile și relațiile. „Fiecare”, „câteva”, „nu” „nu” „nu” „nu” „must” și „înainte” nu sunt adnotări în jurul unei propuneri; ele determină spațiul modelelor. În al doilea rând, construcția semantică ar trebui să fie conștientă de tip. Un interval temporal, o persoană, o organizație, o propunere și o obligație nu pot fi înlocuite în mod liber doar pentru că etichetele lor de suprafață seamănă una cu cealaltă. Verificarea tipului este una dintre cele mai ieftine modalități de a respinge formalizările incoerente înainte de a invoca un rezolvator.

Teoria reprezentării discursului și abordările dinamice-semantice conexe abordează fenomenele pe care logica propoziției cu sentința le gestionează slab. Un discurs introduce referințe, le poartă în propoziții, actualizează contextul și creează constrângeri de accesibilitate pentru pronume și descrieri. Trecătoarele de boxer și DRS au operaționalizat aceste idei pentru analiza [Bos 2008; Liu et al. 2018] semantică cu acoperire largă. Banca de Regati Paralel și sarcinile de parsare comune DRS au făcut posibilă evaluarea reprezentanțelor sensului scopului induse de limbi [Abzianidze et al. 2019]. Aceste eforturi arată atât progrese, cât și dificultăți: legăturile variabile, atribuirea senzorială, domeniul de aplicare și discursul sunt substanțial mai grele decât extragerea superficială.

Un sistem executabil la nivel de document ar trebui să învețe din semantica dinamică fără a necesita un monolit DRS monolitic pentru fiecare sursă. Are nevoie de obiecte de discurs persistente, de ipoteze ale entității cu scop, de evenimente, de lumi și de actualizarea relațiilor. Trebuie să distingă o mențiune de o entitate și o legătură de identitate a candidatului de la una stabilită. Trebuie să păstreze contexte citate, ipotetice, negate și fictive. Un grafic plat de cunoștințe care îmbină șirurile egale într-un singur nod va face erori sistematice în documentele lungi. Numele se repetă, apar pseudonime, rolurile se schimbă și documentele descriu lumi reciproc inconsecvente.

Costul practic al parsingului semantic formal complet explică de ce multe sisteme s-au îndreptat spre reprezentări mai puțin adânci. Etichetarea semantică a rolului identifică structura de argumentare a predicatei. Reprezentarea sensului abstract reprezintă concepte și relații într-un grafic. Extragerea informațiilor identifică entitățile, evenimentele și relațiile. Aceste reprezentări sunt utile, dar semantica lor este adesea incompletă pentru execuție. Acestea pot omite domeniul de aplicare cuantificator, modalitatea, ancorarea temporală și diferența dintre conținutul afirmat și cel raportat. Un timp de funcționare în limba executabilă ar trebui să trateze astfel de structuri ca observații care poartă dovezi, mai degrabă decât ca o formă logică completă. Un grafic poate susține multe reguli locale, lăsând în același timp aspectele nerezolvate explicate.

Logica naturală oferă un alt compromis important. În loc să traducă propozițiile într-un limbaj formal nerestricționat, sistemele naturale-logică raționează transformările structurate în apropierea formei de suprafață. MacCartney și Manning au dezvoltat relații de inferență bazate pe monotonică, implicare lexică și proiectivitate [MacCartney and Manning 2007, 2009]. NaturalLI a folosit ulterior operatori naturali-logici pentru o deducție eficientă față de textul [Angeli and Manning 2014]. LangPro combină o gramatică formală, cunoștințe lexice și o dovadă de teoretă pentru inferența în limba naturală [Abzianidze 2017].

Logica naturală este atractivă, deoarece multe întrebări de documente presupun că implică relații care pot fi capturate fără a construi un model mondial mare. Înlocuirea „câinelui” cu „animal” este valabilă într-un context orientat spre ansamblu, dar nu neapărat sub negare sau

restricții universale. Aceste calcule sunt mai transparente decât un scor de similitudine încorporare. Acestea pot sprijini verificările de contradicție, raționamentul taxonomic și inferența controlată. Cu toate acestea, logica naturală nu este un executor universal. Se ocupă bine de anumite monoconacții și relații lexice, dar nu exprimă în mod natural proceduri arbitrare, aritmetice, planificare sau comportament temporal complex. Într-o arhitectură multi-formalism, ar trebui să fie un motor printre mai multe.

Tradițiile semantice clarifică și ele de ce „împământarea” are mai multe niveluri. Semantica formală poate specifica modul în care o expresie contribuie la condițiile adevărului, dar nu determină automat ce obiect din lumea reală denotă un nume sau dacă o sursă este credibilă. DRS poate reprezenta un referent al discursului, dar rezolvarea identității rămâne o sarcină empirică. Logica naturală poate propaga implicarea lexicală, dar relația lexică poate fi dependentă de domeniu. Prin urmare, execuția necesită un parteneriat între structura lingvistică, ontologia domeniului și gestionarea dovezilor.

O arhitectură utilă poate distinge cel puțin trei produse semantice. Primul este un model sursă care conține structură exactă, întinderi și citate. Al doilea este un strat de interpretare care conține mențiuni, evenimente, relații, modalități și alternative. A treia este o teorie executabilă care utilizează interpretările selectate pentru a răspunde la o anumită întrebare. Tehnicile semantice formale informează în primul rând al doilea strat. Limbile controlate și theorem provers conectează al doilea și al treilea. Proveniența și acoperirea se conectează la ambele înapoi la primul.

Literatura anterioară avertizează, de asemenea, împotriva presupunerii că o reprezentare poate servi fiecare sarcină. DRS este expresiv pentru discurs, dar scump de produs și raționament peste. Logica de prim ordin este bine înțeleasă, dar incomodă pentru fenomenele temporale, probabilistice sau fezabile. Logisticile de descriere oferă raționament ontologic de dorit, dar restricționează expresivitatea. Logica naturală rămâne aproape de limbaj, dar este specializată. Limba engleză controlată îmbunătățește predictibilitatea prin restricționarea inputului. Concluzia corectă nu este de a alege un câștigător, ci de a păstra suficiente informații semantice pentru a reduce o sarcină în formalismul care se potrivește.

LLMs schimbă economia acestui design. Ei pot genera sortimente candidat, pot rescrie limbajul necontrolat în forme controlate, pot sugera cartografieri de ontologie și pot explica alternativele formale utilizatorilor. Ei fac practic încercarea de interpretare semantică pe documente pe care gramaticile scrise de mână le-ar respinge. Acestea nu elimină necesitatea distincțiilor semantice dezvoltate prin lucrări anterioare. Dimpotrivă, aceste distincții devin contractele prin care LLM se judecă propunerile.

O greșală recurentă în sistemele contemporane este de a trata un magazin triplu generat de LLM ca o reprezentare semantică completă. Triplele sunt convenabile pentru regăsire și interogări grafice, dar nu codifică singuri domeniul de aplicare, proveniența, modalitatea, timpul, negarea, alternativele de identitate sau acoperirea. Un strat semantic executabil ar trebui să fie mai bogat, dar totuși finit și operațional. Poate stoca o afirmație ca obiect tipod cu participanții, contextul temporal, polaritatea, modalitatea, lumea, dovezile, producătorul și statutul. Poate reprezenta legături alternative fără a alege unul. Poate declara ce dimensiuni sunt absente. Acest lucru este mai puțin elegant decât o singură formă logică, dar este mai fidel modului în care sistemele reale dobândesc sensul.

Cea mai profundă contribuție a liniei pre-LLM este, prin urmare, disciplina conceptuală. Lizabilitatea umană nu implică o determinare semantică. Încetarea formală se realizează prin

restricționare, tastare, context explicit și respingerea construcțiilor nesuținute. Sensul documentului este dinamic și scop. Inferența în apropierea limbajului de suprafață este posibilă, dar specializată. Reprezentările formale multiple sunt legitime pentru că răspund la întrebări diferite. Aceste lecții definesc arhitectura pe care un compilator cu baza LLM trebuie să se recupereze mai degrabă decât să ocolească.

2.2 Analiză semantică, interogări executabile, inducție de programe și sisteme neuro-simbolice timpurii

Parsingul semantic transformă în limbaj natural în obiecte formale care pot fi executate împotriva unei baze de date, a bazei de cunoștințe, a mediului sau a unui interpret. Este cel mai apropiat predecesor istoric al conduitei de limbă executabilă propusă aici. Domeniul include răspunsuri la întrebări cu privire la tabele și grafice de cunoștințe, text-to-SQL, instrucțiuni care urmează, sinteză de exprimare regulată, interpretarea comenzii robotice și generarea programului. Succesele sale demonstrează puterea execuției externe; eșecurile sale dezvăluie de ce executarea singură nu stabilește înțelegerea.

Pășitorii semantici timpurii s-au bazat pe gramatici, lexiconi și forme logice supravegheate. Acestea ar putea produce reprezentări transparente, dar au necesitat adnotări costisitoare și adesea transferate slab în domenii. Supravegherea slabă a redus sarcina de adnotare prin instruirea de la denotări: o întrebare este asociată cu un răspuns, iar sistemul caută un program care să-l cedeze. Această abordare a permis seturi de date mai largi, dar a creat problema programului de conspirație. Multe programe pot returna accidental același răspuns, în special în mese mici sau baze de cunoștințe. Semnalul de antrenament recompensează apoi comportamentul fără a identifica semantica propusă [Pasupat and Liang 2016].

Problema este structurală, nu doar statistică. Să presupunem că răspunsul la „Ce oraș a găzduit evenimentul din 2012?” este Londra. Se prevede un program de selectare a rândului cu anul 2012 este destinat. Un program care selectează primul rând poate întoarce, de asemenea, Londra în masa de antrenament. Mai multe date pot reduce corelațiile accidentale, dar nu pot garanta că programul învățat reflectă limba. Au fost dezvoltate exemple abstracte, modele de aliniere, constrângeri de tip și filtrare pe bază de execuție pentru a suprima candidații [Goldman et al. 2018; Lee et al. 2023]. Aceste tehnici sunt importante și pentru LLMZ formalizare. Un rezultat de rezolvare ar trebui tratat ca un test al unei formalizări, nu ca certificat semantic.

Mașinile simbolice nevrute combină generarea de programe neuronale cu un interpret simbolic Lisp [Liang et al. 2017]. Componenta neuronală propune programe, în timp ce executorul respinge candidații nevali și returnează emoțiunile. Mou și colegii săi au cuplat în mod explicit execuția neuronală distribuită cu execuție simbolică pentru interogările în limba naturală, raportarea îmbunătățirilor în acuratețe, eficiență și interpretare [Mou et al. 2017]. Aceste sisteme au stabilit un model durabil: modelele neuronale se ocupă de variabilitatea limbajului; interpretele simbolice impun sens operațional.

Text-to-SQL a făcut modelul important din punct de vedere comercial. O întrebare a utilizatorului este alcătuită într-o interogare a cărei execuție returnează datele. Decodificarea ghidată de execuție poate rula candidați parțiali sau finali SQL și le elimină pe cele care produc erori sau rezultate [Wang et al. 2018] imposibile. Schema care leagă numele și relațiile de constrângere. Modernul LLMs îmbunătățește substanțial generarea, dar generalizarea inter-domeniu, întrebările ambigue, convențiile specifice bazei de date și cererile fără răspuns rămân dificile. Lecția cheie pentru limbajul natural executabil este că mediul formal în sine furnizează constrângeri utile. Tipurile de coloane, cheile străine și erorile de interogare oferă feedback pe care îl lipsește modelarea pură a limbajului.

Cu toate acestea, textul în SQL arată, de asemenea, limitele unei singure metrice de precizie finală. Potrivirea exactă a coardei poate respinge interogările echivalente. Acuratețea execuției poate accepta interogări greșite din punct de vedere semantic care se întâmplă să

returneze același rezultat. O evaluare riguroasă are nevoie de echivalență în diferite cazuri de bază sau contraexemple construite cu atenție. Acest lucru este direct analog cu evaluarea formalizării regulilor: un program formal generat ar trebui să fie testat pe scenarii de distingere, nu numai pe un singur răspuns observat.

Sinteza programului din limbajul natural generalizează ideea. Sistemele generează expresii regulate, fragmente de cod, transformări de date sau programe specifice domeniului mic. Exemplele pot constrânge spațiul candidat, iar sistemele de tip pot elimina programele fără sens. Generarea neuronală de expresii regulate a arătat că cartografierea limbajului-la-program poate funcționa chiar și cu cunoștințe limitate în domeniu, ilustrând, de asemenea, fragilitatea atunci când descrierile sunt subspecificate [Locascio et al. 2018]. Programul LLMs a mutat mai târziu calculul în afara modelului, dar problema de sinteză fundamentală a rămas: modelul trebuie să aleagă un program ale cărui semantica se potrivește cu cererea.

Prin urmare, un compilator robust ar trebui să caute mai multe forme de dovezi. Descrierile în limbi naturale constrâng intenția. Exemplele de intrare constrâng comportamentul prelungit. Tipurile de constrângeri operațiuni admisibile. Schemele de domeniu constrâng vocabularul. Testează cazurile de margine. Un program generat care satisface toate aceste constrângeri este mai credibil decât unul care nu face decât să execute. Acest principiu multi-dovezi poate fi aplicat la normele documentate: autoritatea textuală, exemplele pozitive și negative, constatările așteptate și mutațiile semantice ar trebui să modeleze un circuit înainte de publicare.

Programarea logică oferă o altă țintă executabilă. Regulile Datalog calculează consecințele asupra unui punct de fixare și sprijină evaluarea incrementală. Prologul și limbile conexe reprezintă relațiile și căutările. Interpretarea abstractă a fost folosită de mult timp pentru a analiza programele logice, arătând o punte conceptuală între semantica declarativă și analiza programului. Pentru aplicațiile în limbajul natural, programarea logică este potrivită pentru închiderea monotonilor: relații tranzitorii, taxonomii, depanare a dependenței și reguli de politică fără neîndeplinirea obligațiilor complexe. Este mai puțin adecvată atunci când raționamentul depinde în mare măsură de incertitudine, de surse inconsecvente sau de excepții non-monotonice, cu excepția cazului în care este prelungit cu mecanisme explicite.

Tabelele de decizie și sistemele de conducere de producție sunt adesea trecute cu vederea în sondajele academice, dar sunt extrem de relevante. Multe politici nu sunt teorii logice profunde; ele sunt combinații finite de condiții și rezultate. Un tabel de decizie face ca acoperirea, conflictele și combinațiile lipsă să fie vizibile. Poate fi compilat în predicate executabile și testate sistematic. Limbajul natural poate fi folosit pentru a fi scris sau pentru a explica tabelul, în timp ce tabelul rămâne semantica autoritară. Pentru editarea documentelor întreprinderii, tabelele de decizie pot fi mai sigure și mai ușor de validat decât logica arbitrară de prim ordin.

Instrucțiunile temporare și procedurale necesită obiective mai bogate. Mașinile de stat finite, arborii de comportament, limbajele de planificare și logicile temporale reprezintă acțiuni și condiții ordonate. Robotica a folosit mult timp parsing-uri lingvistice care compun primitive executabile. Sistemele recente de manipulare neuro-simbolică continuă acest model: un parser de limbaj cartografiază o instrucțiune către un program ale cărui componente pot include percepția neuronală și numărarea simbolică, raționamentul spațial sau controlul acțiunii. Succesul unor astfel de sisteme depinde de o bibliotecă de abilități restricționate și de un mediu la sol, nu de acoperirea zXC00000QXZ nerestricționată.

Programarea logica probabilista si sistemele neuro-simbolice diferentiabile incearca o integrare mai stricta. DeepProbLog, Logic Tensor Networks și sistemele conexe combină predicății învățați cu structura logică [Manhaeve et al. 2018; Badreddine et al. 2022]. Ele sunt valoroase atunci când incertitudinea și învățarea sunt intrinseci și pot injecta constrângeri în antrenament. Cu toate acestea, diferentabilitatea nu este echivalentă cu asigurarea formală. Valorile adevărului neclare și sancțiunile soft pot ghida modelele fără a produce obligații evidente. Pentru limbajul natural executabil, astfel de sisteme pot estima sau clasifica candidații semantici, dar o limită de acceptare de mare asistare beneficiază încă de un verificator simbolic separat.

Revizuirea neuro-simbolică NLP din 2026 distinge sistemele injectoare, care compilează informații simbolice în modele neuronale, de sistemele federative, care păstrează componentele autonome. Acesta raportează că sistemele injectoarece domină literatura de specialitate, dar adesea arată câștiguri modeste pe sarcini largi NLU, în timp ce sistemele federative prezintă câștiguri mai mari pe sarcini de raționament. Autorii avertizează în mod explicit că comparația este confuză de diferite criterii de referință și că generalizarea în afara domeniului rămân o problemă [Chatzikyriakidis and Lappin 2026] serioasă. Interpretarea echilibrată este că federația este atractivă din punct de vedere arhitectural pentru auditabilitate și descompunere a sarcinilor, dar încă nedovedită empiric ca o strategie universală de scalare.

Linia de parsiere semantică clarifică, de asemenea, rolul unui limbaj intermediar. O reprezentare trebuie să fie suficient de expresivă pentru a capta un sens relevant pentru sarcini, dar suficient de constrâns pentru a se compila cu fiabil. Dacă este prea aproape de text de suprafață, execuția rămâne implicită. Dacă este prea general, traducerea și complexitatea rezolvatoare devin imposibil de gestionat. Beiser și Penz descriu acest lucru ca o problemă de limbă intermediară în sistemele neuro-simbolice contemporane: alegerea unui formalism se poate mișca mai degrabă decât să rezolve dificultatea [Beiser and Penz 2025]. O arhitectură multi-formalism reduce presiunea asupra oricărei limbi, dar introduce o nouă cerință pentru rutarea tipdurilor și consistența întreforma.

Prin urmare, un pivot semantic util nu ar trebui să pretindă că este logica finală. Ar trebui să păstreze dovezile și distincțiile neutre din punct de vedere al sarcinii – entități, mențiuni, evenimente, relații, timp, modalitate, cantități, lumi, proveniență, alternative și acoperire – permițând în același timp coborârea în executori specializați. Unele întrebări pot fi răspunse direct asupra pivotului folosind operațiunile relaționale. Altele necesită o formulă temporală compilată, un sistem de constrângere sau o mașină de stare. Pivotul este valoros, deoarece separă înțelegerea limbajului de orice rezolvator special, păstrând în același timp suficiente informații pentru executarea la sol.

Programul de cercetare pre-LLM a obținut un succes substanțial, dar legat de domeniu. Acesta a arătat că limbajul natural poate fi compilat în programe executabile, că interpreții simbolici îmbunătățesc sistematicitatea, că tipurile și schemele reduc erorile și că supravegherea slabă creează scurtături semantice. De asemenea, a arătat că acoperirea largă și semantica exactă sunt în tensiune. LLMs îmbunătățește substanțial acoperirea și generarea de candidați, dar moștenesc fiecare problemă structurală identificată prin parsare semantică. Sinteza corectă nu este de a arunca mașina anterioară. Este pentru a plasa LLMs în fața și în jurul ei, cu interfețe formale care fac ca lecțiile vechi să fie executorii la o nouă scară.

3. Interpretare abstractă și execuție simbolică pentru limbaj

3.1 Interpretarea abstractă ca semantică a incertitudinii delimitate

Interpretarea abstractă a fost dezvoltată ca o teorie generală pentru calcularea aproximărilor sigure ale comportamentului [Cousot and Cousot 1977] programului. Un program poate avea infinit de multe intrări și stări, ceea ce face imposibilă execuția exhaustivă. În loc să enumere fiecare stat, o analiză reprezintă seturi de stări prin proprietăți abstracte, cum ar fi intervale, semne, posibile tipuri de obiecte, ajungerea la definiții sau alias relațiile. Operațiunile abstracte sunt concepute astfel încât fiecare comportament concret relevant pentru proprietate să fie reprezentat. Analiza poate include comportamente imposibile și, prin urmare, poate raporta alarme false, dar o supra-apropiere sonoră nu omite în tăcere un comportament pe care îl permite semantica de beton.

Ideea operațională este simplă. O execuție concretă urmărește valorile exacte. O analiză a intervalului ar putea urmări că o variabilă se află între zero și zece. Când programul adaugă cinci, operațiunea de transfer abstract produce un interval între cinci și cincisprezece. La o ramură, statele sunt rafinate în funcție de această afecțiune. La o fuziune, se alătură informații. Pentru bucle și procesele recursive, analiza caută un punct de fixare. Extinderea poate forța convergența sacrificând precizia, în timp ce îngustarea poate recupera unele detalii după aceea. Cadrul matematic oferă un vocabular disciplinat pentru precizie, soliditate, convergență și rafinare.

Aplicarea cadrului la limbă necesită o semantică concretă. Obiectul care este aproximat nu poate fi „semnificativ” într-un sens nedefinit. Pentru o sarcină limitată, domeniul de beton poate fi setul de structuri semantice admisibile în cadrul unei gramatici specificate, ontologie, politica de bază, revizuirea surselor și teoria fundalului. Fiecare structură conține posibile entități, misiuni de evenimente, relații temporale, modalități și legături sursă. Ambiguitatea creează mai multe structuri. Informațiile lipsă lasă dimensiunile neconstrânse. Surse contradictorii pot genera lumi alternative, mai degrabă decât o uniune inconsecventă.

O stare semantică abstractă rezumă acel set. Acesta poate consemna că o pronume se poate referi la oricare dintre cele două persoane, că un eveniment are loc într-un interval de timp, că o obligație este prezentă sub o interpretare și absentă sub alta sau că o cantitate se află într-un interval. Un fapt trebuie să țină în fiecare interpretare reprezentată. Un fapt poate fi reținut în cel puțin unul. Un fapt nu se poate ține în nici unul. Necunoscut indică faptul că sistemul nu are reprezentarea, acoperirea sau resursele de calcul necesare pentru a decide. Această distincție în patru este mai informativă decât o singură probabilitate și mai onestă decât forțarea fiecărei întrebări în adevărat sau fals.

Luați în considerare un document de politică care să menționeze: „Operatorul trimite notificarea după validare. Dacă modul de urgență este activ, supraveghetorul poate renunța la perioada de așteptare. Să presupunem că nu este clar dacă „perioada de așteptare” se referă la perioada anterioară validării sau la o perioadă ulterioară. O singură pars face o alegere fragilă. O stare abstractă poate păstra două modele temporale. Dacă ambele implică faptul că notificarea trebuie să urmeze validarea, această concluzie este obligatorie. Dacă numai un singur permis notificarea imediată, este posibilă trimiterea imediată. O regulă de conformitate

poate identifica ce verdict depinde de fraza ambiguă și poate solicita clarificări numai atunci când este necesar.

Beneficiul principal nu este că interpretarea abstractă înțelege în mod magic limbajul. Este faptul că oferă o execuție principială semantică odată ce a fost construit un set de lecturi delimitat. Acesta spune sistemului cum să agrege alternative fără a confunda posibilitatea cu necesitatea. De asemenea, suportă domenii modulare. Identitatea, timpul, cantitatea, modalitatea, proveniența și statutul normativ pot avea fiecare propria absctiune, apoi pot fi combinate prin produse reduse care fac schimb de informații acolo unde este util.

Un domeniu de identitate ar putea reprezenta clasele de echivalență a candidaților cu aceleași relații explicite, diferite și nerezolvate. Nu trebuie să îmbine entitățile doar pentru că numele se potrivesc. Un domeniu temporal ar putea utiliza ordine parțiale, intervale sau constrângeri de diferență. Un domeniu al modalității ar putea distinge conținutul afirmat, raportat, intenționat, permis, cerut, interzis, ipotetic și contrafacual. Un domeniu de cantitate ar putea folosi valori exacte, intervale, unități și compatibilitate dimensională. Un domeniu de proveniență ar putea ordona stări de probă, cum ar fi extrase, propuse, asumate, derivate, verificate și confirmate de om, fără a permite încredere să le șteargă originea. Un domeniu de acoperire ar putea urmări dacă o căutare este suficient de completă pentru a sprijini cererile de absență.

Domeniile trebuie să fie alese de către proprietatea care este verificată. Un felinar de terminologie nu are nevoie de o rețea nedobită de deontică. O regulă temporală nu are nevoie de semantică lexicală completă dacă evenimentele și datele sunt deja materializate. Această selecție bazată pe activități reduce complexitatea și oglindește analiza statică determinată de cerere. Într-o perioadă de funcționare cu documentar lung, circuitele sau regulile declară tipurile de observare și dimensiunile abstracte pe care le necesită. Compilatorul materializează numai aceste dimensiuni și respinge rularea atunci când nu poate fi furnizat un domeniu critic.

Funcțiile de transfer sunt cea mai dificilă parte. În analiza obișnuită a programului, semantica unei sarcini sau a unei operațiuni aritmetice este fixată de limbajul de programare. În analiza în limba naturală, operațiunea care actualizează starea semantică poate fi propusă de un LLM. O sentință ar putea introduce un eveniment, ar putea modifica o stare anterioară, ar putea stabili o relație temporală, ar putea crea o excepție sau ar putea raporta convingerea altcuiva. Dacă un traducător învățat devine direct funcția de transfer, se pierde soliditatea clasică.

Un design de apărare separă generația de candidați de la transferul admis. LLM propune un set finit de actualizări semantice de tip tipare cu intervale de calitate și alternative sursă. Un materializator determinist validează faptul că există dovezile citate, câmpurile satisfac o schemă, unitățile și tipurile sunt compatibile, referințele indică obiecte admisibile și nicio stare nu este modernizată în tăcere. Interpretul abstract aplică apoi funcțiile de transfer înregistrate la înregistrările admise. Analiza rezultată este solidă în raport cu setul de candidați admiși, nu neapărat în raport cu fiecare interpretare pe care un om ar putea să o recunoască. Această condiționalitate trebuie să fie precizată în mod explicit.

Există mai multe modalități de a consolida presupunerea de stabilire a candidaților. Mai multe solicitări sau modele independente pot propune alternative. Generatoarele bazate pe Gramatică sau ontologie pot asigura acoperirea pentru fragmente controlate. Variantele de parafrază pot testa stabilitatea. Exemple contrastante pot dezvălui interpretări lipsă. Revizuirea umană se poate concentra pe ambiguitățile relevante pentru rezultate. Un compilator poate genera în mod deliberat o gramatică delimitată și poate lăsa constrângerile ulterioare să elimine

candidații imposibili. Fiecare metodă tranzacționează costul încrederii că setul conține toate citirile plauzibile din punct de vedere material.

Interpretarea abstractă este relevantă și pentru înmulțirea regulilor la scară de documente. Multe probleme de desfacere și de conformitate seamănă cu analiza fluxului de date. Faptele sunt generate în locațiile sursă, propagate prin domenii, unite în secțiuni și invalidate prin declarații ulterioare. O definiție poate fi disponibilă după declarația sa, dar nu înainte. Un obiect poate schimba statul într-o narațiune. O cerință se poate aplica în cadrul unei jurisdicii sau versiunii. O cerere poate obține probe mai târziu. Un motor incremental de fixare poate calcula consecințele și poate actualiza regiunile afectate numai atunci când documentul se schimbă.

Această perspectivă evită reluarea unui LLM general peste un întreg document după fiecare editare. Artefactele structurale și semantice stabile pot fi încasate prin digerarea conținutului. Un paragraf schimbat invalidează observațiile locale și stările abstracte din aval care depind de ele. Graficul de analiză identifică care circuite au nevoie de reluare. Acest lucru este analog cu cadrele de compilare incrementală și de analiză statică. De asemenea, face explicații precise: sistemul poate arăta ce schimbare sursă a modificat ce invariant abstract și de ce.

Lărgirea și pierderea preciziei au analogi semantici. O narațiune lungă poate conține sute de posibile identități ale obiectelor sau relații temporale. Păstrarea oricărei combinații este nepractică. Sistemul poate rezuma „unul dintre aceste obiecte”, „un moment în acest interval” sau „una dintre aceste jurisdicții”. O astfel de lărgire poate introduce avertismente false. Îngustarea sau rafinamentul local poate recupera precizia în jurul unei încălcări suspectate. Obiectivul de design nu este de a evita abstractizarea, ci de a face pierderea de precizie vizibilă și reversibilă.

Literatura de sinteză NLP creată pentru robuste certificate utilizează idei de interpretare abstractă într-un mod diferit. Jia și colegii au folosit propagarea cu destinația intervalului pentru a antrena modele ale căror predicții sunt certificate împotriva fiecărei substituții de cuvinte într-o familie predefinită, raportând 75% acuratețe adversarului pe IMDB și SNLI sub modelul [Jia et al. 2019] lor de amenințare. Lucrările ulterioare au folosit netezirea randomizată, încorporările conștiente de robustețe și domeniile specializate Transformer pentru a îmbunătăți limitele [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026] certificate. Acestea sunt garanții reale, dar domeniul beton este un set de intrări de model perturbate, nu un set de interpretări semantice. Familia de transformare este definită extern și se presupune că păstrează eticheta.

Această distincție contează. Certificarea faptului că predicția unui clasificator este invariantă sub toate substituțiile într-o listă sinonimă nu dovedește că fiecare substituție păstrează sensul propoziției în context. Nu dovedește că clasificatorul are dreptate la propoziția inițială. Se certifică robustețea în raport cu un set formal de perturbare. Această literatură este încă extrem de relevantă, deoarece demonstrează modul în care domeniile abstracte pot exala peste intrările de limbaj combinațional și modul în care precizia de verificare depinde de reprezentarea corelațiilor. De asemenea, oferă un avertisment metodologic: cea mai puternică afirmație formală este întotdeauna în raport cu setul de beton definit efectiv.

O direcție de cercetare promițătoare este de a conecta cele două utilizări. Analiza neuronală certificată ar putea lega comportamentul modelului de propunere semantică sub parafraze autonome controlate, în timp ce interpretarea abstractă semantică ar putea lega consecințele interpretărilor alternative fundamentate. Primul întreabă dacă extracția unui model se schimbă în cadrul transformărilor permise ale suprafeței. Acesta din urmă întreabă dacă o concluzie la nivel de document se schimbă în structurile semantice admisibile. Împreună, ei ar putea

identifica dacă incertitudinea provine din instabilitatea modelului sau în ambiguitatea semantică autentică.

LLMs poate ajuta, de asemenea, la construirea interpreților abstracți, dar numai sub acceptare mecanică. SAIL demonstrează acest principiu prin utilizarea LLMs pentru a căuta transformatoare abstracte, în timp ce aplică constrângerile sintactice și semantice și optimizarea unui cost care măsoară nesănatatea. Transformatoarele acceptate sunt solide la nivel global, iar sistemul poate sintetiza transformatoarele precise pentru operatorii neliniari care nu au fost disponibile anterior [Gu et al. 2026]. Lecția mai largă este arhitecturală: modelele generative pot explora un spațiu imens de design dacă un mic validator formal controlează admiterea.

A cere direct unui LLM să efectueze interpretarea abstractă este mai puțin fiabilă. Experimentele pe benchmark-urile de analiză a programului arată că modelele pot urma stilurile de interpretare abstractă, dar pot face erori logice pe structurile complexe și în statele halucinogene [Mitchell et al. 2025]. Acest rezultat ar trebui să tempereze propunerile în care un model narează un invariant și narațiunea este acceptată ca analiză. Modelul este valoros pentru a sugera invariante, partiții de domeniu sau predicate de rafinament. Verificările de calcul și de soliditate ar trebui să rămână simbolice.

Pentru limbajul natural executabil, interpretarea abstractă nu este, prin urmare, nici o metaforă, nici o soluție completă. Este o teorie pentru aproximarea disciplinată după ce sistemul a definit ceea ce contează ca o posibilă interpretare. Suportă semantica may/trebuie, propagarea incrementală, compoziția domeniului, raționamentul conservatoare al absenței și rafinamentul vizat. Garanția sa este condiționată de acoperirea semantică. Această limitare nu este fatală; este punctul exact în care interpretarea asistată de model trebuie evaluată și guvernată.

3.2 Execuție simbolică, rezolvarea constrângerilor, verificarea modelelor și rafinare semantică

Execuția simbolică clasică execută un program cu intrări simbolice, mai degrabă decât valori specifice [King 1976]. O stare simbolică conține o locație de control, expresii simbolice pentru variabilele programului și o condiție de cale care descrie ce intrări ajung la această stare. La o ramură, furculițe de execuție. O cale adaugă condiția ramurii; cealaltă adaugă negarea sa. Un rezolvator de constrângeri determină dacă condițiile sunt satisfăcătoare și pot produce intrări concrete care să realizeze o cale fezabilă. Metoda este puternică pentru că produce martori: nu doar un avertisment că ar putea exista o eroare, ci o intrare și o cale care o demonstrează.

Dificultatea este explozia căii. Ramurile se înmulțesc, buclele și recursul creează căi nemărginite, teoriile complexe rezolvatoare de provocări, iar bibliotecile externe sunt dificil de modelat. Sondajele de execuție simbolică descriu un spațiu mare de proiectare a strategiilor de căutare, fuziunea statului, rezumatele compoziționale, execuția conciliană, modelele de mediu și optimizările de rezolvare ZXC200QXZ. Niciun motor nu explorează toate comportamentele software-ului real. Prin urmare, pretențiile de certificare simbolică includ limite, ipoteze de modelare și limitări de acoperire.

Limbajul natural se conectează la execuția simbolică în mai multe moduri distincte. Cel mai simplu este limbajul-la-cod: un LLM generează un program, iar un executor simbolic convențional analizează acel program. Un altul este limba-specificație: modelul generează o proprietate sau o constrângere, care este verificată în conformitate cu codul existent. O a treia este executarea semantică: un document este compilat într-un program de mașină de stat sau de reguli ale cărui interpretări alternative devin alegeri simbolice. Un al patrulea este LLM explorarea asistată: un model prioritizează căile, propune rezumate sau motive legate de constrângerile pe care rezolvatorii convenționali nu le pot exprima cu ușurință.

Terminologia trebuie să rămână precisă. Executarea SQL, Python sau a unei forme logice generate de o întrebare este executarea formală, dar nu este neapărat o execuție simbolică în sensul sensibil la cale. Diferența contează pentru că execuția simbolică oferă condiții de cale și martori concreți, în timp ce execuția obișnuită oferă un rezultat pentru o stare concretă. Ambele sunt utile. Un studiu de sinteză care îi grupează împreună fără distincție face ca garanțiile să pară mai puternice decât sunt.

SymGPT oferă unul dintre cele mai clare exemple de limbaj-specificație. Sistemul studiază 132 de reguli în limba naturală de la trei ERC standarde, le traduce cu un [Hao et al. 2025] într-o gramatică restricționată, sintetizează constrângerile care descriu încălcările și folosește executarea simbolică pentru a căuta contracte Ethereum. Autorii raportează 5.783 de încălcări în 4.000 de contracte, inclusiv 1.375 cu căi de atac explicite pentru furt, și raportează performanțe mai puternice decât șase tehnici automate și un serviciu de audit [Xia et al. 2026] de securitate. Puterea designului constă în gramatica îngustă, în sinteza sistematică a constrângerilor și în căile programului de beton. Riscul său semantic rămas constă în faptul dacă fiecare ERC regulă a fost tradusă cu fidelitate.

Acest model generalizează. O politică poate fi compilată în condiții prealabile, tranziții interzise și excepții. O procedură poate fi compilată în state și acțiuni. O regulă de continuitate narativă poate modela obiecte și evenimente, apoi caută o cale în care un obiect este folosit după ce a fost lăsat în urmă fără recuperare. Un protocol clinic poate modela secvențele de eligibilitate și intervenție. Un flux de lucru științific poate modela transformările datelor și

cerințele de proveniență. În fiecare caz, executorul simbolic ar trebui să funcționeze pe un program tastat, mai degrabă decât pe proza brută.

Execuția simbolică este utilă în special pentru contraexemple. Analiza abstractă poate indica faptul că o încălcare este posibilă deoarece a pierdut o corelație. O căutare pe calea poate încerca să aleagă identități concrete, timpuri, condiții și excepții care să realizeze încălcarea. Dacă calea este satisfăcătoare, martorul poate fi redat ca un scenariu solzos. Dacă este nesatisfăcut, sistemul învață că avertismentul abstract a fost fals sub semantica actuală.

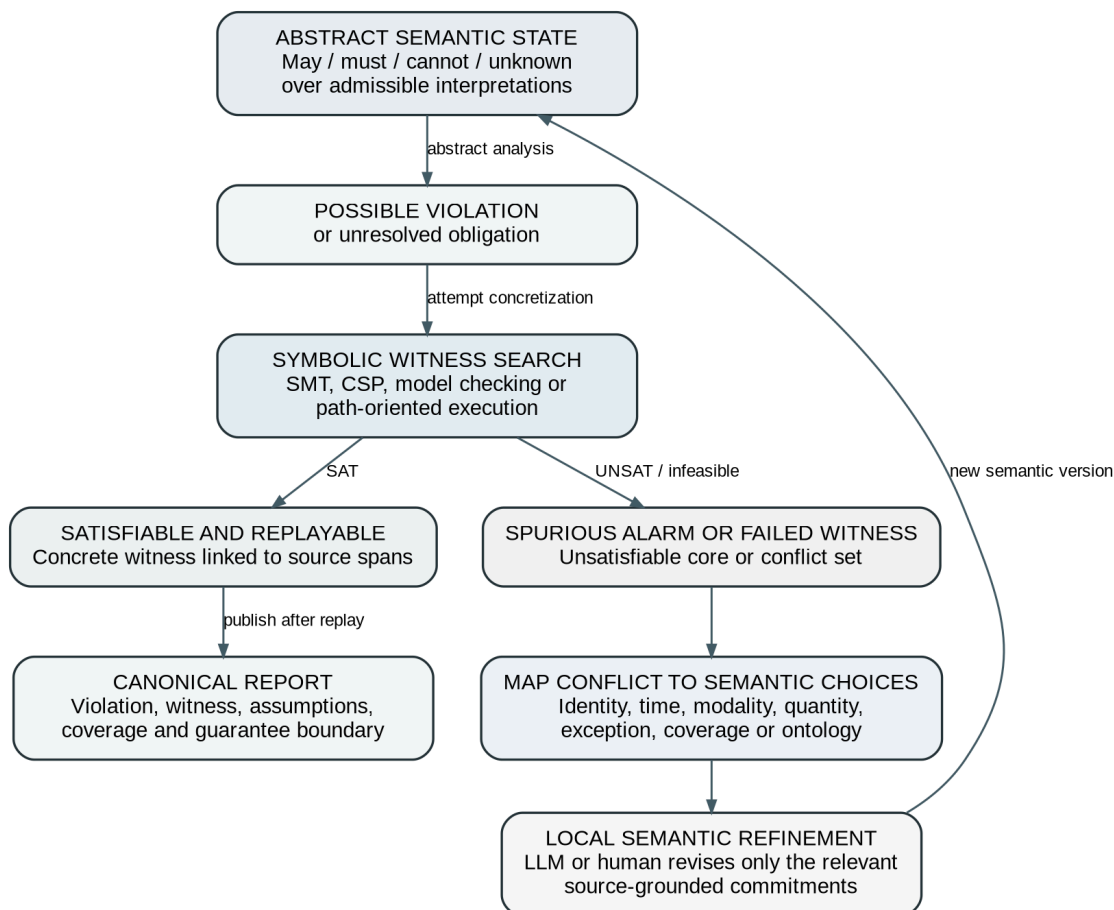


Figura 2. Rafinament semantic ghidat de contraexemplu. Analiza abstractă identifică o posibilă încălcare; căutarea simbolică a martorilor fie confirmă un caz rejucabil, fie expune distincțiile semantice care trebuie rafinate.

Acest lucru duce la o rafinare semantică ghidată de contraexemplu. CEGAR tradițional începe cu un rezumat al programului grosier. Verificarea modelului găsește un contraexemplu abstract. O verificare a fezabilității stabilește dacă contraexpunerea corespunde unei executări concrete. Dacă nu, predicatele sunt adăugate pentru a rafina abstractizarea, iar analiza se repetă. În cadrul limbajului, abstractizarea include alegeri semantice. Un contraexemplu fals poate apărea deoarece au fost introduse două mențiuni, o excepție a fost ignorată, un interval temporal a fost lărgit sau o declarație modală a fost tratată ca un fapt.

O buclă de rafinament semantic ar trebui să păstreze explicația simbolică a eșecului. Să presupunem că un avertisment de conformitate depinde de presupunerea că „tabloul” din două secțiuni denotă același organism. Reluarea simbolică arată că calea devine nefezabilă dacă entitățile sunt distincte. Sistemul cartografiază predicatul identității la mențiunile relevante și pune o întrebare concentrată: sunt aceste referințe destinate să denoteze aceeași tablă? Un LLM poate reexamina contextul local și poate propune alternative; un om poate adjuca dacă

este necesar. Restul modelului de documentar rămâne neschimbat. Acest lucru este mai fiabil și mai eficient decât să ceri unui model să recitească totul și să „încerci din nou”.

Nucleele nesatisfăcute oferă un mecanism aferent. Atunci când un set de constrângeri este inconsecvent, un rezolvator poate returna adesea un subset responsabil pentru conflict. Cartografierea miezului înapoi la sursele de bază identifică setul minim de declarații și interpretări care nu pot coexista. Într-o revizuire a politicii, acest lucru poate dezvălui obligații contradictorii. În parsingul semantic, poate dezvălui că un cuantificator, legătura identitară și asumarea temporală creează în comun imposibilitatea. Utilizatorul primește o explicație fundamentată în conflictul formal, mai degrabă decât o critică generică a modelului.

Nu orice problemă este cel mai bine reprezentată ca execuție pe cale. Verificarea modelului analizează sistemele de tranziție împotriva proprietăților temporale. Logica temporală liniară poate exprima că ceva se întâmplă în cele din urmă, ține întotdeauna sau apare înainte de un alt eveniment. Compartiment-comer și logicile probabilistice exprimă ramificarea sau comportamentul incert. Sistemele naturale-limbaj-textar-logic, cum ar fi NL2TL și nl2spec, arată că LLMs poate construi specificațiile temporale atunci când vocabularul și domeniul sunt controlate [Chen et al. 2023; Cosler et al. 2023]. Executorul este apoi un verificador de model sau planificator, nu un LLM.

Rezolvarea constrângerilor acoperă o cale de mijloc largă. SAT reprezintă alegerile boole. SMT adaugă teorii precum aritmetica, matricele, șirurile și funcțiile neinterpretate. CSP și CP-SAT se ocupă de domenii finite, programare și optimizare. MaxSAT și constrângerile moi exprimă preferințele. Multe sarcini în limba naturală – programarea, alocarea resurselor, verificarea consecvenței, relațiile familiale, puzzle-urile și combinațiile de politici – sunt mai bine compilate în constrângeri decât în codul procedural. SatLM a arătat că specificațiile declarative plus un rezolvator pot depăși abordările asistate de program privind sarcinile [Ye et al. 2023] hard aritmetice și logice. Rezultatul susține o alegere arhitecturală centrală: LLM ar trebui să descrie problema, în timp ce rezolvarea efectuează căutarea.

Planificarea formală oferă semantică de acțiune și căutare direcționată spre scop. Recentă activitate LLMs este mai degrabă formalizatori de planificare decât planificatori. Hao și colegii săi traduc constrângerile de planificare din lumea reală în instrumente formale de verificare, folosesc nuclele nesatisfăcute pentru reparații și raportează 93,9% succes pe TravelPlannerrr în cadrul lor, în comparație cu liniile de bază [Hao et al. 2025] mult mai mici ale modelului direct. Un studiu de sinteză din 2025 concluzionează că principala oportunitate nu este înlocuirea planificatorilor, ci utilizarea LLMs pentru a construi și rafina modelele [Tantakoun et al. 2025] de planificare. Tocmai aceasta este divizia susținută aici.

Motoarele de flow și Datalog oferă execuția punctului de fix, mai degrabă decât căutarea căii. Acestea sunt utile pentru consecințele monotone și pentru propagarea la nivel de documente. Automatul și motoarele de model se ocupă de secvențele de eveniment ordonate. Mesele de decizie se ocupă de politicile finite. Arhitectura ar trebui să direcționeze fiecare problemă către cel mai simplu formalism care o exprimă. Execuția simbolică nu trebuie să devină o etichetă universală pentru fiecare componentă simbolică.

LLMs poate ajuta execuția simbolică tradițională în moduri mai sigure și mai riscante. Un rol mai sigur este prioritizarea căii. Un model poate identifica regiunile de cod susceptibile de a conține vulnerabilități sau căi susceptibile de a atinge o țintă, în timp ce KLEE sau un alt motor păstrează decizia finală de fezabilitate. Lucrările recente, cum ar fi KLEECopilot, raportează îmbunătățiri substanțiale ale acoperirii din căutarea ghidată [Hao et al. 2025], deși starea

preprintului și sensibilitatea modelului necesită o interpretare [Chen et al. 2026] prudentă. Principiul este solid: euristiile învățate decid unde să caute, nu dacă o cale este validă.

Un rol mai riscant este înlocuirea rezolvatorului. AutoBug, prezentat de autorii săi ca o execuție simbolică bazată pe model în limba mare, descompune codul în sarcini de raționament bazate pe cale și cere unui model să raționeze direct despre ele fără a traduce totul în SMT. Abordarea este ușoară și limbajoasă agnostică și au fost raportate, iar experimentele raportate îmbunătățesc analiza [Li et al. 2025] bazată pe [Li et al. 2025] pe bază de [Li et al. 2025]. Autorii îl caracterizează ca fiind aproximativ. Poate analiza construcțiile care sunt dificile pentru rezolvatoarele convenționale, dar nu oferă soliditate clasică. Pentru o utilizare ridicată, un astfel de model este mai bine tratat ca un singur motor într-un portofoliu ale cărui rezultate necesită teste, reluare sau confirmare independentă.

LLMs poate genera, de asemenea, teste, rezumate sau modele pentru API-uri externe. Studiile raportează o acoperire utilă a programelor pe care instrumentele tradiționale se luptă să le gestioneze, dar performanța scade cu căi lungi, API-uri complexe, recursion și clasificare infesible-path. Aceste metode sunt promițătoare, deoarece mediile software conțin semantică care sunt costisitoare de axiomatizat. Statutul lor de rezultat ar trebui să rămână „candidat” până când execuția concretă confirmă calea sau un rezolvator dovedește condiția.

Securitatea este o preocupare majoră. Regulile în limba naturală și documentele sursă sunt intrările neîncredințate. Un document rău intenționat poate încerca injecția promptă, poate fabrica referințe sau poate exploata un traducător generator de coduri. Artefactele formale ar trebui să fie date, nu cod executabil nerestricționat. Când sunt necesari algoritmi personalizați, acestea ar trebui să fie revizuite extensiile de gazdă cu digestoare blocate, intrările și ieșirile tastate, limitele resurselor și nici o autoritate ambientală. Invocațiile de rezolvare au nevoie de bugete de timp și memorie. Un martor ar trebui să includă versiunea exactă a teoriei, compilatorului și a motorului care a produs-o.

Modelul de analiză combinat este acum clar. O analiză sezonieră și un flux de date grosieră identifică posibilele probleme într-un document lung. Motoarele formale specializate testează doar feliile relevante. Execuția simbolică construiește martori pentru afirmații dependente de cale. Verificarea modelului se ocupă de comportamentele temporale. Dispozitivele de rezolvare a constrângerilor se ocupă de alegerile combinatorii. Martorii înșelați și nucleele de rezolvare conduc la rafinamentul semantic local. Fiecare concluzie acceptată este returnată cu un statut și o proveniență care reflectă cea mai slabă verigă din lanț.

Această arhitectură nu promite verificarea totală a limbii nerestricționate. Promite ceva mai operațional: fiecare afirmație formală are o semnificație executabilă; fiecare simbol are o sursă sau o asumare declarată; fiecare contraexemplu poate fi reluat; fiecare cerere de absență este legată de acoperire; și fiecare ambiguitate rămasă este reprezentată mai degrabă decât ascunsă. Interpretarea abstractă oferă o agregare la scară și conservare. Execuția simbolică oferă precizie și martori. Combinația lor este una dintre cele mai plauzibile fundații pentru limbajul natural executabil.

4. Ce a realizat de fapt era LLM

4.1 Autoformalizare, raționament asistat de programe, logică, specificații temporale și planificare

Sosirea unor modele lingvistice mari a schimbat limita practică a compilației semantice. Sistemele anterioare necesitau gramatici specifice sarcinilor, forme logice supravegheate sau lexiconi atent proiectate. LLMs poate propune artefacte formale din descrieri deschise, cu puțină sau fără pregătire specifică sarcinii. Acest lucru extinde substanțial acoperirea, dar creează, de asemenea, o tentație de a deduce mai mult din câștigurile de referință decât justifică mecanismele. Întrebarea relevantă nu este pur și simplu dacă o metodă îmbunătățește acuratețea răspunsului. Este ce erori au fost transferate către o componentă simbolică, care erori rămân în formalizare, și ce dovezi independente există că artefactul formal reprezintă sursa.

Modelele lingvistice asistate de program, sau PAL, oferă un punct de plecare curat. LLM citește o problemă, generează un program Python și delegă calculul interpretului [Gao et al. 2022]. Pe GSM8K, sistemul raportat s-a îmbunătățit substanțial din cauza solicitării lanțului de gândire. Mecanismul este convingător, deoarece modelele lingvistice sunt adesea mai bune la descompunerea unei probleme decât la realizarea aritmetică lungă exact. Odată ce descompunerea este corectă, Python furnizează operațiuni deterministe și stat.

Limitarea PAL este la fel de instructivă. Interpretul certifică doar că a executat programul generat. Nu certifică faptul că programul modelează cu fidelitate problema în limba naturală. O variabilă greșită, o condiție omisă sau o formulă greșită poate rula fără erori. Prin urmare, succesul sistemului depinde de o distribuție în care programele sunt scurte, exemplele care constrâng puternic structura, iar erorile de execuție dezvăluie multe greșeli. Pentru raționamentul documentelor, analiza politicilor și interpretarea științifică, cartografierea este mai puțin limitată, iar interpretul furnizează un feedback mai slab.

SatLM înlocuiește codul procedural cu constrângerile [Ye et al. 2023] declarative. Modelul afirmă variabile și relații, iar un motor de satisfacere o soluție. Acest lucru poate fi mai ușor decât generarea unui algoritm, deoarece modelul nu trebuie să decidă ordinea operațiunilor. Reprezentările declarative permit, de asemenea, mai multe modele, dovezi de infesiune și depanare la nivel de constrângere. Câștigurile raportate pe sarcini de raționament dur susțin ideea că alegerea rezolvatorului ar trebui să urmeze structura problematică. Un program este în mod natural o problemă de constrângere; un proces dinamic poate fi o mașină de stare; o problemă de cuvânt aritmetică poate fi fie un program, fie constrângeri.

Logic-LM oficializează problemele logice în limba naturală și invocă solvenții [Pan et al. 2023] simbolici. De asemenea, utilizează erori de rezolvare pentru a determina autofinanțarea. În mai multe seturi de date, autorii raportează îmbunătățiri medii mari față de solicitările standard și lanțul de gândire. Arhitectura demonstrează două principii utile. În primul rând, motorul simbolic elimină inconsecvența deductivă odată ce există o formalizare validă. În al doilea rând, mesajele de eroare formale oferă un semnal de reparație mai bun decât o solicitare generică de a reconsidera.

Cu toate acestea, repararea sintaxei nu este o reparație semantică. O formulă poate fi bine formată, satisfăcătoare și greșită. Conducța Logic-LM poate corecta predicatele malformate sau

eșecurile de rezolvare, dar este posibil să nu detecteze faptul că „unele” au devenit „toate” sau că a fost omisă o excepție. Lucrările ulterioare, cum ar fi Logic-LM+++ adaugă rafinament în mai multe etape, iar LTRAG recuperează exemple de formalizare gândită, raportând îmbunătățiri față de Logic-LM și LINC pe FOLIO și AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]Z. Aceste câștiguri sugerează că formalizarea în sine beneficiază de exemple și de critică structurată. Ei nu elimină necesitatea de a evalua direct fidelitatea semantică.

LINC face modularitatea explicită: un LLM servește ca un parser semantic, logica de prim ordin reprezintă problema, iar un demonstrator de teoremă efectuează deducerea [Olausson et al. 2023]. Pe ProofWrter, combinația raportată a produs câștiguri mari față de sistemele de lanț de gândire, inclusiv modele substanțial mai mari decât parser. Analiza erorilor a arătat moduri complementare de eșec: sistemul simbolic evită multe erori de raționament, în timp ce eșecurile de formalizare rămân. Acestea sunt exact dovezile așteptate de la o arhitectură federată. Sistemul este cel mai puternic atunci când sarcina în limba naturală corespunde deja cu o teorie compactă de prim ordin.

FOLIO oferă un punct de referință util, deoarece conține probleme de raționament în limba naturală, autorizate de om, asociate cu logica [Han et al. 2024] verificată de prim ordin. Evaluează atât raționamentul, cât și traducerea [Han et al. 2024]-to-FOLZ. Setul de date rămâne o provocare chiar și pentru modelele puternice, ceea ce subminează ideea că formalizarea primului ordin este un pas de preprocesare rezolvat. FOLIO ilustrează, de asemenea, importanța complexității autorului uman. Seturile de date cu reguli sintetice pot face limbajul neobișnuit de regulat și pot permite modelelor să exploateze șabloane. Exemplele umane introduc variația lexicală, cunoașterea lumii și alegerile mai realiste ale domeniului de aplicare.

FoVer are o altă abordare prin verificarea raționamentului în limba naturală prin logica [Pei et al. 2025] de prim ordin. În loc să trateze textul raționamentului fluent ca autoautenționist, acesta construiește obligații formale și le verifică. Acesta este un model promițător pentru generația constrânsă de dovezi: un LLM poate propune un argument, dar un verificator acceptă doar pași inferențiali care corespund consecințelor formale valide. Problema rămasă este alinierea dintre fiecare propoziție și predicatele formale folosite pentru a o verifica.

Cercetarea de autonomie tratează din ce în ce mai mult traducerea ca pe o disciplină distinctă, mai degrabă decât un truc prompt. Sondaje publicate în 2025 traducerea catalogului în proverbele teoretice, limbaje logice și specificații formale, subliniind necesitatea buclelor de feedback, recuperarea, reprezentările intermediare și reperele care separă sintaxa de semantica [Weng et al. 2025]. Lucrările directe NL-to FOL raportează progrese semnificative, dar și erori sistematice în cunoașterea implicită, domeniul de aplicare al cuantificatorului și al designului [Yang et al. 2024; Lalwani et al. 2025] ontologic. Adăugarea cunoștințelor implicite de fond poate îmbunătăți performanța sarcinii, crescând în același timp riscul ca sistemul formal să dovedească o premisă neliceantă de sursă.

Tratamentul cunoștințelor de fond este o alegere arhitecturală decisivă. Într-un punct de referință de puzzle, spațiile de bun simț pot fi implicit așteptate. În verificarea juridică sau științifică, introducerea spațiilor nedecarate poate invalida rezultatul. Un sistem riguros ar trebui să distingă premisele sursă, spațiile de ambalare a autorității, axiomele de domeniu și ipotezele sugerate de modele. Soluționarul poate raționa peste toate, dar explicația finală trebuie să arate care concluzii depind de ce autoritate. Dacă o presupunere este esențială și neverificată, rezultatul ar trebui să fie condiționat.

Traducerea temporală aduce provocări diferite. NL2TL utilizează reprezentări ridicate pentru a învăța structura temporală în domenii și raportează o adaptare puternică cu relativ puține date [Chen et al. 2023]. Strategia de ridicare ascunde propuneri atomice concrete în timp ce învață modele logice reutilizabile. Aceasta este o perspectivă valoroasă de design: separați structura „întotdeauna înainte”, „în cele din urmă după” sau „până când” de fundamentarea specifică domeniului a evenimentelor.

nl2spec subliniază interacțiunea [Cosler et al. 2023]. Sistemul cartografiază părți ale cerinței în limba naturală la părți ale formulei temporale generate, permițând utilizatorilor să repare traducerile greșite locale. Această interfață recunoaște că ambiguitatea nu poate fi întotdeauna rezolvată automat. De asemenea, sugerează modul în care ar trebui să funcționeze rafinamentul semantic bazat pe LLM: utilizatorul ar trebui să vadă decizia lingvistică, nu doar un șir de operatori temporali.

Împământarea rămâne punctul fragil. O formulă temporală poate avea forma corectă, în timp ce propunerile sale atomice se referă la evenimente sau obiecte greșite. Lucrând la rapoartele de echivalență logică fundamentate conform cărora mecanismele explicite de împământare îmbunătățesc corectitudinea, consolidând distincția dintre sintaxa formula și atașamentul semantic. Pentru o perioadă de funcționare în limba executabilă, atomii temporali ar trebui să fie referințe la observațiile tastate cu întinderi și identități, nu la etichetele de formă liberă inventate de traducător.

Planificarea automată oferă un backend simbolic deosebit de matur. Limbajurile de planificare definesc tipurile, predicatele, acțiunile, condițiile, efectele, stările și obiectivele inițiale. Planificatorii caută apoi o secvență de acțiune validă. LLMs sunt planificatori săraci de orizont lung atunci când trebuie să mențină statul și constrângerile interne, dar pot fi eficienți în oficializarea unei probleme pentru un planificator. Un studiu de sinteză din 2025 identifică construcția modelului și rafinamentul ca fiind rolul central pentru LLMs în această zonă [Tantakoun et al. 2025].

Hao și colegii săi demonstrează o versiune riguroasă a modelului privind planificarea [Hao et al. 2025] din lumea reală. Sistemul formalizează constrângerile, invocă instrumente de verificare și utilizează nuclee necontabile pentru a repara problemele. Autorii raportează un succes de 93,9% pe TravelPlannerrr în cadrul lor experimental, cu mult peste liniile de bază directe LLM cu care se compară. Rezultatul este important deoarece arată că instrumentele formale pot transforma fiabilitatea planificării atunci când problema este reprezentativă. Aceasta nu înseamnă că [Hao et al. 2025] a devenit un planor de sunet. Înseamnă că planificatorul și verificatorul poartă statul long-orizont, în timp ce LLM construiește și repară cazul formal.

Planificarea expune, de asemenea, constrângeri ascunse. Cererile în materie de limbaj natural includ așteptări de bun simț, preferințele și condițiile de fezabilitate care nu pot fi menționate ca cerințe formale. Sisteme precum LRPlan încearcă colaborarea între modelele lingvistice și cele de raționament pentru constrângeri explicite și implicite, în timp ce repere precum PlanningArena arată că chiar și modelele puternice se luptă cu selecția instrumentelor, memoria contextului și consistența zXC0000QXZ logică. Un sistem de înaltă asigurare trebuie fie să formalizeze constrângerile implicite dintr-o sursă de încredere, fie să le eticheteze drept presupuneri. „Comunsense” nu este o categorie de proveniență suficientă.

RuleArena și indicii de referință aLIși testează LLMs împotriva sistemelor de reguli realiste și constată că augmentarea instrumentelor ajută, dar nu face formalizarea banală. Schemele SemEval 2026 privind conținutul și raționamentul formal arată în continuare că modelele sunt

influențate de conținutul semantic chiar și atunci când forma logică ar trebui să determine răspunsul. Intrările neuro-simbolice modulare folosesc FOL și Z3 pentru a reduce prejudecățile de conținut, dar traducerea formală și zgomotul multilingv rămân dificile [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Aceste rezultate întăresc necesitatea de a desprinde interpretarea lingvistică de la inferența formală.

Una dintre cele mai importante observații din această literatură este că sistemele de succes utilizează limbi intermediare specifice țintei. PAL utilizează Python. SatLM utilizează constrângeri declarative. Logic-LM și LINC folosi logica. NL2TL folosește logica temporală. Sistemele de planificare utilizează modele de tip PDDL. SymGPT folosește o gramatică adaptată regulilor ERC. Câmpul nu a descoperit o reprezentare formală universală pentru limbajul natural. Acesta a descoperit că LLMs poate direcționa limba în mai multe ecosisteme formale atunci când sunt disponibile exemple, scheme și feedback.

Aceasta susține o arhitectură de portofoliu, mai degrabă decât un „cod de limbă” universal. Un pivot semantic poate păstra dovezi, entități, evenimente, domeniu de aplicare și modalitate, în timp ce un selector de formalism alege ținta de execuție. Selecționerul însuși poate fi asistat de un LLM, dar decizia sa ar trebui validată împotriva contractelor de capacitate. Un formalism este compatibil numai dacă poate exprima distincțiile cerute și există observațiile întemeiate necesare. Un planificator nu trebuie selectat pentru o interogare pur taxonomică; logica de prim ordin nu ar trebui să aproximeze în tăcere dinamica temporală incertă; un simplu motor relațional nu ar trebui să pretindă că decide excepțiile juridice în mod fiabil.

Dovezile empirice susțin, prin urmare, o afirmație mai îngustă, dar substanțială. LLMs poate îmbunătăți dramatic construcția obiectelor formale executabile. Sistemele simbolice externe pot îmbunătăți corectitudinea, coerența și interpretarea. Feedback-ul pentru rezolvare poate ghida reparația. Cele mai puternice rezultate apar în domenii legate de margine, cu limbi explicite, scheme clare și răspunsuri evaluabile. Provocarea nerezolvată este compilația semantică: asigurarea faptului că obiectul formal păstrează sensul relevant al sursei și că fiecare premisă este fundamentată și autorizată.

4.2 Analiză de programe asistată de LLM, explorare simbolică, transformatori abstracți și NLP certificat

Interacțiunea dintre LLMs și analiza programului se desfășoară în ambele direcții. LLMs poate ajuta instrumentele formale să gestioneze specificațiile, modelele de cod și mediile care sunt dificil de modelat. Instrumentele formale pot verifica codul generat de LLMs, pot verifica rețelele neuronale și pot constrânge componentele învățate. Această literatură este deosebit de valoroasă pentru limbajul natural executabil, deoarece testează aceeași întrebare arhitecturală sub semantica mai clară: unde poate contribui un model probabilistic fără a deveni oracolul final?

Analiza statică tradițională depinde de abstracțiile proiectate manual, funcțiile de transfer, rezumatele și modelele de mediu. Execuția simbolică depinde de căutarea căii și codificările rezolvatoare. Aceste instrumente oferă garanții principiale, dar se luptă cu caracteristicile limbajului dinamic, codul bibliotecii, manipularea complexă a șirurilor, serviciile externe și scalabilitatea. LLMs poate recunoaște intenția de cod, poate propune invariante, poate rezuma funcțiile, poate identifica locațiile probabile ale bug-urilor și poate genera teste. Un studiu de sinteză din 2025 al analizei programului asistat de LLM categorii lucrează la metode statice, dinamice și hibride și subliniază atât oportunitatea, cât și riscurile ieșirilor [Wang et al. 2025] probabilistice.

Un rol arhitectural este îndrumarea euristă. Un executor simbolic își poate păstra semantica și poate folosi un LLM pentru a clasifica căile, pentru a identifica regiunile vulnerabile sau pentru a selecta limitele buclei. Dacă euristul este greșit, acoperirea poate avea de suferit, dar căile acceptate rămân rezolvate. Acest lucru este analog cu utilizarea unui model neuronal pentru ordonarea de căutare în teorema dovedită. Distincția dintre îndrumare și acceptare este critică. Un scor învățat poate determina unde sunt cheltuite resursele; nu ar trebui să determine dacă o condiție a căii este satisfăcătoare.

KLEECopilot reprezintă această direcție. Preprintul 2026 utilizează LLM îndrumarea pentru a direcționa KLEE către vulnerabilitățile probabile și introduce euristica pentru evadarea în buclă, raportând îmbunătățiri substanțiale ale acoperirii de bază și linii și zeci de încălcări unice [Chen et al. 2026]. Ca o preprint recentă, rezultatele necesită o replicare independentă și sensibilitate la problemele de familie model. Cu toate acestea, modelul de design este plauzibil: LLMs contribuie la prioritizarea semantică pe care o lipsește căutării clasice, în timp ce KLEE păstrează semantica execuției.

Un alt rol este generarea de teste care satisfac cale. Un model poate citi cod și propune intrările care se așteaptă să traverseze o cale țintă. Execuția concretă validează apoi dacă s-a ajuns la calea. Această abordare poate gestiona API convențiile sau formatele de date care sunt împovărătoare pentru SMT. Studiile raportează rate de succes utile, dar și eșecuri pe căi lungi, recurs, ramuri nefezabile și biblioteci complexe. Interpretarea sigură este că LLM este un generator de testare. O rulare de beton, o urmă de instrumentare sau un rezolvator rămâne validatorul.

AutoBug are o abordare mai radicală prin utilizarea LLMs pentru a efectua raționamentul descompus de cale direct [Li et al. 2025]. În loc să traducă toate constrângerile într-o teorie de rezolvare, aceasta prezintă condiții de cale la nivel de cod față de model. Acest lucru poate fi limbaj agnostic și poate analiza semantica care sunt greu de codificat. Autorii raportează îmbunătățiri ale mai multor repere de analiză a programelor, în special pentru modelele mai

mici. De asemenea, ele caracterizează analiza ca fiind aproximativă. Sistemul nu oferă soliditate clasică, iar concluziile sale pot varia cu modelul.

AutoBug este valoros pentru că arată unde metodele simbolice eșuează din punct de vedere operațional. Multe programe reale numesc biblioteci opace, manipulează obiecte complexe sau depind de contracte informale. Un model de limbaj poate oferi un comportament probabil din nume, comentarii și modele de cod învățate. Dar această flexibilitate este aceeași sursă de incertitudine. Într-un portofoliu orientat spre asigurare, analiza aproximativ LLM ar trebui să genereze candidați, rezumate sau cecuri prioritizate. Rezultatele trebuie confirmate prin teste, contracte, monitorizarea timpului de funcționare sau un motor formal ori de câte ori se face o cerere de siguranță.

LLMs poate propune invariante pentru interpretare abstractă. Un candidat invariant poate accelera dovezile dacă un verificator verifică că se menține inițial, este păstrat prin tranziții și implică proprietatea. Aceasta este o diviziune puternică a muncii: descoperirea invariantă este creativă și combinatoare, în timp ce verificarea invariantă este mecanică. Modele similare apar în proba de probă și sinteză a programelor. Modelul de limbă caută; verificatorul acceptă.

SAIL extinde acest principiu la sinteza transformatoarelor [Gu et al. 2026] abstracte. Construirea unui transformator atât solid, cât și precis pentru un operator neliniar este dificilă. SAIL cere LLMs să genereze candidați, dar impune constrângeri sintactice și semantice dure și utilizează un cost fundamental pentru nesănătoale. Sistemul raportat sintetizează la nivel global transformatoare de sunet în mai multe domenii, se potrivește cu modele manuale și găsește transformatoare precise care nu sunt anterior în literatura de specialitate. Semnificația merge dincolo de verificarea rețelei neuronale. Este o dovadă că LLMs poate automatiza construcția de metacode formal atunci când fiecare artefact acceptat este supus unor obligații de dovadă independentă.

Contrastul cu a cere unui LLM „acționează ca un interpret abstract” este ascuțit. Mitchell și colegii săi au determinat modele să facă rațiuni printr-o interpretare abstractă pe programele SV-COMP [Mitchell et al. 2025]. Modelele ar putea urma stiluri compoziționale sau de fixare în unele cazuri, dar au făcut erori logice și halucinate pe structuri complexe. Rezultatul este în concordanță cu un model general: modelele pot imita procesele formale și pot genera candidați utili, dar fiabilitatea scade atunci când procesul necesită stare exactă persistentă. Un interpret abstract real ar trebui să execute operațiunile de zăbrele; modelul poate sugera domenii, predicate sau explicații.

Verificarea formală a modelelor neuronale NLP este o altă ramură majoră. Munca de robustețe certificată întreabă dacă predicția unui model rămâne neschimbată față de toate intrările într-un set de perturbare definite. Jia și colegii săi au folosit propagarea la interval pentru substituiri verbale și au raportat o precizie adversă de 75% pe IMDB și SNLI sub familiile de substituție construite. SAFER a folosit netezirea randomizată și s-ar putea aplica modelelor black-box, inclusiv BERT [Ye et al. 2020]Z. Încorporările concentrate de robustețe au înălsprit limitele de certificare, îmbunătățind precizia certificată robustă pe IMDB de la 76,73 la 84,78% în cadrul setării [Wang et al. 2023] raportate.

Verificarea transformatorului este dificilă, deoarece auto-atenția conține produse, operațiuni softmax și corelații între poziții. Domeniile specializate îmbunătățesc limitele. Interpretarea abstractă parametriză pentru verificarea transformatorului a introdus domenii parametrizate ale afinelor pentru produsele interioare și a raportat limite mai strânse și verificarea unor cazuri care nu au putut dovedi [Huang et al. 2026]. Această linie de lucru demonstrează că analiza formală

se poate scala la arhitecturile netriviiale NLP atunci când proprietatea și setul de intrare sunt definite cu precizie.

Domeniul de aplicare al garanției nu trebuie să fie estompat. Un clasificator certificat poate fi robust pentru fiecare înlocuire extras dintr-un set specificat și încă nu este în regulă. O listă de substituție poate include „sonianime” inadecvate din punct de vedere contextual sau poate omite parafraze semnificative. Certificarea spune că modelul este invariabil cu privire la familia de perturbare formală. Nu stabilește echivalența semantică în limba umană. Această distincție ar trebui să fie explicită în orice sistem de limbă executabilă care utilizează modele de extracție certificate.

O integrare utilă ar atașa un certificat de robustitate la o observație semantică. Să presupunem că un extractor de evenimente identifică faptul că o sentință prevede o obligație. Un model certificat ar putea arăta că această clasificare este stabilă sub o familie controlată de substituții lexicale. Observația este încă propusă de model, dar are o proprietate suplimentară de stabilitate. Analiza simbolică din aval poate înregistra aceste dovezi fără a îmbunătăți observația la fapt. Un astfel de statut stratificat este mai informativ decât un punct de încredere.

Metodele oficiale verifică, de asemenea, codul generat de LLMs. Generația de testare, analiza statică, verificarea tipului și asistenții dovediți pot detecta multe erori. Sistemele de sinteză a programelor utilizează din ce în ce mai mult compilator și testează feedback-ul pentru repararea iterativă. Același model ar trebui să guverneze circuitele semantice generate și codul de formalizare. Un agent de codare poate autoriza o implementare a regulilor, dar publicarea ar trebui să necesite compilarea, testele de mutație, cazurile de referință în limba naturală, reluarea și verificatorii independenți. Un LLM nu ar trebui să fie lăsat să creeze un comportament executabil ascuns în interiorul unui artefact teoretic declarativ.

Literatura de specialitate privind modelarea protocolului din limba naturală oferă un rezultat sobru. Lucrările prezentate la ICSE 2025 au investigat generarea de modele Tamarin sau ProVerif din documentația protocolară. Utilizarea naivă LLM a fost inefficientă; conducta de succes a necesitat descompunere semantică, dezambiguizare umană limitată și transformări intermediare sonore. Acesta a generat modele corecte la scară moderată pentru 10 din 18 protocoale reale [Mao et al. 2025]. Rezultatul este semnificativ, deoarece protocoalele conțin roluri, mesaje, ipoteze criptografice, ordine de stat și temporală. De asemenea, arată că nici măcar documentația structurată puternică nu se compilează automat în modele formale de securitate.

Cazurile de eșec sunt instructive. Documentația omite ipotezele pe care experții le deduce. Același termen poate denota un tip de mesaj, o stare sau o instanță. Proprietățile de securitate pot fi menționate în mod informal sau distribuite în secțiuni. Un model formal de protocol trebuie să facă capacitățile adversare explicite. Acestea sunt analogii directe ale raționamentului documentului. Soluția nu a fost un prompt mai mare, ci o arhitectură care separă parsingul semantic, dezambiguizarea și transformarea verificată.

Revizuirile externe ale câmpului mai larg converg către o concluzie similară. Revizuirea neuro-simbolică NLP din 2026 favorizează o investigație mai puternică a arhitecturilor federative, dar avertizează că comparațiile actuale nu sunt controlate și că nu a fost stabilit [Chatzikyriakidis and Lappin 2026] niciun sistem federativ la scară de bază. Sondajul de planificare-formalizator tratează LLMs ca lideri și rafinării de modele formale, nu înlocuitori pentru planificatori [Tantakoun et al. 2025]. Sondajul programului-analiză pune accentul pe

sistemele hibride și necesitatea de a gestiona halucinația și reproductibilitatea [Wang et al. 2025]. Acestea sunt linii independente de dovezi pentru aceeași diviziune a muncii.

Domeniul a realizat, prin urmare, mai mult decât un model superficial „LLM plus rezolvator”. A demonstrat reparații ghidate de rezolvare, formalizări verificate, planificare formală, orientare a căii, propunerea invariantă, sinteza transformatorului și robustețea certificată. De asemenea, a expus o limită stabilă. Când LLM controlează acceptarea finală, garanțiile devin aproximative. Atunci când LLM propune artefacte care verifică un sistem formal separat, pot fi păstrate garanții puternice pentru proprietatea verificată. Întrebarea fără răspuns este cum să faci legătura în limba naturală-la-artifact să fie disciplinată în mod comparabil.

Arhitectura propusă mai târziu în această carte aplică direct lecții de analiză a programului. Transformările semantice candidate sunt tratate ca un cod sintetizat. Au contracte de tipare, hărți sursă, limite de resurse și teste. Domeniile abstracte sunt mai degrabă înregistrate decât improvizate în timpul unei alergări. Predicții de rafinare sunt sugerați de LLMs, dar validați prin distincții concrete. Portofoliile de soluții de rezolvare expun obiecte sau martori. Randamentul de limbaj consumă rezultate canonice, nu urme brute. În acest design, flexibilitatea LLM este un atu tocmai pentru că este înconjurată de porți simbolice de acceptare.

5. De ce eșuează sistemele de limbaj executabil

5.1 Fundamentare, identitate, formalizări spurii, ontologii și cunoaștere despre lume

Eroarea dominantă în sistemele de limbă executabilă nu este o deducere incorectă din spațiile acceptate. Este admiterea premiselor, obiectelor sau relațiilor greșite. Temerea este procesul prin care simbolurile dobândesc referințe operaționale: un predicat este conectat la o frază, la o entitate la una sau mai multe mențiuni, un eveniment la dovezi, o expresie de timp la un interval, un câmp de bază la un concept sau o acțiune la o capacitate de mediu. Un program formal fără fundament fiabil este exact în sensul cel mai puțin util. Se execută în mod consecvent în timp ce descrie lumea greșită.

Împământarea începe cu locația sursă. Orice observație semantică ar trebui să fie legată de dovezile sursă exacte sau marcate în mod explicit ca o presupunere externă. Un citat trebuie revalidat împotriva revizuirii sursei imuabile. Un rezumat al documentului nu poate înlocui sursa deoarece aceasta poate omite calificările. Rezultatele de recuperare nu pot susține o cerere de despăgubire cu privire la absență, cu excepția cazului în care procesul de recuperare este complet pentru domeniul relevant. Această disciplină sursă împiedică un eșec comun în sistemele LLM: un model produce text de susținere plauzibil care nu apare în document, iar producția structurată în aval conferă invenției o aparență de autoritate.

Împământarea Lexicală conectează limbajul cu ontologia. Expresia „aprobare” poate desemna o acțiune, un stat, un artefact semnat, o permisiune sau o decizie instituțională. O schemă de domeniu trebuie să aleagă și să documenteze interpretarea necesară printr-o regulă. LLMs sunt utile deoarece pot deduce contextul și pot propune cartografierea, dar o cartografiere trebuie testată împotriva exemplelor și alternativelor. O potrivire de coarde între o frază și un nume de caracter nu este compatibilitatea semantică. Documentația nllAgent avertizează în mod explicit împotriva cartografierii a două concepte doar pentru că etichetele lor se potrivesc; necesită un adaptor sau o sondă semantică atunci când semnificațiile diferă.

Împământarea identității este mai dificilă și mai consecventă. Analiza documentelor trebuie adesea să știe dacă două mențiuni denotă aceeași persoană, organizare, obiect, set de date, cerință sau eveniment. Numele egale nu implică identitatea, iar numele diferite nu implică diferența. Titlurile se schimbă, pseudonimele apar, pronumele sunt ambigue, organizațiile au subdiviziuni, iar mențiunile fictive sau citate nu pot aparține lumii sursă. Un strat de identitate sigur separă mențiunile de ipotezele entității și de candidații la identitate. Acesta înregistrează producătorul, dovezile, domeniul de aplicare, lumea și alternativele pentru fiecare legătură.

Fuziunea identității premature creează erori în cascadă. Într-o narațiune, două personaje cu același nume de familie pot fi îmbinate, producând încălcări imposibile ale continuității. Într-un document de reglementare, „autoritatea” se poate referi la diferite organisme din diferite jurisdicții. În documentația software, „client” poate denota un utilizator, o componentă de aplicație sau un punct final HTTP. Odată îmbinate, regulile ulterioare tratează toate proprietățile ca aparținând unui singur obiect. Eroarea devine dificil de diagnosticat, deoarece rezolvatorul vede un identificator consistent. Un sistem conservator ar trebui să permită o identitate nerezolvată și să revină necunoscută atunci când o regulă necesită o legătură care nu a fost stabilită.

Împământarea temporară are straturi similare. Datele pot fi explicite sau relative. „În termen de treizeci de zile” necesită un eveniment ancoră și o politică de calendar. „Înainte de eliberare” se poate referi la o lansare planificată sau efectivă. „După aprobare” poate însemna imediat după sau în orice moment ulterior. Jurisdicțiile definesc zilele de afaceri în mod diferit. Un rezolvator temporal poate gestiona intervalele și comanda exact odată ce aceste alegeri sunt făcute, dar nu poate deduce ancora intenționată de pe o etichetă arbitrară. Observațiile temporare ar trebui, prin urmare, să includă expresia textuală, valoarea normalizată sau constrângerea, ancorarea, calendarul, fusul orar, încrederea și alternativele.

Împământarea normalității este frecvent manipulată greșit. „Trebuie”, „să trebuiască”, „să trebuiască”, „poate”, „poate” și „este de așteptat” să aibă o forță specifică domeniului. Convențiile de redactare juridică diferă de limba obișnuită. O excepție poate învinge o regulă, poate restrânge domeniul de aplicare sau poate crea o permisiune separată. Autoritățile aflate în conflict pot avea priorități. O simplă reprezentare booleană a conformității pierde aceste distincții. Logica dezonzantă, prioritățile de reguli, tabelele de decizie sau sistemele fiabile le pot modela, dar cartografierea sursei față de reguli rămâne o judecată lingvistică și instituțională.

Formalațiile false apar atunci când programul greșit produce rezultatul așteptat. Parsing-ul semantic slab supravegheat a făcut această problemă explicită, dar este omniprezentă în evaluarea LLM. O formulă generată poate rezolva un punct de referință, deoarece exemplele au șabloane regulate. O interogare text-to-SQL poate returna rândurile actuale corecte, în ciuda unei aderări greșite. Un model de planificare poate găsi un plan, deoarece o constrângere omisă este irelevantă în cazul testat. Un circuit de reguli poate trece zece exemple în timp ce codifică conceptul greșit. Precizia răspunsului final singur nu poate distinge aceste cazuri.

Apărarea adecvată este mutația semantică. Evaluatorul schimbă un aspect al sursei sau mediului, păstrând în același timp pe alții, apoi verifică dacă formalizarea se schimbă în modul așteptat. Dacă „toți” devin „unele”, cuantificatorul ar trebui să se schimbe. Dacă o excepție este eliminată, calea exceptată anterior ar trebui să devină interzisă. Dacă două entități sunt redenumite în același șir, identitatea nu trebuie să fuzioneze automat. Dacă se modifică cazul bazei de date, o interogare corectă SQL accidentală ar trebui să eșueze. Testarea mutantă evaluează semnificația programului, mai degrabă decât doar comportamentul său pe un singur dispozitiv.

Generarea contraexemplă poate compara formalizările concurente. Având în vedere două programe logice sau executabile, un rezolvator poate căuta un model în care ieșirile lor diferă. Scenariul rezultat este tradus în limbaj natural și prezentat unui recenzor. Aceasta transformă o comparație opacă de formulă într-o întrebare concretă: „În interpretarea A, un supraveghetor care nu a finalizat instruirea poate aproba; sub interpretarea B, nu s-ar putea să nu o facă. Care este intenționată?” Metoda este utilă în special atunci când mai multe LLMs produse forme plauzibile, dar neechivalente.

Designul ontologie creează un alt mod de eșec. O schemă universală este atractivă pentru reutilizare, dar poate ascunde dezacordurile din domeniu și poate forța conceptele în categorii inadecvate. O schemă extrem de îngustă, în schimb, produce multe limbi locale incompatibile. Un design stratificat este de preferat. Stratul superior definește sursa, dovezile, ipotezele identității, lumile, timpul, modalitatea, statutul, acoperirea și lacunele. Pachetele de domenii definesc tipurile și relațiile de fond. Pachetele de cartografiere declară corespondențe, pierderi și ipoteze între domenii.

Compatibilitatea cu ontologia ar trebui să fie operațională, nu retorică. Un circuit declară tipurile și garantează că le consumă. Un producător declară ce tipuri se poate materializa, sub ce statusuri și moduri de acoperire. Publicarea verifică dacă există un producător compatibil. Timpul de rulare verifică dacă documentul concret a dat observațiile necesare. Dacă circuitul are nevoie de identități confirmate de om, dar există doar candidați de identitate propuși de model, nu poate rula la garanția revendicată. Dacă o regulă temporală are nevoie de un calendar jurisdicțional care să abslechească, rezultatul este un decalaj operațional, nu o dată ghicită.

Cunoștințele lumii sunt deosebit de periculoase. LLMs conțin cunoștințe largi, dar nu există o proveniență stabilă, o dată sau jurisdicție efectivă. Tratarea memoriei modelului ca o bază formală de fapt creează dependențe invizibile și concluzii învechite. Un sistem simbolic ar trebui să încarce pachetele de cunoștințe explicite cu identificatori sursă, digestie, intervale eficiente, jurisdicții, politici de conflict și plafoane de garantare. Pachetul poate fi curatoriat, recuperat sau asistat de model, dar pretențiile sale acceptate sunt artefacte externe. Un pachet legal poate expira. Un pachet științific poate fi înlocuit. Un pachet financiar-lume poate contrazice în mod deliberat realitatea actuală.

Conflictele dintre surse nu ar trebui șterse prin votul majorității decât dacă sarcina definește o astfel de politică. Sistemul poate avea nevoie de raționament paraconsecvent, de prioritate sursă, de o versiune temporală sau de lumi separate. O afirmație poate fi susținută de o sursă și contravine de alta. Rezultatul canonic ar trebui să le păstreze pe amândouă și să precizeze care politică, dacă există, le rezolvă. Acest lucru este esențial pentru sondajele științifice și analiza de reglementare, în cazul în care dezacordul este mai degrabă informații decât zgomet.

Ipotezele de fond trebuie să fie vizibile. Etagoniile logice presupun adesea că numele denotă indivizi distincți sau că categoriile sunt disjuncte. Documentele reale rareori afirmă toate aceste axiome. Un LLM le poate introduce pentru că fac puzzle-ul rezolvabil. Într-un sistem de cercetare sau de conformitate, aceste ipoteze ar trebui să apară într-un strat de autoritate separat. Răspunsul final poate afirma apoi că urmează o concluzie numai dacă presupunerea se menține. Acest lucru împiedică cunoașterea implicită a lumii să se deghizeze ca dovadă a documentului.

Acoperirea este fundamentul absenței. Pentru a concluziona că un document nu conține o aprobare necesară, sistemul trebuie să știe ce a fost căutat, ce secțiuni și canale au fost incluse, cum a fost recunoscut conceptul și dacă metoda de căutare a fost completă pentru afirmație. O scanare lexicală poate fi completă pentru o frază exactă, dar nu pentru parafraze. Recuperarea poate fi de mare rechemare, dar nu în zona închisă. Un registru curățat de om poate fi complet pentru o eliberare. Acoperirea ar trebui să fie un obiect de primă clasă consumat de reguli negative.

Arhitectura nllAgent conține o versiune importantă a acestui principiu. LongTextJS înregistrează acoperirea și lacunele alături de observații. Compatibilitatea circuitului verifică dacă programul real satisface starea și acoperirea necesare. Persoanele lipsă nu sunt raportate ca fiind conformare. Acest lucru este mai puternic decât modelul comun de a cere unui LLM să returneze adevărat, fals sau nu găsit, deoarece nu se găsește în raport cu un domeniu de căutare [AssistOS-AI 2026] documentat.

Dificultatea rămasă este modul de validare a observațiilor produse de model. Valecția schemei este necesară, dar slabă. Dovezile de ancorare împiedică citatele inventate, dar nu și interpretarea greșită. Scorurile de calibrare ajută candidații să clasifice, dar nu stabilesc sensul.

Cea mai puternică abordare practică combină semnalele independente: parsiere în limba controlată pentru fragmente tractabile; ansambluri model sau solicitări diverse; constrângeri ontologice; consistență încrucișată; mutații semantice; exemple negative; revizuire umană pentru distincții cu impact ridicat; și contraexemple în aval care expun consecințe.

Împământarea ar trebui, de asemenea, să fie determinată de interogare. Un sistem nu trebuie să rezolve fiecare entitate sau eveniment într-un document de 300 de pagini. Ar trebui să obțină cererea semantică din regula care este executată, să recupereze domeniile de aplicare relevante și să materializeze numai tipurile necesare. Cu toate acestea, prelucrarea în primul rând nu trebuie să confunde relevanța cu caracterul complet. Dacă o regulă întreabă dacă există o excepție oriunde în textul autorității, recuperarea trebuie să ofere un argument de acoperire sau răspunsul rămâne necunoscut. Execuția determinată de cerere reduce munca; nu justifică închiderea prematură.

Concluzia fundamentală este severă, dar constructivă. Nu există un remediu pur simbolic pentru un simbol atașat la referentul greșit. Temerea nu este un detaliu preliminar NLP; face parte din calculul de încredere. Sistemul trebuie să expună lanțul de la sursă la simbol, să păstreze alternative, să urmărească acoperirea și să refuze execuția incompatibilă. Odată ce acel lanț este disciplinat, instrumentele formale mature pot adăuga o fiabilitate substanțială. Fără ea, instrumentele certifică doar consecințele unei ficțiuni plauzibile.

5.2 Pragmatică, nonmonotonicitate, scară, securitate și eșecul evaluării

Chiar și o reprezentare bine fundamentată poate eșua, deoarece limbajul natural face mai mult decât să precizeze faptele monotone. Vorbitorii implică, presupun, revizuiesc, citează, glumesc, negociază și se bazează pe contextul instituțional. Documentele conțin excepții, priorități, definiții care se schimbă mai târziu și declarații a căror forță depinde de gen. Limbajul natural Executabil trebuie să decidă pe care dintre aceste fenomene intenționează să le susțină și trebuie să se întoarcă necunoscut în afara acestui domeniu de aplicare.

Pragmatica este prima graniță. O propoziție poate comunica mai mult decât condițiile sale literale de adevăr. „Raportul este aproape complet” poate implica faptul că nu este complet. „Unele teste au trecut” pot sugera pragmatic că nu toți au făcut-o, deși logica clasică nu implică asta. Ironia inversează sentimentul aparent. Cererile indirecte utilizează declarațiile ca acțiuni. Un formalizator care tratează deducțiile pragmatice ca fapte poate depăși; unul care le ignoră poate rata intenția practică a autorului. Tratatamentul adecvat depinde de aplicare. Un motor de conformitate ar trebui, de obicei, să justiționeze autoritatea explicită și să eticheteze interpretarea pragmatică ca fiind propusă. Un agent conversațional ar putea avea nevoie de modele pragmatice mai bogate, dar nu ar trebui să le reprezinte ca dovezi solide.

Presupoziția creează o provocare conexă. „Comitetul a încetat revizuirea propunerii” presupune că a revizuit-o. Descrierile concrete presupun adesea existența sau unicitatea. Documentele pot exploata astfel de construcții fără a afirma în mod explicit fundalul. Un sistem formal trebuie să decidă dacă presuposițiile intră ca ipoteze, deducții fezabile sau cerințe de clarificare. Afirmarea automată a lor poate fi nesigură; ignorarea lor poate face discursul incoerent. Sistemul de stare ar trebui să-și păstreze originea.

Regulile în limba naturală sunt adesea non-monotonice. Noile informații pot retrage o concluzie. „Angajații pot accesa sistemul” pot fi înfrânți de „dacă certificarea lor nu a expirat”. Logica implicită, programarea seturilor de răspunsuri, logicile fiafiere și normele prioritizate oferă mecanisme formale pentru excepții. Logica clasică de prim ordin poate codifica excepțiile în mod explicit, dar seturile mari de reguli devin dificil de gestionat și informațiile inconsecvente pot trivializa inferența dacă se aplică explozia obișnuită.

O perioadă practică de alergare ar trebui să susțină semantica cu mai multe reguli. Monoton Datalog este adecvat atunci când faptele se acumulează numai. Normele disponibile sunt adecvate pentru neîndeplinirea obligațiilor de plată și excepții. Raționamentul paraconstabil este adecvat atunci când sursele conflictuează și sistemul trebuie să evite să deriveze totul. Tabelele de decizie sunt adecvate atunci când combinațiile de politici sunt finite și explicite. Semantica aleasă trebuie să fieozată cu pachetul de reguli; nu poate fi dedus ad hoc de către LLM în timpul executării.

Sistemele normative adaugă autoritate și prioritate. O regulă ulterioară poate anula una anterioară. Un regulament de nivel superior poate preîntâmpina o procedură locală. Jurisdicția și data efectivă contează. Permișiunile și obligațiile pot intra în conflict. Paradoxurile deontice arată că codificările logice naive pot produce rezultate contraintuitive. Multe cazuri practice sunt mai bine tratate de prioritățile explicite ale regulilor, excepțiile și condițiile de încălcare decât printr-un calcul nedeontic universal. Limbajul natural executabil ar trebui să favorizeze claritatea operațională față de grandoarea teoretică.

Scala creează mai multe explozii care interacționează. Documentele lungi conțin multe întinderi și observații ale candidaților. Mențiunile ambigue creează sarcini de identitate

combinatorii. Relațiile temporare creează alternative parțiale. Regulile creează consecințe derivate. Execuția simbolică creează căi. Rezolvelle se confruntă cu sisteme mari de constrângere. Ferestrele și costurile contextului LLM impun limite separate. O arhitectură naivă care formalizează fiecare propoziție în orice logică posibilă este imposibilă.

Materializarea determinată de cerere este primul răspuns. Circuitele declară ceea ce consumă. Structura indexurilor de alergare și informațiile complexe stabile, apoi produc semantică mai bogată numai pentru domeniile de aplicare relevante. Calculul incremental reutilizează artefacte nemodificate. Interpretarea abstractă rezumă alternativele. Căile de control simbolic ale explorării de către stat și delimitat. Portofoliile de soluții de rezolvare direcționează cazurile simple către motoarele ieftine. Bugetele dure produc rezultate explicite incomplete, mai degrabă decât trunchiere ascunse.

Al doilea răspuns este modularitatea semantică. Un document poate fi analizat prin priveliști: secțiuni, ferestre de timp, jurisdicții, scene narrative, etape experimentale sau straturi de autoritate. Identitatea și consecințele regulilor pot fi locale pentru o viziune. Relațiile de tip cross-view sunt introduse numai atunci când este necesar. Acest lucru reduce hrinismul global accidental și susține paralelismul. Sistemul ar trebui să înregistreze în continuare când o concluzie depinde de presupunerea că o vedere este completă.

Al treilea răspuns este reprezentat de precizie. O trecere structurală sau lexicală ieftină elimină multe cazuri. Un permis de extracție asistat de model produce candidați pentru cei rămași. Analiza abstractă identifică potențiale încălcări. Rafinamentul simbolic se ocupă doar de ambiguități relevante pentru rezultate. Revizuirea umană este rezervată alegerilor nerezolvate de mare impact. Acest lucru seamănă cu conductele de analiză statică care folosesc analize ieftine înainte de a dovedi teorema scumpă.

Amenințările la adresa securității apar în fiecare etapă. Injecția promptă în documentele sursă poate instrui LLM să ignore schemele sau rezultatele fabricate. Traductorul ar trebui să primească conținut sursă ca date sub un prompt specific rolului, iar ieșirea sa ar trebui să fie validată în mod independent. Sursa nu trebuie să obțină niciodată autoritatea de a modifica calendarul de funcționare, pachetul de reguli sau politica de publicare. Orice text care pretinde că este o instrucțiune de sistem este încă conținutul documentului.

Codul generat este un risc și mai mare. Un DSL care permite arbitrarul JavaScript, Python, importurile sau accesul la rețea transformă învățarea semantică în executarea codului. Circuitele declarative ar trebui să se normalizeze la date simple și să apeleze numai operatorii înregistrați. Algoritmii personali ar trebui să fie instalați ca extensii revizuite cu contracte tipd, efecte, limite de resurse și digestii blocate. Sandboxing-ul rămâne necesar, dar restricția de capacitate este mai puternică decât să te bazezi doar pe o limită de proces.

Atacurile de rezolvare includ refuzul serviciului prin constrângeri patologice, obiecte cu dovezi malformate și exploatarea bibliotecilor native. Sunt necesare bugete, restricții teoretice, verificarea dovezilor și fixarea versiunii. Un model poate construi în mod deliberat sau accidental o formulă care este valabilă din punct de vedere tehnic, dar explozivă din punct de vedere computațional. Selecția formalismului ar trebui să estimeze costurile și să respingă sarcinile din afara politicii.

Otrăvirea datelor afectează producătorii semantici învățați și suitele de referință. Dacă un agent de codare generează atât un circuit, cât și exemplele care îl validează, artefactele pot împărtăși aceeași neînțelegere. Generarea independentă de testare, mutațiile semantice,

cazurile de ancorare curatoriate manual și rolurile de revizuire separate reduc acest risc. Arhitectura de învățare nllAgent împiedică corect un agent de codificare să publice propriul candidat. Publicarea manuală este un control al guvernantei, deși eficacitatea sa depinde de calitatea indicilor de referință și de verificările independente.

Realizarea în limbaj natural poate reintroduce revendicări nesustținute. Un model poate converti „posibil sub o singură interpretare” în „documentul încalcă regula”. Aceasta poate omite presupuneri sau poate transforma o căutare incompletă în stare de absență. Prin urmare, realizarea ar trebui să fie constrânsă de o schemă canonică de rezultat și verificată la nivel de revendicare. Leșirile de mare impact pot utiliza șabloane controlate. Comentariile explicative ar trebui să fie clar separate de constatările verificate.

Evaluarea este poate cel mai sistemic eșec. Multe criterii de referință măsoară doar acuratețea răspunsului final. Acest lucru încurcă extracția, formalizarea, raționamentul și redarea. Un model poate ajunge la răspunsul corect pentru un motiv greșit. Un sistem formal poate fi penalizat pentru revenirea necunoscută în cazul în care indicele de referință presupune o convenție nedeclarată. Seturile de date sintetice pot recompensa recunoașterea șablonului. Mecismurile LLM-as-a-un-juditate pot împărți prejudecățile cu sistemele evaluate.

Un punct de referință riguros ar trebui să conțină mai multe straturi. Stratul sursă include text și structură exactă. Stratul semantic include una sau mai multe interpretări admisibile cu dovezi. Stratul formal include programe țintă sau teste de echivalență prelungită. Stratul de execuție include modele așteptate, dovezi, martori sau nuclee insasifibile. Stratul de rezultate include statusuri canonice și acoperire. Stratul de realizare include pretenții permise și supraestimări interzise. Erorile pot fi apoi atribuite mai degrabă decât prăbușite într-un singur scor.

Coechipierii minimali sunt esențiale. Schimbarea unui cuvânt sau a unei relații ar trebui să producă o diferență formală previzibilă. Familiile parafrazelor testează invadarea. Retragere de încercare a secțiunilor distractorii. Coliziunile de identitate testează la sol. Dovezile lipsă testează un comportament necunoscut. Surse contradictorii testează paraconsistență sau politica autorității. Cazurile de graniță temporară testează calendarele și intervalele. Textul promptav aversar testează izolarea de securitate. Mutațiile semantice testează dacă un circuit își implementează de fapt regula.

Evaluarea ar trebui, de asemenea, să măsoare abținerea. Un sistem de încredere trebuie să știe când îi lipsește o ontologie, producător, acoperire sau buget. Necunoscut nu este un eșec atunci când întrebarea este nesustținută. Valorile ar trebui să distingă abținerea justificată de incompletitudinea evitabilă. Curbele de calitate a acoperirii pot arăta modul în care acuratețea și schimbarea falsă a asigurării, pe măsură ce sistemul admite mai multe semantice propuse de model.

Implementarea independentă contează. Un compilator de circuite și evaluatorul său de „din umbră” nu ar trebui să împărtășească fiecare funcție critică dacă se pretinde testarea diferențiată. Scorurile de mutație ar trebui să provină din mutații executabile, nu din descrieri de proză. Profilurile de model live trebuie evaluate separat de corpurile de iluminat deterministe. Planificarea și realizarea necesită repere specifice sarcinilor, mai degrabă decât să fie deduse din performanța de audit. Acestea sunt tocmai mai multe dintre problemele grave recunoscute în depozitul [AssistOS-AI 2026] [AssistOS-AI 2026].

Recenziile externe ar trebui interpretate cu o îngrijire similară. Sondajul Frontiers 2026 este evaluat și util, dar afirmația sa că sistemele federative îndeplinesc mai bine este calificată în

mod explicit prin confuzie. Sondajele de planificare și analiză a programelor cartografiază tendințele, dar nu pot stabili că o arhitectură propusă funcționează pe documente nerestricționate. Nu a fost găsită nicio revizuire independentă a nllAgent. Prin urmare, o carte responsabilă ar trebui să utilizeze revizuirile pentru a identifica consensul și dezacordul, apoi să-și precizeze propriile judecăți în cazul în care dovezile sunt absente.

Principiul evaluării centrale este atribuirea cauzei. Când un sistem se îmbunătățește, cercetătorii ar trebui să determine dacă câștigul a venit de la o mai bună parsare semantică, corectitudinea rezolvatorilor, recuperarea, eșantionarea timpului de testare sau scurgerile de referință. Când nu reușește, ar trebui să identifice stratul. Aceasta nu este doar igiena științifică. Datele de eroare stratificate sunt cele care permit arhitecturii să se îmbunătățească fără a înlocui întregul sistem sau a ascunde regresiiile în scoruri agregate.

Limbajul natural executabil va rămâne o capacitate limitată. Pragmatica, cunoașterea lumii deschise, contextul social și instituțiile în evoluție asigură că nici un nucleu simbolic finit nu captează orice semnificație. Scopul nu este de a elimina această graniță, ci de a o face explicită și utilă. Un sistem poate oferi garanții puternice pentru fragmente structurale, relaționale, temporale, cantitative și reglementate de reguli, în timp ce returnează rezultate condiționate în altă parte. Cel mai periculos sistem nu este unul care să admită limite. Este una care transformă limba nemodelată într-o ieșire simbolică încrezătoare și numește rezultatul verificat.

6. Familii arhitecturale pentru limbaj natural executabil

6.1 Proiecte concurente: reprezentări universale, portofolii de formalisme, compilare controlată și sisteme query-first

Arhitecturile în limba de execuție pot fi organizate în funcție de locul în care plasează angajamentul semantic. Unii încearcă să traducă fiecare intrare într-o singură reprezentare universală. Alții păstrează mai multe formulare de candidat și selectează un formalism pe sarcină. Unele restricționează limbajul de intrare până când compilarea este deterministă. Alții păstrează limbajul deschis, dar în formalizează doar observațiile cerute de o interogare. Fiecare design rezolvă o problemă diferită, iar un sistem matur le va combina probabil mai degrabă decât să selecteze exclusiv unul.

Reprezentarea intermediară universală este cea mai intuitivă arhitectură. Textul este compilat într-un grafic semantic bogat sau limbaj logic și fiecare operațiune din aval se interogă sau transformă acel obiect comun. Avantajele sunt unitatea conceptuală, reutilizarea și posibilitatea verificărilor de coerență globală. O singură reprezentare poate suporta mai multe aplicații și poate crea o interfață stabilă între NLP și raționamentul.

Slăbiciunea sa este o depășire semantică. Pentru a servi fiecare sarcină, reprezentarea trebuie să acopere entitățile, evenimentele, discursul, timpul, modalitatea, cantitatea, cauzalitatea, normele, incertitudinea, proveniența și conceptele specifice domeniului. Fie limba devine extrem de complexă, fie simplifică distincțiile de care are nevoie unele activități. O reprezentare universală creează, de asemenea, un singur punct de eșec: un aspect global greșit contaminează fiecare analiză. Presiunea de a alege o identitate, un domeniu de aplicare sau ontologie devreme poate ascunde ambiguitatea.

Un pivot semantic este o variantă mai modestă. Nu pretinde că este o formă logică completă. Standardizează întinderea surselor, menționările, ipotezele entității, observațiile, lumile, valorile purtătoare de timp, modalitatea, proveniența, alternativele, acoperirea și lacunele. Recipientele specializate mai mici porțiunile selectate în țintele formale. Acest lucru păstrează infrastructura comună fără a pretinde că o singură execuție semantică se potrivește tuturor sensului. LongTextJS, așa cum este documentat în nllAgent, este aproape de această idee: este mai degrabă o lume sursă adresabilă și un program de observare a teoriei universale.

Arhitectura cu mai multe candidați tratează formalizarea ca percheziție. Un LLM produce mai multe forme logice, programe sau grafice semantice. Controalele de tip și constrângerile oncologice elimină candidații nevalizi. Execuția, exemplele și contraexampoanele clasifică sau disting supraviețuitorii. Sistemul poate returna o concluzie obligatorie atunci când toți candidații sunt de acord și o concluzie în care doar unii o fac. Această arhitectură este adecvată atunci când ambiguitatea este autentică sau atunci când fiabilitatea formalizării este limitată.

Costul său este combinator. Formularele candidate se înmulțesc în propoziții și alegeri identitare. Un produs încrucișat naiv devine nefezabil. Soluția este de a reprezenta cu totul structură comună și alternative locale, de a folosi domenii abstracte pentru a le rezuma și de a rafina doar alegerile relevante pentru rezultate. Încrederea poate ghida explorarea, dar candidații cu probabilitate redusă nu ar trebui să fie eliminați dintr-o analiză solidă, cu excepția cazului în care politica acceptă în mod explicit incompletitudinea probabilistică.

Compilarea în limba controlată are o abordare opusă. Intrarea este rescrisă într-un limbaj natural restricționat a cărui gramatică și semantică sunt deterministe sau foarte constrânse. Utilizatorii pot scrie direct în limba controlată, sau un LLM poate propune o parafrază controlată pentru confirmare. Odată acceptată, compilația este de încredere. Această arhitectură este deosebit de puternică pentru cerințe, contracte, proceduri și autorizarea regulilor.

Limitarea sa este acoperirea și adoptarea. Documentele obișnuite conțin construcții în afara fragmentului controlat. Rescrierea poate schimba sensul. Utilizatorii pot aproba o parafrază fără a observa o diferență subtilă. Arhitectura funcționează cel mai bine atunci când forma controlată este afișată alături de sursă, diferențele sunt localizate, iar contraexemplele formale fac alegeri importante concrete. Limbajul controlat ar trebui să fie un punct de control semantic explicit, nu o normalizare invizibilă.

Arhitecturile interogare evită formalizarea totală. Un circuit sau o întrebare declară ce observații și relații are nevoie. Timpul de rulare păstrează sursa structural, recuperează regiunile relevante și se materializează doar tipurile semantice cerute. Interogările, tabelele și regulile de decizie fac obiectul unei decizii în lumea finită rezultată. Această scară de design este mai bună la documente lungi și aliniază calculul cu scopul.

O viziune strâns legată de sistemele de baze de date din 2026 distinge operatorii semantic interpretați de cei compilați. În designul interpretat, un LLM este invocat în interiorul buclei de prelucrare a datelor pentru fiecare rând, înregistrare sau pereche de candidați. În proiectarea compilată, LLM este invocat în timpul unei faze separate de construcție pentru a traduce operațiunea semantică în cod local determinist care poate fi executat în mod repetat fără un apel model. Experimentele preliminare au raportat reduceri substanțiale ale lantei și ale utilizării modelului, păstrând în același timp o mare parte din calitatea de ieșire [Dong and Wang 2026]. Propunerea este importantă deoarece ajunge în mod independent la aceeași intuiție arhitecturală dezvoltată în această carte: modelul lingvistic este cel mai util ca compilator sau constructor de plan, în timp ce timpul de funcționare repetat ar trebui să fie stabil și inspectabil. Limitarea sa este la fel de instructivă. Codul generat poate aproxima eficient operatorul definit de model, fără a furniza o dovadă semantică că acesta captează conceptul dorit de utilizator. Compilația elimină stocasticitatea per-timp; nu rezolvă în sine fidelitatea de bază sau specificație.

Riscul este o închidere falsă. Recuperarea poate rata dovezile relevante, iar analiza cererii poate omite un concept necesar. Sistemele interogare trebuie să urmărească acoperirea și producătorii. Dacă o concluzie negativă necesită căutarea întregului document pentru orice excepție, câteva pasaje recuperate sunt insuficiente. Arhitectura ar trebui să distingă un rezultat de interogare delimitat de o garanție a lumii închise. Contractele de compatibilitate și acoperire ale nllAgent sunt un mecanism important în această familie.

O arhitectură cu un singur formalism scade toate sarcinile într-o singură logică, adesea logica de prim ordin, Datalog, sau SMT. Acesta simplifică obligațiile de punere în aplicare și de probă. Pentru un domeniu îngust, aceasta poate fi cea mai bună alegere. Logic-LM și LINC câștigă o mare parte din valoarea lor dintr-o limbă țintă stabilă. Un sistem de programare poate folosi în mod rezonabil un limbaj de constrângere. Un verficator de protocol poate utiliza un calcul al procesului.

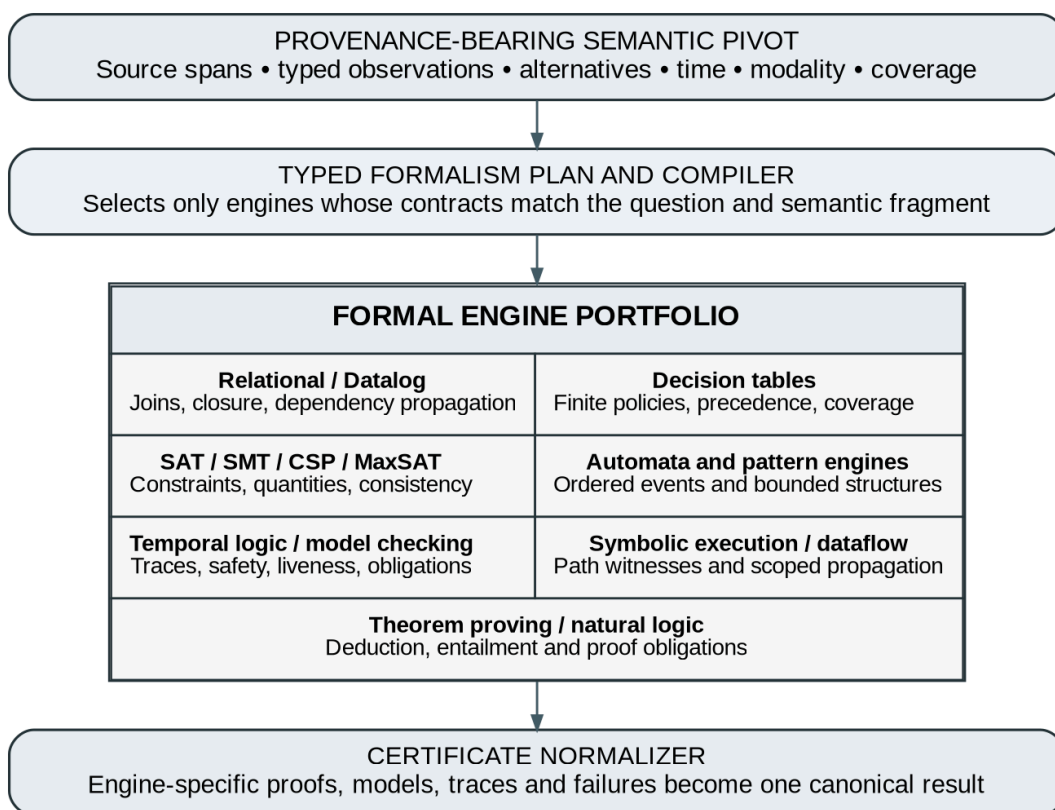


Figura 3. Un pivot semantic purtător de proveniență poate fi alcătuit într-un portofoliu de motoare formale specializate și normalizat într-un singur rezultat canonic.

Pentru sarcini largi în limba naturală, un portofoliu de solvenți este mai plauzibil. Evaluarea relativă se ocupă de interogările documentelor. Datalog calculează închiderea monotonă. Tabelele de decizie exprimă politici finite. SAT și SMT gestionează constrângerile și aritmetica. Verificarea modelului gestionează comportamentul temporal. Execuția simbolică construiește martorii căii. Logica naturală se ocupă de anumite relații de implicare în apropierea limbajului de suprafață. Automata recunoaște modelele comandate. Pivotul semantic furnizează atomi împământați, iar un plan formalism selectează unul sau mai multe motoare.

Portofoliul creează noi obligații. Routerul trebuie să cunoască capacitățile expresive și operaționale ale fiecărui motor. Un compilator trebuie să păstreze sensul pe deprecieri. Rezultatele de la diferite motoare au nevoie de un model comun de stare și proveniență. Dacă două motoare sunt motive de suprapunere, consistența trebuie verificată sau lăsată în mod explicit deschisă. Politicile de resurse trebuie să împiedice un model să selecteze un rezolvator inutil de scump.

Un contract de formalism util prevede tipurile de intrare, operatorii, semantica, acoperirea necesară, clasa de garantare, certificatul preconizat, modelul de cost și limitările cunoscute. Un motor temporal-logic poate necesita evenimente la sol și un sistem de tranziție finit. Un motor SMT poate suporta funcții reale neliniare, dar nu nerestricționate. Un tabel de decizie poate necesita condiții finite enumerate. Un motor natural-logic poate suporta monoconifică, dar nu și o interferență arbitrară de trimitere încrucișată. Routerul poate respinge apoi un plan propus ale cărui cerințe depășesc motorul.

Generația de proactivi este o altă familie de arhitectură. Artefactul principal nu este răspunsul în limba naturală, ci un rezultat canonic plus o dovadă, martor, urme de execuție sau

derivare verificabilă. Modelul de limbă generează o explicație din acel artefact. Fiecare afirmație explicativă este legată de un element de rezultat. Un verificator independent asigură faptul că explicația nu depășește conținutul formal.

Acest design este valoros, deoarece generarea și verificarea au puncte forte diferite. Un LLM poate scrie o explicație clară a unei contraexațiuni sau a unei constrângeri insațiabile. Un rezultat simbolic poate asigura că scenariul este real sub modelul acceptat. Explicația poate fi încă înșelătoare dacă ascunde ipoteze sau incertitudine, astfel încât rezultatul canonic trebuie să includă acele dimensiuni, iar contractul cu turnătorul trebuie să le solicite atunci când sunt materiale.

Arhitecturile abstracte-semantice plasează un strat de interpretare abstract înainte de execuția specializată. Observațiile candidate pot și trebuie să facă domenii. Exprângerea ieftină determină ce proprietăți sunt cu siguranță satisfăcute, cu siguranță imposibile sau potențial încălcate. Doar potențialele încălcări sunt coborâte în probleme simbolice costisitoare. Acest lucru reduce sarcina de rezolvare și oferă stări principiale necunoscute. De asemenea, creează o arhitectură incrementală naturală pentru editările documentelor.

Arhitecturile ghidate de contraexemplă închid bucla. O dovadă eșuată sau un martor fals nu este doar o eroare; ea identifică care distincție semantică contează. Sistemul cartografiază faptele de rezolvare a surselor, cere LLM sau utilizatorului să perfecționeze interpretarea relevantă și redă analiza. Această arhitectură este deosebit de promițătoare, deoarece transformă feedback-ul formal într-o înțelegere a limbajului direcționat, mai degrabă decât auto-reflecție generică.

Arhitecturile de învățare guvernează modul în care noile reguli, ontologiile și circuitele intră în producție. O extremă permite unui agent LLM să genereze cod executabil în timpul fiecărei runde. Acest lucru maximizează flexibilitatea și minimizează asigurarea. Extrema opusă permite doar reguli scrise de mână, verificate în mod oficial. Un mijloc practic utilizează agenți de codificare în spațiile de lucru, generează repere și mutații, restricționează artefactele la formulele declarative și necesită o publicare explicită. Producția de alergare citește lansări imuabile. Acesta este modelul de guvernare implementat în nllAgent și ar trebui generalizat.

Următoarea comparație rezumă compromisul de bază, fără a pretinde că un design domină fiecare sarcină.

Tabelul 1. Principalele familii de arhitectură și compromisurile lor esențiale.

Arhitectură	Puterea esențială și riscul principal
Reprezentarea semantică universală	Reutilizare și consecvență globală; riscă supracomplexitatea și angajamentul semantic prematur.
Pivot semantic plus coborâșuri specializate	Păstrează dovezi comune și acceptă mai multe motoare; necesită compilator atent și contracte cu motor încrucișat.
Multiple formalizări ale candidaților	Reprezintă ambiguitatea și permite concluziile pot/trebuie; riscă creșterea combinată.
Compilație în limba controlată	Semantica puternică și formele inspectabile ale utilizatorilor; sacrifică acoperirea și pot introduce modificări semnificative în timpul rescrierii.
În primul rând materializare	Scalează la documente lungi și nevoi specifice sarcinilor; pot confunda recuperarea cu exhaustivitatea, cu excepția cazului în care acoperirea este explicită.
Ținta unică formală	Baza simplă de încredere și evaluarea într-un domeniu delimitat; potrivire slabă pentru fenomenele lingvistice eterogene.

Arhitectură	Puterea esențială și riscul principal
Portofoliul de rezolvare	Se potrivește cu metoda computațională cu structura problematică; adaugă obligații de rutare, interoperabilitate și normalizare a certificatelor.
Generație de profecție	Constrânge răspunsurile în limba naturală la conținutul verificat; necesită verificarea robustă a realizării la nivel de pretenție.
Analiza abstractă plus rafinamentul simbolic	Echilibrează scara și precizia; soliditatea rămâne condiționată de acoperirea interpretării.
Circuitele învățate guvernate	Permite evoluția fără auto-modificare la timp, depinde de repere puternice, de testarea mutațiilor și de revizuirea publicării.

Cea mai credibila arhitectura generala este un hibrid al optiunilor de mijloc. Folosește un pivot semantic purtător de proveniență, păstrează alternative acolo unde contează, efectuează materializare bazată pe interogare, rute către un portofoliu de formalism și acceptă doar rezultate susținute de dovezi sau martori. Limbajul controlat poate servi ca punct de control interactiv pentru formalizări cu impact ridicat. Consolidarea performanței abstracte a interpretării, în timp ce rafinamentul ghidat de contraexapel îmbunătățește precizia. Învățarea rămâne în afara producției.

Această concluzie nu este doar teoretică. Se reflectă în literatura empirică. PAL, SatLM, Logic-LM, LINC, traducători temporal-logici temporale, formalizatori de planificare, SymGPT și sisteme de verificare certificată reușesc toate prin restrângerea țintei formale. Niciun rezultat nu demonstrează că o logică universală sau un model end-to-end poate înlocui portofoliul. Prin urmare, arhitectura ar trebui să îmbrățișeze eterogenitatea, minimizând în același timp interfețele de încredere între componente.

6.2 Un pipeline de referință de la text la simboluri, execuție și răspunsuri constrânse în limbaj natural

O conductă de referință în limba executabilă ar trebui să fie înțeleasă ca un compilator și un lanț de instrumente de verificare, mai degrabă decât un prompt de agent. Fiecare etapă materializează un artefact imuabil sau adaptat conținutului. O alergare poate fi reluată fără a cere modelului să-și reconstruiască raționamentul anterior. Erorile sunt atribuite în etape. Învățarea poate îmbunătăți un producător sau o teorie fără a schimba în tăcere sensul rezultatelor vechi.

Primul artefact este pachetul sursă. Acesta conține textul canonic sau datele, revizuirea, digestia, limbajul, structura, canalele și intervalele stabile. Paragrafele, pozițiile, tabelele, citatele și blocurile de cod își păstrează ordinea și identitatea. Înregistrările externe au chei la fel de stabile. Pachetul sursă este autoritatea lexicală: stabilește ceea ce a fost prezent, nu ceea ce înseamnă.

Al doilea artefact este contractul de sarcini. Se precizează că întrebarea, eliberarea de reguli, domeniul sursă, garanția dorită, ipotezele permise, lumea și jurisdicția, bugetele și politica de producție. Sarcina determină cererea semantică. O solicitare de a găsi acronime nedefinite necesită observații diferite de la o solicitare de a verifica conformitatea temporală. Contractul definește, de asemenea, dacă absența poate fi încheiată și ce forme de incertitudine necesită o revizuire umană.

A treia etapă construiește un grafic de probă. Producătorii determinați adaugă observații structurale. Păstorii în limba controlată adaugă observații pentru fragmentele pe care le recunosc. Producătorii susținuți de LLM adaugă evenimente candidate legate de schemă, relații, identități, modalități sau cantități. Fiecare observație înregistrează ancora, producător, model sau versiune de algoritm, status, alternative și acoperire. Modelul nu scrie direct în graficul canonic; un materializator reverifică, tipuri și integritatea referențială.

Identitatea și managementul mondial apar în același strat, dar nu sunt rezolvate în tăcere. Mențiunile sunt dovezi locale. Ipotezele de entitate colectează mențiuni sub o lume și o sferă de aplicare. Candidații la identitate afirmă relații diferite sau nerezolvate cu sprijinul. Interpretările reciproc exclusive aparțin unor lumi separate sau alternative păzite. Prin urmare, graficul poate reprezenta dezacordul fără a corupe consecvența globală.

A patra etapă calculează o stare semantică abstractă. Abstracțiile specifice domeniului rezumă posibilele identități, ordinele temporale, statutul normative, cantitățile și nivelurile de probe. Trebuie ca faptele să țină în toate lumile admise; faptele pot avea cel puțin unul. Necunoscutul înregistrează un decalaj, mai degrabă decât o presupunere probabilistă. Starea abstractă susține propagarea ieftină a regulilor și identifică ce întrebări necesită o precizie mai mare.

A cincea etapă produce un plan de formalism. Planificatorul nu este o decizie de formă LLM liberă. Acesta compară sarcina cu contractele de formalism înregistrate. O simplă interogare a documentelor poate rămâne relațională. O politică se poate compila în tabelele de decizie și constrângerile. O procedură se poate compila într-un sistem de tranziție. O încălcare dependentă de cale poate invoca execuția simbolică. O afirmație deductivă poate invoca o dovadă de teoremă. Numele planului au necesitat intrări, coborâșuri, versiuni de motor, limite de resurse și certificate preconizate.

LLM poate propune un plan pentru că recunoaște structura problematică și limbajul domeniului. Un verficator determinist validează faptul că planul utilizează formalisme sprijinite, că fiecare observație necesară are un producător compatibil și că nu se pierde nicio distincție semantică esențială pentru sarcină. De exemplu, o regulă care implică excepții și prioritate nu poate fi compilată într-un motor monoton fără o codificare explicită. O regulă care implică identitatea nu poate continua dacă domeniul identității este nerezolvat și verdictul depinde de aceasta.

A șasea etapă efectuează compilație tastată. Acest compilator ar trebui să fie mic, determinist și puternic testat. Transformă obiectele semantice în variabile, relații, tranziții sau predicate. Hărțile sursă conectează fiecare element formal înapoi la observații și se întinde. Compilatorii ar trebui să genereze obligații de dovadă atunci când o scădere este pierdută sau depinde de ipoteze. Diferite compilatoare pot fi comparate prin mutații semantice și modele distinctive.

A șaptea etapă este execuția oficială. Motoarele relaționale și Datalog calculează închiderea. Tabelele de decizie identifică rândurile potrivite și contradictorii. SAT/SMT/CSP simplificanții găsesc modele sau dovedesc inefectivitatea în cadrul teoriilor lor susținute. Verficatorii de modele evaluează proprietățile temporale. Executorii simbolici explorează condițiile de cale și returnează martorii betoni. Asistentele de dovadă sau proverele de la teoreme produc derivații. Puterea motorului nu este încă un verdict orientat spre utilizator; este un cazier de execuție purtătoare de certificate.

A opta etapă este verificarea reluării. Un verficator separat verifică martorul sau dovada împotriva problemei formale legate de sursă și a teoriei publicate. Pentru o constatare a unui document, acesta confirmă faptul că observațiile citate există, se potrivesc cu revizuirea sursei, statutul satisface regula, iar urmele de execuție susțin afirmația. Pentru un rezultat negativ, confirmă faptul că ipotezele de acoperire și exhaustivitate sunt suficiente. Reluarea este valoroasă chiar și atunci când este de încredere rezolvatorul subiacent, deoarece validează legătura dintre producția candidatului și politica eliberată.

Dacă analiza abstractă a prezis o încălcare, dar execuția simbolică nu o poate realiza, începe rafinamentul semantic. Sistemul extrage nucleul nesatisfăcător, predicatele contradictorii sau eșecul dovezilor. Hărțile sursă identifică întinderile relevante și alegerile semantice. LLM primește o sarcină limitată: reconsiderați această legătură de identitate, domeniul temporal, cartografierea excepțiilor sau alinierea predicată. Acesta propune candidați revizuiți; materializatorul și compilatorul le revalidează; execuția se repetă. Revizuirea umană este solicitată atunci când distincția este cu impact ridicat sau rămâne nerezolvată.

Această buclă ar trebui să fie monotonă în ceea ce privește dovezile, nu doar încrederea. O rafinare poate diviza o entitate, poate restrânge un interval de timp, poate adăuga o excepție trecută cu vederea sau poate marca o premisă nesusținută. Nu ar trebui să modernizeze o observație propusă la fapt, deoarece noul rezultat este convenabil. Fiecare schimbare este înregistrată ca o nouă revizuire semantică. Căile anterioare rămân reproductibile.

Al nouălea artefact este un rezultat canonic. Conține regula sau interogarea identității, statutul acceptat, ipotezele, trebuie și poate concluzii, candidații respinși, necunoscutele, acoperirea, dovezile sursă, referința sau referințele martorilor și limitele de execuție. Acesta poate include, de asemenea, un grafic concis de explicații verificabile pentru mașină. Acest artefact este autoritar pentru orice redare de proză.

Scena a zecea realizează un limbaj natural. Remorca primește doar rezultatul canonic și fragmentele sursă selectate. Nu reia raționamentul original. Fiecare teză generată este asociată cu una sau mai multe revendicări de rezultat. Un verficator compară sentința cu afirmațiile permise, asigură păstrarea modalității și incertitudinea și respinge adăugirile nesustținute. Rezultatul poate distinge constatările, ipotezele, interpretările și recomandările.

De exemplu, răspunsul corect poate fi: „În conformitate cu interpretarea faptului că ambele referințe la Consiliu denotă același organism, regula este încălcată deoarece aprobarea precede revizuirea de securitate înregistrată. Documentul nu stabilește dacă referințele denotă același organism; prin urmare, rezultatul general este necesar o revizuire. Un tractor fluent, dar nesigur, ar putea omite starea și ar putea anunța o încălcare. Rezultatul canonic împiedică această simplificare.

Conducta susține, de asemenea, generarea, nu numai scafandrul. Un utilizator oferă o idee sau un obiectiv. Circuitele de planificare construiesc o specificație controlată cu martori de la regula la plan. Un LLM realizează un draft. Circuitele de audit analizează proiectul utilizând aceeași cale de probă și de verificare. Revizuirea continuă până când auditul canonic satisface politica de eliberare sau bugetul este epuizat. Modelul de generare nu devine niciodată oracolul final pentru a stabili dacă propriul text satisface regulile.

Guvernarea înconjoară lanțul de instrumente. Ontologiile, producătorii, circuitele, compilatoarele, contractele de formalism și redactorii au versiuni și digestii. Agenții de învățare operează în zone de stadiere de unică folosință. Publicarea necesită repere, mutații semantice, verificări de compatibilitate și aprobare explicită. Producția citește lansări imuabile. Profilurile de modele live sunt evaluate separat, deoarece corpurile de iluminat deterministe nu stabilesc calitatea furnizorului. Pachetele de cunoștințe din lumea actuală au date și jurisdicții eficiente.

Arhitectura ar trebui să expună o ierarhie a garanțiilor. Garanțiile structurale se referă la întinderi exacte, ordonare și valabilitatea schemei. Garanțiile de fundamentare se referă la alinierea probelor și la statutul de identitate. Execuția garantează consecințele unei probleme formale. Garanțiile end-to-end le combină condiționat. Utilizatorul ar trebui să poată vedea cel mai slab strat. Un rezultat derivat din evenimentele propuse de model și o tranziție verificată mecanic nu ar trebui să fie etichetat pur și simplu „verificat”; ar trebui să precizeze că execuția este verificată prin observațiile propuse.

Un mic sâmbure de încredere este un obiectiv de design major. Sarcina include canonicalizarea sursei, verificarea schemei și de tip, compilatoarele formale, executorii înregistrați sau verficatorii de verificare, verficatorii de reluare și validarea rezultatului canonic. Prognozele LLM, generatoarele candidaților, euristica de recuperare și modelele de explicații rămân afară. Componentele de încredere pot conține în continuare bug-uri, dar comportamentul lor este stabil, testabil și revizuiabil. Aceasta este o îmbunătățire substanțială față de încrederea într-un model de fundație în evoluție ca un factor monolitic.

Conducta propusă nu este singura implementare posibilă, ci captează obligațiile dezvăluite de literatura de specialitate. Explică de ce este necesar un rezolvator, dar insuficient. Oferă interpretări abstracte și execuție simbolică roluri distincte. Acesta susține mai multe limbi formale fără a abandona un strat de dovezi comune. Acesta permite LLM creativitatea în descoperire, reparare și explicație, păstrând în același timp autoritatea simbolică asupra împământării și rezultatelor acceptate. Cel mai important, transformă eșecul într-un artefact numit – un tip nesustținut, o identitate nerezolvată, o acoperire incompletă, o pierdere de

compilație, o formalizare nesatisfăcătoare sau o sentință neliceală – mai degrabă decât o halucinație vagă.

7. LongTextJS, CircuitJS și nllAgent: un studiu de caz critic

Depozitul public NaturalLanguageLinterAgent este una dintre puținele încercări de a face explicită limita dintre interpretarea limbajului și executarea regulilor într-o implementare executabilă. Proiectul nu este un factor general în limba naturală, iar documentația sa nu îl prezintă ca unul. Este o bibliotecă Node.js 22+ ESM și o durată de funcționare a liniei de comandă cu două moduri de produs în limba controlată, peste aceeași arhitectură LongTextJS-CircuitJS. Modul de audit aplică o teorie versiunii unui document Markdown existent și emite un produs de audit canonic. Modul de specificare aplică teorii de planificare la o idee la nivel înalt și emite un plan de generare verificat; un model de limbă opțională poate realiza acest plan ca proză, dar circuitele de audit neschimbat rămân oracolul de acceptare [AssistOS-AI 2026].

Depozitul este util ca studiu de caz, deoarece întruchipează teza centrală a acestei cărți sub formă de software: modelele pot propune observații semantice sau proză, dar autoritatea de producție este atribuită eliberărilor imuabile, verificărilor explicite de compatibilitate, execuției circuitului restricționat, verificatorilor și artefactelor canonice. Proiectul menține, de asemenea, un registru public de probleme grave. Această practică contează pentru că o suprafață mare de documentare poate face ca intențiile arhitecturale să pară garanții implementate. Discuția de mai jos distinge ceea ce este stabilit operațional de ceea ce rămâne o ipoteză de cercetare.

Nicio evaluare independentă evaluată de colegi sau o evaluare tehnică a nllAgent, LongTextJS, sau CircuitJS a fost localizată prin căutări ale numelor proiectului, denumirea depozitului, documentația URL și combinațiile cu „revizuire”, „evaluare”, „de referință” și „replicare” începând cu 2 august 2026. Rezultatele căutării au condus la proiectul în sine, mai degrabă decât la o evaluare externă. Prin urmare, hotărârea în acest capitol este revizuirea arhitecturală independentă a autorului, bazată pe depozitul public, documentarul, exemplele executabile, procesul de eliberare declarată și dosarul actual al problemelor grave. Absența unei revizuii externe nu este nici laudă, nici critică; înseamnă că afirmațiile proiectului ar trebui să rămână înguste până la reproducerea independentă.

7.1 Ce implementează arhitectura publică și de ce contează granițele sale

Sistemul definește trei activități de producție care sunt separate în mod deliberat. Auditul de producție se arată într-o versiune publicată imuabilă, compilează un document în LongTextJS, execută circuitele de validare ale versiunii, verifică constatările candidaților și persistă un rezultat canonic CNL/Audit-1 cu un raport realizat. Specificația de producție compilează o idee în LongTextJS observații, aplică circuite de planificare și produce un artefact canonic CNL/Plan-1; realizarea opțională poate genera apoi Markdown și să-l prezinte la aceeași mașină de audit. Învățarea are loc într-un spațiu de lucru de unică folosință. Un agent de codare poate genera artefacte și repere de identificare a candidaților, dar controalele de pe partea gazdă admit doar fișiereshed, iar un întreținător trebuie să publice și să activeze în mod explicit o eliberare. Acest lucru face ca învățarea să fie analogă dezvoltării software-ului guvernat, mai degrabă decât auto-modificarea de rulare.

Cea mai importantă unitate de lucru nu este raportul final, ci conducerea directorului. Un audit păstrează intrarea copiată, programul sursă, rezultatele de bază selectate, compatibilitatea și acoperirea, urma circuitului, rezultatele verificatoare, constatările, raportul canonic care poate fi citit automat și vizualizarea care poate fi citită de om. Planificarea păstrează artefactele planului corespunzător. O alergare care se oprește pentru că cunoștințele lipsesc încă înregistrează eșecul și poate crea o problemă reutilizabilă. Regula de proiectare conform căreia lipsa cunoștințelor nu este raportată niciodată ca conformitate este fundamentală. Previne un eșec comun al agenților de documente: traducerea „Nu am găsit dovezi” în „documentul satisface regula”.

LongTextJS ar trebui să fie înțeleas ca un program de sursă numai într-un sens constrâns. Nu atribuie un comportament imperativ propozițiilor arbitrare. Ea compilează o revizuire a sursei fixe într-o lume finită, interogabilă, de structuri și observații. Proprietățile valoroase sunt identitatea sursă, întinderile stabile, tipurile de observație versiuni, identitatea producătorului, acoperirea, lacunele explicite și replayabilitatea. Un artefact pe partea de document poate fi executat deoarece consumatorii înregistrați pot interoga și transforma aceste relații în cadrul semantică declarată; este mai aproape de o reprezentare intermediară semantică sau de un program de încărcare a bazelor de date decât de codul de aplicare fără restricții.

Această distincție este mai mult decât terminologie. Atunci când un model traduce o propoziție, producția sa nu ar trebui să devină autoritară doar pentru că este serializată într-un limbaj de programare. Textul sursă rămâne o dovadă; observația tradusă înregistrează o interpretare a dovezilor respective, generată de un producător numit sub o schemă și un profil model. Circuitul rămâne o teorie separată. O constatare devine publicabilă numai după ce un verificat verifică condițiile așteptate de eliberare. Aceste straturi împiedică un jucător să devină în tăcere judecător și împiedică o explicație să devină singura înregistrare a unei inferențe.

Pachetul de fundație actual ilustrează domeniul semantic intenționat. Auditul obișnuit, planul și comenzile de referință selectează o fundație limitată-core-@1.1.0, cu excepția cazului în care sunt dezactivate în mod explicit. Pachetul recunoaște formele controlate-engleze documentate pentru stat, tip, precepția temporală exactă exprimată prin „înainte”, aritmetica exactă, cantitatea și afirmațiile literale de emoție. Cinci circuite reformate verifică numai invariantele publice, temporale, aritmetice, elementare fizice și de tip emoțional. Pachetul selectat și digestia sunt persistente. Schimbarea faptelor politice, geografice, sociale, economice și juridice este exclusă

în mod intenționat de la adevărul fundației atemporal. Aceasta este o limită de model de apărare. Un sistem care limitează faptele actuale fără date eficiente, jurisdicții, surse și politici de conflict nu poate oferi o execuție semantică reproductivă.

Sistemul susține materializarea deterministă și asistată de model. Selecția de traducere poate prefera un agent de limbaj-model configurat AhilesAgentLib, să utilizeze adaptorul actual de codificare constrâns sau să ruleze fără un model pentru agenții determiniști. Fiecare apel model primește o abilitate specifică rolului, mai degrabă decât întregul spațiu de lucru al agentului. Dacă o observație critică nu poate fi materializată în cadrul schemei și bugetului de resurse necesare, rezultatul este incompatibil, incomplet sau epuizat. Modelul nu controlează construcția surselor, politica de publicare sau semnificația unui verficator. Aceasta este exact direcția corectă pentru un sistem neuro-simbol federativ: modelul este puternic la margine, în timp ce artefactele simbolice și acceptarea controlului politicii gazdă.

Acoperirea este tratată mai degrabă ca o obligație executabilă decât un scor informal de încredere. Compilarea circuitului funcționează înapoi de la nodurile accesibile pentru a determina care observații externe pot influența un rezultat. Publicarea înregistrează contracte care leagă aceste cereri de producătorii disponibili. Compatibilitatea run-time întreabă apoi dacă documentul concret a dat de fapt observații cu tipul, statutul, cardinalitatea, structura, canalul și acoperirea necesară. Capacitatea declarată a unui producător nu este confundată cu dovezile că documentul actual a fost prelucrat cu succes. Când o cerere critică este neîndemânatică, alergarea se oprește la o limită numită.

Acest design bazat pe cerere este potrivit pentru documente lungi. Se evită să se ceară unui model să producă o interpretare universală a fiecărei propoziții înainte de a fi cunoscută orice întrebare. Un circuit poate solicita dimensiunile semantice de care are nevoie: termeni exacti, afirmații de stat, cantități, relații temporale sau observații de domeniu. Compilerul poate materializa numai observații relevante pentru ieșire și poate persista ceea ce a fost și nu a fost acoperit. Avantajul științific nu este doar un cost mai mic. Aceasta limitează angajamentele semantice inutile și face posibilă formalizarea incompletă.

CircuitJS este reprezentarea teoriei. Modulele de circuite adaptabile la agent sunt limitate la o expresie declarativă, normalizate la date simple și transmise prin același compilator utilizat pentru descrierile circuitului non-cod. Ei pot invoca numai operatorii înregistrați și verificatorii; ei nu pot ascunde un comportament executabil arbitrar în interiorul unei reguli învățate. O aplicație gazdă poate instala o extensie de rulare revizuită care conține algoritmi reali, dar intrările și contextul sunt copii de date simple înghețate, rezultatele trebuie să fie date simple, identitatea cache include digestia de extensie și versiunile care depind de blocarea de extensie care digeră. Interfața standard de comandă nu acceptă căi de extensie executabile arbitrare.

Această limită abordează unul dintre cele mai grave riscuri practice în sistemele de reguli „învățate de agent”. Dacă un agent de codificare poate genera JavaScript arbitrar în fiecare circuit, învățarea unui fel de scurgere devine generarea de coduri de la distanță și comportamentul semantic al unei eliberări devine dificil de inspectat. Restricționarea autorizării circuitului obișnuit la operațiunile înregistrate creează un vocabular operațional fin. Algoritmii specializați rămân posibili, dar intră printr-o traiectorie separată de inginerie, revizuire și versiune. Prin urmare, arhitectura distinge evoluția teoriilor de evoluția timpului de rulare de încredere.

Granița verificatoare este la fel de importantă. Execuția circuitului poate crea constatări ale candidaților, dar un verificat controlează dacă acei candidați intră în auditul sau planul canonic.

În exemplele legate, verificatorul poate reciti ancorele sursă, observațiile, poate elibera metadate și politica aplicată. Nu este suficient ca un nod de execuție să emită un obiect ale cărui câmpuri arată ca un avertisment. Verificatorul stabilește proprietatea specifică așteptată de produs. Acest lucru nu dovedește automat interpretarea semantică corectă, dar oprește ieșirea accidentală sau rău intenționată a circuitului de la mascat ca o constatare acceptată.

Modul de specificație demonstrează modul în care aceeași arhitectură poate sprijini generația controlată fără a acorda autoritate modelului de scriitor. Circuitele de planificare construiesc un plan canonic și necesită martori de la regula la plan pentru regulile aplicate. Realizarea opțională transformă planul în proză. Proza candidatului trece apoi prin modul de audit, iar modelul de realizare nu judecă propria sa conformitate. Aceasta este mai puternică decât generația constrânsă doar promptă, deoarece obiectul autoritar este planul și urma de audit, nu aderența auto-raportată a modelului.

Subset-ul experimental de interogare-primul este, de asemenea, semnificativ. LongTextJS canonic este expus ca o lume finită de citire. Interogările tastate și valorile tabelor conștiente de dovezi sunt coborâte în execuția obișnuită CircuitJS. Controalele controler au sprijinit relațiile, expresiile, anti-împortirea, ordonarea și politicile de lovire; înregistrările de publicare înregistrează contractele de interogare și hărțile sursă; benchmark-urile compară evaluarea directă a referințelor cu execuția scăzută. Actuala lansare editorială a producției rămâne în mod deliberat mai simplă, în timp ce agregatele, modelele comandate, indicii nativi și raționamentul avansat sunt manipulate prin operatori direcți sau grafice. Această reținere este de preferat să pretindă un limbaj universal matur de interogare înainte ca semantica și optimizarea să fie stabile.

Modelul de publicare a depozitului întărește reproductibilitatea. Agenții numiți dețin materiale de autoritate separate, scheme, profiluri de extracție, circuite, suite de referință, candidați, lansări, run-uri, probleme și feedback. Un agent de codificare poate propune candidații, dar nu îi poate face activi. Producția spune o versiune imuabilă. Această structură face posibilă reproducerea teoriei, a fundației, a profilului de producător, a extensiilor a digerat și a verificat un rezultat generat. De asemenea, permite mai multor agenți de codificare sau familiilor de model să construiască teorii concurente fără a împărtăși în tăcere o tonologie sau o eliberare activă.

Cea mai puternică afirmație implementată este, prin urmare, mai degrabă arhitecturală decât semantică: nllAgent are o cale disciplinată de la sursă imuabilă la observațiile proiectate, de la cererea de observare la compatibilitate, de la circuite restricționate la publicare controlată de verificaor și de la un produs canonic la un raport care poate fi citit de om. Are stări de oprire explicită. Nu trebuie să pretindă că fiecare document este compatibil ontologic. Acesta tratează rezultatele modelelor ca propuneri legate de rolul. Acestea sunt realizări substanțiale în comparație cu conductele convenționale LLM-agente, în cazul în care promptul, interpretarea, raționamentul și explicația există adesea doar într-un singur context tranzitoriu.

Arhitectura nu stabilește însă că limbajul natural arbitrar a fost fundamentat corect. Spațiile stabile dovedesc că dovezile există într-o locație; ele nu dovedesc că un actor, un eveniment, o excepție sau o modalitate a fost interpretată corect. O observație tipizată poate fi în continuare referiri la entitatea greșită. O cursă completă poate fi completă numai pentru contractul de observare limitat, nu pentru fiecare sens relevant pe care un om l-ar putea deduce. Documentația și registrul de probleme proprii ale proiectului sunt în mod corespunzător

precaute cu privire la acest punct. Valoarea implementării este că aceste limitări au locuri de reprezentare, mai degrabă decât să fie ascunse în interiorul unui răspuns final.

7.2 Evaluare independentă: puncte forte, probleme serioase actuale și evoluția necesară

Evaluarea mea generală este pozitivă cu privire la limitele încrederii și precaută cu privire la afirmațiile semantice. nllAgent este mai mult decât un ambalaj prompt, mai mult decât o conductă vector-reprilie și mai mult decât o colecție de scheme JSON. Artefactele sale din partea sursei și teoriei, publicarea imuabilă, limita de autor restricționată, verificarea reluării, verificările de acoperire și produsele canonice constituie un substrat credibil de cercetare. Arhitectura este deosebit de bine aliniată cu modelul federativ identificat în recenziile neuro-simbolice recente: componenta neuronală propune sau realizează limba, în timp ce componenta simbolică își păstrează propria identitate, semantica și criteriile ZXC000QXZ de acceptare.

Dovezile prezente ale proiectului susțin totuși fezabilitatea delimitată, nu un limbaj natural general executabil. Fundatia implicita recunoaste un mic fragment controlat. Demonstrația editorială utilizează operatori și verificatori îngusti. Compilarea cu intemperii este experimentală. Realizarea controlată este limitată. Nicio evaluare independentă nu a stabilit fidelitatea formalizării, rezolvarea identității, transferul în domenii, calibrarea erorilor sau superioritatea față de liniile de bază simple. Prin urmare, strategia de publicare corectă este de a preciza exact ce proprietăți sunt puse în aplicare și de a atașa orice pretenție mai puternică la un nivel de referință și de garantare.

Actualul registru al problemelor grave identifică șase limitări ingineresti și semantice. Prima este incrementalitatea programatorului. Durata de rulare are mecanisme utile de dependență și caching, dar nu revendică încă un motor de fixare complet general pentru circuite arbitrare, relații recursive, lumi alternative și actualizări non-monotonale. Acest lucru contează pentru documente lungi, deoarece o mică editare ar trebui să invalideze numai artefactele semantice și statele circuit care depind de el. O reluare completă naivă este corectă, dar costisitoare; un program incremental nesănătos este rapid, dar periculos. Soluția necesită monoconectivitate și efecte explicite, proveniență dependentă, separarea epocii pentru retractări și comparații de referință între execuția incrementală și completă.

A doua problemă este că suportul de interpretare nu curge încă uniform prin fiecare cale de execuție. Un document poate conține citiri concurente, lumi, contexte sau condiții de prezență, dar un operator, cheia cache, agregarea sau verificatorul le pot prăbuși accidental. Acesta este obstacolul central în tratarea LongTextJS ca o stare semantică abstractă. Fiecare valoare care depinde de o interpretare trebuie să poarte contextul său. Înscrierile trebuie să combine contexte compatibile. Agregarea trebuie să spună dacă variază într-o singură lume sau în alternative. Identitatea cache trebuie să includă formula contextuală. Operatorii care nu pot păstra alternativele ar trebui să fie declarați incompatibili, mai degrabă decât să aleagă în tăcere unul.

Această problemă nu este un decalaj minor de caracteristici. Promisiunea științifică de bază a limbajului natural executabil este adesea că sistemul poate spune ceea ce este adevărat în toate citirile admisibile, ceea ce este posibil sub cel puțin unul și care rezultă depinde de o interpretare disputată. Dacă se pierde alternative pe un traseu intern de execuție, o interfață lustruită poate / must înșelătoare ar fi înșelătoare. Prin urmare, proiectul ar trebui să facă din conservarea interpretării un invariant pe tot parcursul anului și să-l testeze prin lumi finite generate, proprietăți metamorfice și reluare diferențială.

A treia problemă este compatibilitatea conservatoare pentru comportamentul procedural opac. Extensiile înregistrate și operatorii sunt necesare deoarece unii algoritmi utili sunt incomode pentru a exprima în mod declarativ. Dar un compilator nu poate deduce cerințe semantice precise, efecte, monotonicitate, utilizarea resurselor sau obligațiile martorilor de la opac JavaScript. Respingerea conservatoare este mai sigură decât acceptarea optimistă, dar opacitatea excesivă reduce compozabilitatea și face ca materializarea bazată pe cerere să fie mai puțin eficientă. Fiecare primitiv procedural are nevoie de un contract deținut de registru care să declare tipurile de intrare și ieșire, statusurile epistemic acceptate, cerințele de acoperire, efectele posibile, determinismul, monotonismul, comportamentul cache, clasa costurilor și martorul ușor de citit. Contractul ar trebui să fie mai autoritar decât comentariile scrise de autorul prelungirii.

A patra problemă este planificarea minimă a interogării și indexarea temporală. O lume de query finită poate executa corect fără o optimizare sofisticată, dar utilizarea la scară documentată va necesita planificarea de asociere, estimări de selectivitate, indexuri de anvergură și entități, indexuri de interval temporal și întreținere incrementală. Aceste optimizări nu sunt doar detalii de performanță. Un planificator care rescrie interogările trebuie să păstreze multiplicitatea dovezilor, formulele de context, comanda, semantica anti-unire și acoperirea. Indicii nativi trebuie să rămână acceleratoarele de unică folosință, mai degrabă decât sursele de adevăr. Testele diferențiale ar trebui să compare evaluarea optimizată și de referință în lumea generată, în special pentru interogările de absență, suprapunerea temporală, interpretările alternative și dovezile duplicate.

A cincea problemă este în mod deliberat generația controlată îngustă. Realizarea exactă sau verificată a constatărilor și planurilor legate este valoroasă, dar nu reprezintă încă o proză verificată în formă lungă. Generația de formă lungă introduce agregarea, pronumele, elipsiunea, accentul retoric, relațiile de vorbire și implicațiile create de ordinea propoziției. O realizare poate conține doar declarațiile atomice licențiate și implică în continuare o narațiune causală sau normativă nesusținută. Următorul pas ar trebui să fie realizarea unei dovezi, mai degrabă decât un scriitor mai larg neconstrâns. Un rezultat canonic ar trebui să furnizeze identificatori de creație, entități și sloturilor de cantitate, modalități, incertitudine și relațiile de discurs permise. Textul generat ar trebui să fie re-paradis și comparat cu aceste afirmații. Nepotrivirile critice ar trebui să respingă textul; materialul stilistic din afara fragmentului verificat ar trebui să fie etichetat la un nivel de asigurare mai scăzut.

A șasea problemă este suportul de mutație specifici scenariului. Decizia de referință obișnuită poate ascunde reguli care sunt inerte, redundante sau incorecte. Un circuit poate trece toate exemplele curatoriate, chiar dacă o excepție este ignorată, deoarece niciun caz nu o activează. O evaluare serioasă ar trebui să genereze mutanți semantici de tip care să schimbe pragurile, comparatorii, cuantificatoarele, negarea, modalitățile, legăturile de identitate, ordinea temporală, starea dovezilor, predicatele de acoperire, domeniul de aplicare al excepțiilor, prioritatea de reguli și rutele de verificare. O suită de testare „ucide” un mutant atunci când rezultatul canonic așteptat se schimbă în modul necesar. Factorii mutanți supraviețuitori dezvăluie teste slabe sau clauze redundante semantic. Mutația ar trebui să acopere programele sursă, teoriile circuitelor, coborârea compilatorului și constrângerile de realizare, nu numai șirurile în exemple.

Aceste șase aspecte sunt mai informative decât o etichetă generică „prototip”, deoarece identifică unde arhitectura actuală poate pierde fie scară, fie semnificația. Ele sugerează, de

asemenea, o cale coerentă de evoluție. Prima etapă ar trebui să fie o semantică formală delimitată pentru magazinul semantic și lumea interogării. Nu trebuie să formalizeze limba engleză nerestricționată. Ar trebui să definească lumile sursă imuabile, identitatea de observare, statutul epistemic, acoperirea, alternativele, tranzacțiile, rezultatele interogării, publicarea și retragerea. Invariabilele cheie pot fi verificate prin cazuri finite și testate din punct de vedere al proprietății în implementare.

A doua etapă ar trebui să fie execuția conștientă de interpretare. Condițiile de prezență sau contextele factorizate ar trebui să însoțească observațiile dependente de alternative. Programatorul, motorul interogării, registrul primitiv, cache-ul, verificatorul și produsele canonice ar trebui fie să păstreze aceste contexte, fie să declare incompatibilitate. Poate, trebuie, nu poate și necunoscut ar trebui să fie calculată din seturi explicite sau abstracții ale lumilor admisibile, nu din încrederea modelului. Aceasta ar crea puntea de beton de la arhitectura existentă la interpretarea abstractă.

Cea de-a treia etapă ar trebui să fie mai degrabă un registru de formalism decât o extindere CircuitJS pe termen nelimitat. CircuitJS este cel mai bine folosit ca orchestrație, contract de dovezi și limbaj de publicare. Motoarele specializate ar trebui să dețină semantică specializată: evaluare relațională și Datalog pentru închidere și unire; tabele de decizie pentru precepția politicii; SAT, SMTZ, CSP, sau MaxSATZ pentru constrângeri; automate pentru modelele ordonate; rezolvatoare temporale și verificatori de model pentru urme; execuție simbolică pentru martorii căii; cadre de flux de date pentru propagarea. Fiecare adaptor ar trebui să declare un fragment de introducere tipd, ipotezele de traducere, certificatul de ieșire, verificarea reluării și modurile de eșec.

Cea de-a patra etapă ar trebui să fie rafinarea semantică ghidată de contraexapel. O analiză abstractă grosieră poate identifica o posibilă încălcare. Un eliberator sau un executor simbolic caută apoi un martor coerent. Dacă martorul este satisfăcător și rejucat, sistemul îl raportează cu intervale de sursă. Dacă este fals, un punct de bază nesatisfăcător sau un set de conflict trebuie să identifice ce identitate, temporală, modală, cantitate, excepție sau s-a pierdut distincția de acoperire. Cererea de rafinare ar trebui să vizeze numai sursa și producătorul relevant. Artefactul semantic reparat devine o versiune nouă, iar contraexapelul devine un caz de regresie. Această buclă ar fi o sinteză cu adevărat nouă a propunerii semantice bazate pe LLM, a interpretării abstracte și a execuției simbolice.

Cea de-a cincea etapă ar trebui să fie un strat mai puternic de împământare. Ancorele de siguranță și sursă sunt necesare, dar insuficiente. Timpul de rulare are nevoie de seturi de identitate candidate, aceleași constrângeri explicite / diferite, valabilitate temporală, verificări ale unității și jurisdicției, autoritatea sursă și politicile privind conflictul. Legăturile de top generate de modele nu ar trebui să fuzioneze în mod distructiv obiecte. Entitățile critice ar trebui să utilizeze identificatori autorizați sau confirmarea umană. Cunoștințele externe ar trebui să intre prin intermediul unor pachete de citire, cu conținut, cu surse, intervale efective, jurisdicție, expirare, soluționarea conflictelor și plafoane de garantare. Recuperarea web poate propune actualizări ale pachetelor, dar nu ar trebui să devină un adevăr implicit atemporal.

Cea de-a șasea etapă ar trebui să fie o evaluare independentă. Corpurile de fixare deterministe stabilesc reproductibilitatea software-ului, dar nu calitatea traducerii semantice live. Un profil de evaluare opt-in ar trebui să fixeze, furnizorul, solicitările, versiunile de instrumente, politica de temperatură și captare; să repete cazurile pentru a măsura varianța; costul record și latența; și să interzică regresul silențios. Un evaluator de referință pus în aplicare separat ar

trebui să compare rezultatele interogării și circuitului acolo unde este posibil. Anotarea umană ar trebui să se concentreze pe fidelitatea sursă-semantică și pe ambiguitățile relevante din cauza rezultatului. Dezechilibrele ar trebui să includă o judecată LLM directă, LLM plus recuperarea, LLM-la-unu-formalismul, compilarea în limba controlată și sistemele de model determinist. Proiectul ar trebui să demonstreze atunci când arhitectura sa suplimentară produce valoare măsurabilă.

O evaluare utilă poate fi rezumată fără a umfla revendicările. Conservarea sursei, eliberările imuabile, persistența urmelor, separarea observațiilor documentelor de teoria reutilizabilă, acoperirea explicită, scrierea restricționată a circuitului, publicarea controlată de verificatori și învățarea guvernată sunt decizii puternice de proiectare. Înțelegerea semantică generală, execuția completă a interpretării, planificarea interogării la scară de producție, generarea de forme lungi purtătoare de dovezi, mutația cuprinzătoare și validarea empirică independentă rămân deschise. Credibilitatea arhitecturii vine parțial din admiterea acestei limite.

Principala oportunitate științifică nu este de a adăuga mai multă DSL sintaxă. Este pentru a demonstra o somare semantică condiționată pentru un fragment delimitat, dar netrivial. Un rezultat publicabil ar putea specifica un limbaj document care conține entități, evenimente, relații temporale, cantități, modalități, excepții explicite și referințe alternative; să definească setul de interpretare admisă; să dovedească sau să testeze mecanic că fiecare rezultat publicat poate/trebuie să fie corect în raport cu setul respectiv; să demonstreze că rafinamentul simbolic elimină o clasă măsurabilă de avertismente false; și verifică explicațiile generate de păstrarea rezultatului canonic. Un astfel de rezultat ar fi mai îngust decât „engleza executată”, dar substanțial mai importantă.

Principala oportunitate practică este o nouă clasă de sisteme de documente și agenți care sunt dispuse să se oprească. Un asistent LLM normal este optimizat pentru a continua să producă un răspuns. O perioadă de rulare fundamentată simbolic poate reveni ontologia incompatibilă, identitatea nerezolvată, acoperirea insuficientă, formalismul nesusținut, epuizarea bugetară, contraexapsamentul fals sau realizarea fără licență. Aceste rezultate pot părea mai puțin impresionante într-o demonstrație, dar sunt premise pentru utilizarea fiabilă în domeniul științei, reglementării, ingineriei și luării deciziilor instituționale.

Prin urmare, judecata mea independentă este că nllAgent a ales multe dintre limitele arhitecturale corecte și ar trebui să reziste presiunii de a-și lărgi pretențiile mai repede decât își lărgeste dovezile semantice. Este deja un prototip credibil de analiză lingvistică guvernată, fundamentată în surse, separat de teorie. Nu este încă o dovadă că limbajul natural nerestricționat poate fi executat cu garanții formale. Cea mai bună cale de urmat este de a deveni o platformă experimentală pentru pretenția mai îngustă și mai apărabilă dezvoltată pe parcursul acestei cărți: LLMs descoperiți candidații formali, un nucleu simbolic controlează împământarea și consecințele, iar producția în limba naturală rămâne subordonată unui rezultat canonic rejucabil.

8. Construirea și validarea unui runtime neuro-simbolic

8.1 Strategie de implementare: un sistem de limbaj executabil minimal, dar onest științific

Arhitectura propusă în această carte este ambițioasă numai dacă este descrisă vag. Odată ce obligațiile sale sunt separate, primul sistem implementabil este relativ modest. Nu este nevoie să înțeleagă limbajul arbitrar, să construiască o ontologie universală sau să dovedească fiecare concluzie. Trebuie să accepte o clasă de documente delimitată, să păstreze dovezile exacte, să materializeze o mică familie de observații de tip tip, să compileze o familie restrânsă de reguli în unul sau două motoare formale, să returneze martori sau necunoscute explicite și să împiedice stratul final de limbă naturală să schimbe rezultatul simbolic. Valoarea științifică provine din a face fiecare limitare vizibilă și măsurabilă.

Prima decizie de proiectare este revendicarea țintă. Un prototip de cercetare nu ar trebui să înceapă cu „sistemul înțelege și execută limbajul natural”. Ar trebui să înceapă cu o afirmație precum: având în vedere documentele dintr-un gen declarat, o orologie prezentată, o familie finită de producători de observație și un set de reguli publicate, runtime-ul determină dacă fiecare regulă este satisfăcută, încălcată, potențial încălcată, nesusținută sau în afara acoperirii semantice a eliberării. Rezultatul este rejucabil și legat de întinderile sursei. Această afirmație este suficient de îngustă pentru a fi testată și suficient de largă pentru a sprijini revizuirea conformității, analiza cerințelor, informarea științifică, inspecția protocolului și planificarea controlată.

Stratul sursă trebuie să fie în mod deliberat plictisitor. Fiecare document este depozitat ca o revizuire imuabilă cu un digest stabil. Unități structurale – titluri, paragrafe, propoziții, elemente de listă, celule de masă, subtitrări, note de subsol și obiecte încorporate – concepățește identificatori persistenți. Normalizarea este mai degrabă înregistrată decât aplicată în tăcere. Compensările caracterului ar trebui să se refere la o reprezentare a sursei canonice, în timp ce cartografierea păstrează pozițiile în fișierul original. O declarație dintr-o tabelă și o propoziție identică vizual într-un paragraf pot avea roluri structurale diferite; această distincție trebuie să supraviețuiască extracției. Când OCR nu este evitabilă, ieșirea OCR și imaginea paginii sunt obiecte de probă separate, iar incertitudinea din transcrierea nu este prăbușită în textul obișnuit.

Această disciplină sursă nu este în mod administrativ. Este ceea ce contestă concluziile simbolice. O constatare care citează doar un rezumat generat nu poate fi verificată independent. O constatare legată de text, celule de masă și înregistrări de transformare poate fi revizuită chiar dacă fiecare model de limbă implicat s-a schimbat. Prin urmare, pachetul sursă ar trebui să fie considerat parte a înregistrării de audit de încredere, deși extractorii care îl creează rămân failibili și necesită teste specifice formatului.

Al doilea strat este mai degrabă un grafic de probă decât o singură parsă. Graficul conține mențiuni, entități, evenimente, stări, cantități, expresii temporale, modalitate, relații și legături sursă. În mod crucial, reprezintă, de asemenea, alternative și statut. O legătură cu o entitate propusă de model nu este stocată ca o egalitate necondiționată. Este o relație de identitate a candidaților cu un producător, scorul, intervalele de susținere, dovezile conflictuale și un statut epistemic, cum ar fi propus, coroborat, acceptat, respins sau nerezolvat. O relație temporală

dedusă de la „la scurt timp după” nu trebuie normalizată la o durată exactă arbitrară; ar trebui să păstreze relația calitativă și orice interpretare declarată a domeniului.

Acest strat trebuie să rămână finită și legat de sarcini. Sistemul nu construiește un model mondial complet. Se materializează distincțiile cerute de circuitele publicate și de interogarea actuală. Materializarea cu primul caz, deja explorată în nllAgent, este implicitul corect. Un circuit declară că are nevoie de identitate de actor, de relații cu documentele, demodare nedobândită și de date eficiente. Compilatorul derivă aceste cereri înainte de o interpretare costisitoare. Producătorii generează apoi numai observațiile necesare pentru a decide compatibilitatea și a executa regula. Acest lucru reduce costurile și, mai important, limitează suprafața semantică pe care pot apărea erori.

Al treilea strat este un sâmbure simbolic de împământare. „Lăsarea” este adesea folosită vag pentru a însemna că un răspuns citează un pasaj. Aici are o semnificație mai strictă: fiecare simbol admis la executarea formală are un referent tipizat, o proveniență, o domeniu de aplicare și un statut suficient pentru formalismul țintă. Un predicat ca $\text{aprobat}(x, y, t)$ nu poate fi introdus doar pentru că un LLM a produs aceste jetoane. Sâmbetei verifică x și y se referă la tipurile de entități admisibile, că este o valoare sau un interval de timp susținut, încât producătorul de observație este autorizat pentru această predicat, că sursa există în revizuirea declarată și că statutul îndeplinește cerința de garantare a regulii.

Contractele de fundament ar trebui să distingă dovezile de ontologie. Dovezile spun că un interval susține o observație. Ontologia spune ce tip de lucru notează observația și ce relații sunt admisibile. Axiomurile domeniului spun ce rezultă din aceste relații. Cunoștințele lumii furnizează afirmații din surse externe. Aceste straturi nu trebuie îmbinate. Un registru al companiei poate stabili că două nume denotă o persoană juridică într-o perioadă de timp declarată. O oncologie de domeniu poate declara că persoanele juridice pot semna acorduri. O teorie a politicii poate declara că o semnătură a unui reprezentant autorizat contează ca aprobare. Un model poate sugera toate cele trei conexiuni, dar fiecare conexiune aparține unei autorități diferite și ar trebui să fie acceptată sau respinsă separat.

Identitatea își merită propriul serviciu pentru că este cea mai comună dependență ascunsă în raționamentul pe termen lung. Serviciul ar trebui să gestioneze grupurile de menționare, pseudonimele explicite, identificatorii, rolurile organizaționale, validitatea temporală și lumile selectate. Ar trebui să permită mai multor candidați nerezolvați, mai degrabă decât să forțeze unificarea timpurie. Într-un prim comunicat, identitatea poate fi în mod intenționat conservatoare: sunt acceptate identificatorii expliți și pseudonimele exacte declarate; legăturile bazate pe roluri și contextuale rămân propuse; identitatea încrucișată nesusținută oprește normele care o impun. Această politică conservatoare va reduce acoperirea aparentă, dar va îmbunătăți considerabil sensul rezultatelor acceptate.

Timpul are nevoie de un domeniu la fel de explicit. O reprezentare inițială practică poate combina instantanee exacte, intervale închise sau deschise, ordine calitativă și limite necunoscute. Durata de funcționare trebuie să distingă ordinea documentului de ordinul evenimentului și timpul de publicare de la ora efectivă. Frazele temporale, cum ar fi „de înainte de desfășurare”, „în termen de treizeci de zile”, „în timp ce autorizația rămâne valabilă” și „după orice schimbare materială” induc structuri diferite. Compilatorul ar trebui să respingă o regulă temporală în cazul în care graficul de probă are doar observații document-comandă atunci când este necesară semantica timpului de eveniment. Substituirea silențioasă între domeniile temporale este o sursă clasică de concluzii plauzibile, dar nevalide.

Modalitatea și forța normativă necesită o mică zăbrele, mai degrabă decât un boolean. Cel puțin, sistemul ar trebui să distingă afirmația, raportul, credința, posibilitatea, capacitatea, permisiunea, obligația, interdicția, recomandarea, intenția, condiția și combaterea factuală. De asemenea, ar trebui să înregistreze titularul și beneficiarul unei norme, dacă este cazul. „Furnizorul poate dezvălui date” și „furnizorul trebuie să dezvăluie datele” în vocabularul de partajare, dar are consecințe opuse conformării. „Documentul afirmă că furnizorul s-a conformat” este o dovadă a unui raport, nu neapărat dovezi ale evenimentului de bază. O durată formală devine utilă tocmai prin refuzul acestor prăbușiri.

Odată ce există observații la sol, al patrulea strat calculează o stare semantică abstractă. Un domeniu minimal poate reprezenta, pentru fiecare propunere relevantă, dacă este adevărată în toate interpretările admisibile, adevărată în unii, falsă în toate, contrazisă sau nerezolvată din cauza lipsei de acoperire. Implementarea nu trebuie să înceapă cu o bibliotecă sofisticată de zăbrele. Poate începe cu alternative finite explicite și un rezumat derivat din punct de vedere mecanic poate/trebuie. Pe măsură ce scara crește, domeniile abstracte pot rezuma candidații identității, intervalele numerice, intervalele temporale, statusurile modale și seturile de proveniență fără a enumera fiecare lume.

Statul abstract îndeplinește două locuri de muncă. În primul rând, oferă o propagare conservatoare la nivel de document. Dacă fiecare interpretare acceptată leagă o definiție de un termen, această relație este un fapt obligatoriu. Dacă doar una dintre cele trei misiuni de identitate plauzibile creează un conflict de interese, conflictul este mai degrabă o constatare decât o încălcare verificată. În al doilea rând, decide unde este justificată execuția mai scumpă. Un rezultat must-safe se poate termina devreme. Un rezultat de încălcare declanșează o căutare a martorilor. O necunoscută cauzată de acoperirea absentă a producătorilor declanșează o problemă de compatibilitate semantică, mai degrabă decât un apel de rezolvare.

Al cincilea strat este un plan de formalism. Planul nu este proza în formă liberă scrisă de un agent. Este o decizie de compilație tastată care afirmă ce întrebare va fi adresată de ce motor, ce observații sunt intrările, ce garantează randamentele motorului și modul în care rezultatele sale se rotesc la vocabularul rezultatului canonic. Alegerea ar trebui să fie condusă de o formă semantică, mai degrabă decât de modă.

Evaluarea relativă este primul motor pe care trebuie să îl implementeze, deoarece multe verificări utile ale documentelor se reduc la adrese, compararea comenzilor, grupări și diferențele stabilite. Reguli precum „fiecare termen tehnic definit este utilizat în mod consecvent”, „fiecare revendicare acceptată are cel puțin o sursă”, „niciun element de identificare al cerinței nu este duplicat” sau „toate riscurile enumerate au un proprietar de atenuare” nu necesită o dovadă a antecedentelor. Un nucleu relațional este transparent, eficient și ușor de testat. Datalog extinde acest nucleu atunci când sunt necesare închideri tranzitive sau dependențe recursive. Exemplele includ moștenirea, propagarea integrală parțială, lanțurile de dependență și accesibilitatea prin intermediul graficelor de citare sau cerințe.

Tabelele de decizie sunt al doilea formalism util pentru politicile legate delimitate. Un tabel de decizie face combinații de condiții și rezultate explicite, expune suprapunerea sau lipsa rândurilor și susține analiza acoperirii. Este deosebit de potrivit pentru regulile de eligibilitate, rutare, clasificare și politici a căror complexitate provine mai degrabă din combinații decât din recurs. Un circuit poate compila o politică în limba naturală într-o tabelă a candidatului, dar publicarea ar trebui să solicite exemple la nivel de rând, detectarea suprapunerii, verificările complete a completării și aprobarea umană a partiției semantice.

Motoarele SAT, SMT, și MaxSAT, devin utile atunci când sarcina conține constrângeri interacționante, aritmetică, programări, sarcini sau optimizare. Compilerul trebuie să aleagă teoriile susținute și să evite ascunderea semanticii nesusținute în corzi neinterpretate. Un rezultat SMT poate dovedi infecțiozitatea în cadrul constrângerilor codificate, poate produce un model satisfăcător sau poate expune un nucleu nesatisfăcător. Nucleul ar trebui să fie tras înapoi la obligațiile sursă și ipotezele. Această cartografiere este mai informativă decât să ceri unui LLM să explice o eroare de rezolvare generică; identifică cele mai mici angajamente formale care nu pot deține împreună.

Logica temporală și verificarea modelului sunt adecvate pentru proceduri, fluxuri de lucru, protocoale și reguli privind ciclul de viață. Documentul de probă furnizează evenimente și propuneri de stat; compilerul construiește un sistem de tranziție și o proprietate temporală; verificatorul returnează satisfacția, o urmă de contraexempl sau un rezultat neconcludent în limite. Partea dificilă nu este verificatorul temporal. Se decide dacă sursa definește un sistem de tranziție complet, indiferent dacă tranzițiile omise sunt imposibile sau doar fără documente și dacă corectitudinea sau ipotezele privind mediul sunt justificate. Prin urmare, rezultatul canonic trebuie să expună limita modelului.

Execuția simbolică este adecvată atunci când o regulă se referă la căi fezabile prin cod executabil, o procedură de stat sau un artefact semantic asemănător programului. Ieșirea sa ar trebui să fie un martor concret: misiuni, condiții de ramură, tranziții de stat și legături sursă. În setările numai în documente, execuția simbolică poate explora legănări alternative sau ramuri procedurale, dar nu ar trebui invocată doar pentru că sistemul conține simboluri. Termenul ar trebui să-și păstreze sensul sensibil la cale. SymGPT este instructivă, deoarece regula în limba naturală este compilată într-o stare de încălcare și apoi testată împotriva căilor contractuale reale, mai degrabă decât împotriva prozei [Xia et al. 2026] mai generate.

Logica naturală, automatele finite, parsarea diagramelor, calculul semi-aprindere și proverele de teoremă specializate aparțin portofoliului atunci când ipotezele lor structurale se potrivesc. Logica naturală poate gestiona eficient monotonicitatea și anumite implicări lexicale în apropierea limbajului de suprafață. Automatele sunt excelente pentru modelele locale ordonate. Metodele de diagramă și semi-aderare pot menține mai multe măsuri cu calculul comun. Asistenții de probă sunt potriviți atunci când un număr mic de revendicări de mare valoare justifică dovezile oficiale manuale și verificarea nucleului. Arhitectura ar trebui să facă din adăugarea unui motor o extensie de tipare, nu o invitație de a încorpora codul executabil arbitrar în circuite.

Cel de-al șaselea strat este rafinamentul semantic ghidat de contraexapetie. O constatare poate fi alcătuită într-o problemă de fezabilitate. Dacă dispozitivul răspunde un martor, runtime-ul îl redă împotriva graficului de probe și raportează alegerile exacte de interpretare necesare. Dacă calea este infestabilă, motorul returnează un nucleu nesatisfăcător, o precondiție eșuată sau un set de conflicte. Sistemul urmărește acest obiect înapoi prin compilare la observațiile sursă și cartografierea candidaților care l-au produs. Numai atunci este un LLM rugat să ajute.

Solicitarea de reparație trebuie să fie locală și să fie tastată. Acesta poate întreba dacă două mențiuni au fost unificate incorect, dacă o condiție a fost intenționată ca necesară sau suficientă, dacă se aplică o excepție sau dacă o relație temporală are un criteriu final greșit. Ar trebui să prezinte sursele relevante, contractele de ontologie, alternativele actuale și conflictul simbolic. LLM propune unul sau mai multe artefacte semantice revizuite. Aceste propuneri trec

prin aceleași verificări de împământare și compilare ca și originalele. Un om poate fi inserat atunci când ambiguitatea este material normativ sau științific.

Acest proces este fundamental diferit de auto-reflecția generică. Modelul nu i se spune că răspunsul său a fost greșit și invitat să improvizeze un alt lanț de gândire. Acesta este dat un conflict identificat mecanic într-un artefact semantic persistent. Reparația poate fi testată în regresie. Dacă reparația schimbă o ontologie sau un producător, publicarea necesită noi cazuri de referință și un diff semantic. Contraexemplu-ul devine o parte permanentă a corpului de evaluare.

Cel de-al șaptelea strat este rezultatul canonic. Ar trebui să fie un produs stabil, independent de rezolvare, care separă verdictul, dovezile, ipotezele, interpretările, execuția și limitările. Pentru o regulă de audit, acesta înregistrează versiunea de regulă, domeniul de aplicare aplicabil, starea de compatibilitate, acoperirea, observațiile acceptate, observațiile respinse, rezultatul execuției, referința martorului sau a dovezilor, rezultatul verficatorului și problemele nerezolvate. Pentru un plan, înregistrează obiective, constrângeri, pași, dependențe, resurse, alternative și verificările exacte pe care planul le satisface. Produsul canonic este autoritatea pentru orice răspuns ulterior în limba naturală.

Calea de întoarcere spre limbă trebuie tratată ca o compilație, nu ca o generație liberă. Un randator primește un set finit de revendicări licentiate. Fiecare afirmație are modalitate, polaritate, cantitate, referințe de entitate, incertitudine și proveniență. Randamentul poate alege ordinea discursului și fraza explicativă, dar nu poate introduce noi entități, nu poate fi întărit să trebuiască, să convertească dovezile lipsă în absență, să suprimă o presupunere materială sau să inventeze o explicație causală. Identificatorii de cerere la nivel de sentință ar trebui să rămână disponibili pentru audit. Un al doilea verficator ar trebui să compare textul redat cu rezultatul canonic, folosind teste deterministe pentru nume, cantități, date, negare și modalitate și, dacă este necesar, un model separat constrâns la implicarea textuală, mai degrabă decât la evaluarea deschisă.

Realizarea inversă este deosebit de importantă, deoarece utilizatorii vor judeca sistemul după proza sa. O urmă simbolică perfectă poate fi subminată printr-un rezumat elegant, dar inexact. Prin urmare, arhitectura ar trebui să facă obligatorie verbozitatea opțională, dar fidelitatea. În modul de mare presiune, sistemul poate genera un limbaj clar, repetitiv, deoarece fiecare propoziție urmează un șablon licențiat de tipul de rezultat. În modul explicativ, un LLM poate îmbunătăți lizibilitatea, dar sentințele nesustținute sunt respinse și regenerate. Artefactul canonic, nu proza, rămâne recordul deciziei.

Securitatea și guvernarea trebuie integrate de la început. Documentele în limba naturală sunt intrările neîncredințate și pot conține injecție promptă, exemple contradictorii, formatarea înșelătoare sau fragmente de cod. Producătorii semantic ar trebui să primească date sursă prin interfețe de tipărire, mai degrabă decât o solicitare privilegiată a sistemului care poate fi suprascrisă prin conținutul documentului. Artefactele formale generate trebuie să fie analizate de gramatici restrictive. Rezolvatorii ar trebui să se bazeze pe limitele resurselor. Extensiile de alergare ar trebui revizuite, versiuni și încuiate în digerare. Locurile de muncă de învățare nu ar trebui să publice direct la producție. Aceste limite sunt deja parțial prezente în nllAgent și ar trebui consolidate, mai degrabă decât ocolite pentru comoditate [AssistOS-AI 2026].

Un program de implementare realist poate continua în cinci comunicate. Prima versiune ar trebui să susțină marca de marca video sau ingestia DOCX, întinderi de surse, observații structurale, un motor de interogare relațională, observații controlate restricționate în limba

engleză, stări de acoperire, circuite reutilizabile, verificarea reluării și constatări canonice. Aceasta ar trebui să rezolve sarcinile limitate, cum ar fi caracterul complet de proveniență, coerența de definire, acoperirea cerințelor, regulile secțiunii ordonate și distincțiile modale simple. Nu trebuie revendicată nicio identitate generală, deducție temporală sau de recuperare a lumii actuale.

A doua versiune ar trebui să adauge identitate și timp tastat. Identitatea începe cu identificatori, pseudonime și roluri redactate de documente; semantica temporală începe cu date exacte, intervale, ordine de document și un set mic de modele de ordine de eveniment. Reperele ar trebui să conțină pseudonime ambigue, schimbarea rolurilor, suprapunerea intervalelor și pierderea deliberată a legăturilor. Sistemul ar trebui să se oprească, mai degrabă decât să ghicească când un circuit necesită o identitate mai puternică decât oferă eliberarea.

A treia eliberare ar trebui să adauge tabele de decizie și SMT/CSP, cu cartografiere insatisfăcută a nucleelor și rafinament semantic ghidat de contraexaple. Această eliberare poate aborda politicile de eligibilitate, programele, constrângerile de atribuire, coerența cantitativă și fezabilitatea procedurii degradate. Fiecare rezultat rezolvator trebuie reluat printr-un verficator independent sau un verficator mai mic, acolo unde este posibil. Testarea mutației ar trebui să modifice constrângerile și observațiile pentru a demonstra că suitele de referință detectează regresii semantice.

A patra versiune ar trebui să adauge un adaptor temporal/de verificare a modelului și un adaptor de execuție simbolic sensibil la cale pentru un domeniu concret, cum ar fi specificațiile fluxului de lucru sau regulile contractului inteligent. Scopul nu este o acoperire largă, ci dovada că registrul formalismului și interfața certificatului canonic suportă semantica cu adevărat diferită. Testele de coerență a formelor încrucișate ar trebui să verifice faptul că faptele comune bazate păstrează identitatea, timpul, modalitatea și proveniența în jos.

A cincea versiune ar trebui să adauge planificare și realizare controlată. O idee este compilată într-un plan canonic, verificată împotriva constrângerilor formale, realizată în proză și auditată de același sistem de reguli. Evaluarea trebuie să distingă validitatea planului, fidelitatea realizării, calitatea lingvistică și eficiența reparației. Planul nu devine de încredere doar pentru că proza a fost generată de un obiect structurat; ea devine mai auditabilă, deoarece obiectul și verificările sale sunt persistente.

Artifact stack for a replayable executable-language runtime

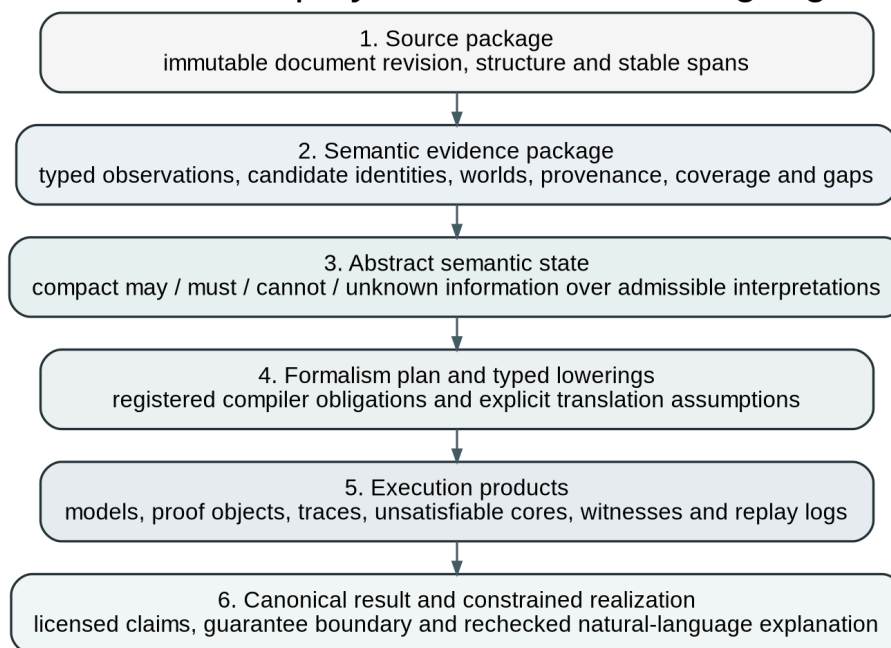


Figura 4. Stive de artă pentru o perioadă de rulare reîncărcabilă executabilă-înflorire-naturală. Fiecare strat este vopsit, mapat cu sursă și inspectabil independent.

Această punere în aplicare înscenată întruchipează un principiu metodologic important: ambiția semantică ar trebui să crească numai atunci când sistemul poate detecta că reprezentarea sa anterioară este inadecvată. Timpul de alergare nu ar trebui să ascundă limbajul nesușținut în spatele încrederii modelului. Ar trebui să expună decalajele de compatibilitate ca rezultate de primă clasă și să transforme decalajele recurente în noi producători, ontologie, motor și lucrări de referință. Limbajul natural executabil avansează prin expansiunea guvernată a unui plic de sos, nu printr-un singur salt la înțelegerea universală.

8.2 Întrebări de cercetare, benchmarkuri și un program de publicare

Arhitectura devine o contribuție științifică numai atunci când pretențiile sale sunt de neînțelegere. O demonstrație în care un LLM generează un artefact formal plauzibil și un rezolvator returnează un răspuns este insuficientă. Același model poate genera indiciile de referință, traducerea și explicația, creând o buclă închisă de eroare corelată. O evaluare serioasă trebuie să descompună conducta și să testeze fiecare obligație în mod independent. Aceasta trebuie să includă, de asemenea, condițiile de eșec în care abordarea propusă ar trebui respinsă sau restrânsă.

Prima întrebare de cercetare se referă la reprezentare: menținerea mai multor interpretări semantice admisibile îmbunătățește fiabilitatea față de selectarea unei traduceri? Ipoteza corespunzătoare este că semantica setată de candidați, combinată cu analiza may/must, reduce asigurarea falsă pe intrările ambigue și subspecificate. Poate reduce rata aparentă a răspunsului, deoarece mai multe cazuri devin condiționate sau necunoscute. Acesta nu este neapărat un defect. Compararea relevantă măsoară dacă sunt acceptate concluziile sunt mai des corecte și dacă constatările pot capta alternative plauzibile din punct de vedere material fără recenzori copleșitori.

Un punct de referință pentru această întrebare ar trebui să conțină texte cu seturi de interpretare adnotate de experți, mai degrabă decât o formă logică de aur. Perechile minime ar trebui să dirijeze domeniul de aplicare cuantificator, negarea, atașamentul de excepție, coordonarea, coreferența, limitele temporale, forța modală și jurisdicția. Unele elemente ar trebui să aibă o lectură intenționată în mod clar, unele mai multe lecturi legitime și unele dovezi insuficiente pentru orice răspuns hotărât. Sistemele sunt evaluate pe baza acoperirii citirilor admisibile, excluderea citirilor inadmisibile, calibrarea statutului de mai/tâmplă și verdictele din aval. Meciul grafic exact este prea fragil; echivalența semantică trebuie testată prin suite de întrebare sau impenetrare bidimensionale sub ontologia de referință.

A doua întrebare de cercetare se referă la rafinament: conflictele derivate din rezolvatoare sunt mai utile pentru repararea formalizării decât autocritica modelului generic? Ipoteza este că contraexapellările, nucleele insasifibile și obligațiile antiepuizante produc reparații mai localizate, stabile și mai eficiente din punct de vedere al probei, deoarece identifică dependența responsabilă pentru rezultat. Un experiment controlat ar trebui să compare patru condiții: rejudecarea directă, auto-reflecție verbală, feedback-ul erorii de execuție și rafinamentul semantic ghidat de contraexapel cu cartografierea surselor. Același model, temperatura și bugetul ar trebui să fie utilizate acolo unde este posibil.

Calitatea reparației trebuie măsurată nu numai prin faptul dacă exemplul țintă devine corect. Reparația trebuie să păstreze cazurile fără legătură, să reducă diff-ul semantic și să evite introducerea ipotezelor nesustținute. Suitele de regresie bazate pe mutații sunt esențiale. O reparație care rezolvă un exemplu prin slăbirea regulii la nivel global nu este un succes. Experimentul ar trebui să înregistreze numărul de iterații, jetoane, intervenții umane, mutanți supraviețuitori, observații schimbate și dacă artefactul final a fost rejucat în mod independent.

Cea de-a treia întrebare de cercetare se referă la contractul de probă. O separare de tipare a surselor, observațiilor, afirmațiilor ontologice, axiome de domeniu și spațiile cunoașterii lumii îmbunătățesc transferul și australierea în comparație cu un grafic plat de cunoștințe sau un lanț de gândire în formă liberă? Ipoteza este că contractele de tipare reduc introducerea premiselor

silențioase și fac reutilizarea cross-domain mai previzibilă. Costul probabil este de a se scrie deasupra capului și mai multe run-uri oprite.

Etajul ar trebui să includă aceeași regulă aplicată în cadrul documentelor din mai multe domenii, cu variație controlată a vocabularului și a ontologiei. De exemplu, o cerință generală ca fiecare afirmație acceptată să aibă dovezi care pot fi urmărite pe documente de cercetare, documente de politică, specificații tehnice și planuri. Observațiile structurale pot transfera direct; entitatea specifică domeniului și producătorii de relații nu pot să se efectueze. Evaluarea înregistrează ce verificări de compatibilitate opresc, dacă motivul de oprire este corect și cât de mult este necesar un nou material semantic pentru a relua executarea. Transferul de succes nu este doar obținerea unui răspuns; recunoaște cu exactitate ceea ce poate și nu poate fi reutilizat.

Cea de-a patra întrebare privind cercetarea se referă la alegerea formalismului. Rutarea către un portofoliu de motoare specializate depășește un limbaj intermediar universal sub supraveghere semantică egală? Beiser și Penz arată că alegerea în limba intermediară poate modifica în mod substanțial precizia generală și a execuția în mod substanțial pe sarcini [Beiser and Penz 2025] de raționament neuro-simbolică. Ipoteza aici este mai specifică: un router de tip care selectează cel mai simplu formalism adecvat va îmbunătăți succesul complicării, calitatea de explicație și eficiența fără a sacrifica consistența încrucișată.

O comparație semnificativă nu ar trebui să permită sistemului de portofoliu mai multe informații ascunse. Fiecare sarcină primește aceleași dovezi și ontologie sursă. Valoarea de bază a unui singur formalism ar putea utiliza logica de prim ordin, ASP, SMT, sau un limbaj general de interogare grafică. Sistemul de portofoliu poate alege interogări relaționale, Datalog, tabele de decizie, SMTZ, logica temporală sau execuția simbolică. Valorile includ validitatea formalizării, echivalența semantică, succesul rezolvatorului, timpul de rulare, calitatea martorilor și costul de întreținere. Cazurile de formalism încrucișat ar trebui să necesite două motoare și să testeze dacă simbolurile partajate rămân consecvente.

Experimentul ar trebui să caute, de asemenea, contravidențe. O reprezentare universală poate depăși portofoliul în domenii înguste, deoarece rutarea introduce erori și mai mulți compilatori cresc baza de încredere. Concluzia așteptată nu este că portofoliile sunt întotdeauna superioare. Este faptul că formalismul este justificat numai atunci când distribuția sarcinilor conține cu adevărat structuri de calcul diferite și atunci când interfețele păstrează semantica.

Cea de-a cincea întrebare de cercetare se referă la stratul de limbaj final. Realizarea constrânsă de dovezi reduce pretențiile nesusținute sau consolidate semantic, păstrând în același timp lizibilitatea acceptabilă? Ipoteza este că un rezultat canonic, plus generarea autorizată de pretenții, scade substanțial halucinația și derivarea modalității relative la generarea directă a răspunsului sau generarea de răspuns de la o urmă de rezolvare brută. Costul poate fi mai puțină proză naturală și mai multă regenerare.

Indicele de referință ar trebui să includă rezultate canonice cu incertitudine subtilă, dovezi contradictorii, condiții cuibărite, cantități și date. Evaluatorii umani consideră fidelitatea factuală, caracterul complet al limitărilor materiale, lizibilitatea și utilitatea. Verificatorii determinați măsoară entitatea, numărul, data, polaritatea și conservarea modalităților. Un set contradictoriu ar trebui să tenteze redarea să producă o concluzie mai puternică, mai simplă decât susține rezultatul. Meseria centrală nu este preferința stilistică, ci rata cu care proza depășește autoritatea produsului simbolic.

Cea de-a șasea întrebare de cercetare se referă la arhitectura federativă în sine. Revizuirea sistematică din 2026 a neuro-simpolim NLP raportează câștiguri promițătoare pentru sistemele federative, dar avertizează că dovezile sunt confuze de diferențele [Chatzikyriakidis and Lappin 2026] de sarcini. Un experiment direct ar trebui să compare o conductă federativă cu o alternativă injectabilă sau end-to-end învățată pe aceleași sarcini, date și scară de model. Beneficiile relevante pot apărea mai puțin în acuratețe brută decât în ceea ce privește auditabilitatea, abținerea, localizarea erorilor și repararea erorilor.

Prin urmare, studiul trebuie să raporteze mai multe dimensiuni. Precizia sarcinii brute rămâne necesară. Acesta trebuie să fie însoțit de o precizie semantică, precizie condiționată de rezolvare, calibrare necunoscută, reproductibilitate pe modele, fidelitatea explicației, timpul de revizuire umană și fracțiunea erorilor care pot fi atribuite extracției, împământării, compilației, execuției sau realizării. Un sistem modular care se potrivește mai degrabă decât bate precizia end-to-end poate fi încă de preferat pentru munca de înaltă asigurare dacă transformă erorile silențioase în artefacte diagnosticabile. În schimb, modularitatea nu ar trebui să fie apărută ca o virtute intrinsecă dacă adaugă doar birocrăție fără a îmbunătăți dovezile.

O a șaptea întrebare de cercetare se referă la interpretarea abstractă asupra alternativelor semantice. Poate fi definit un domeniu abstract util care este conservator în raport cu o interpretare de referință stabilită și mai scalabil decât enumerarea explicită? Primele experimente pot folosi seturile finite ca referință concretă. Domeniile abstracte pentru identitate, intervale temporale, modalități, cantități și acoperire sunt implementate separat și apoi combinate. Sunetismul este verificat mecanic împotriva enumerării în cazuri mici. Precizia este măsurată prin rata constatărilor care pot fi eliminate prin căutare sau rafinare a martorilor.

Acest program ar trebui să fie atent cu sunetul de cuvinte. Dacă adnotarea de specialitate definește un set de interpretare finit, interpretul abstract poate fi solid în raport cu acel set. Prin urmare, nu poate dovedi că setul epuizează sensul uman. Contribuția este o semantică condiționată executabilă și dovezi despre utilitatea sa. O lucrare teoretică ulterioară poate defini funcțiile de reabilitare, abstractizarea și funcțiile de compresie, condițiile de transfer și operatorii de rafinare. Sistemul empiric nu ar trebui să aștepte o semantică completă a limbajului, dar ar trebui să facă explicită natura condiționată a afirmațiilor sale.

O a opta întrebare de cercetare se referă la execuția simbolică asupra programelor semantice. Construcțiile martorilor sensibili la cale îmbunătățesc înțelegerea recenzorului și dezertează descoperirea în raport cu rezultatele statice ale regulilor? Într-un domeniu al fluxului de lucru sau de politică, runtime-ul poate compara un simplu evaluator de reguli cu explorarea simbolică a căilor. Evaluatorul spune că este posibilă o încălcare; executorul returnează o secvență concretă de evenimente și misiuni. Recenzorii umani judecă dacă martorul este inteligibil și util. Defectele adevărului de la sol și mutații însămânțați determină dacă martorul corespunde unui caz real.

Explozia căii trebuie să facă parte din evaluare, mai degrabă decât să fie tratată ca un inconvenient de punere în aplicare. Studiul ar trebui să dirijeze ramificarea, limitele buclei, alternativele identitare și alegerile temporale. Ghidul LLM poate prioritiza căile, așa cum sugerează lucrările recente LLM de execuție simbolică, dar motorul trebuie să distingă acoperirea euristică de caracterul formal al completitudinii. Valorile de acoperire ar trebui să raporteze statele explorate, stările tăiate, timeout-urile rezolvatoare și comportamentul extern nesuținut. Un martor este o dovadă puternică a existenței; negăsirea unuia nu este o dovadă a absenței decât dacă explorarea este legată și modelul justifică această concluzie.

Arhitectura de referință ar trebui să fie stratificată. Stratul sursă testează ingestia și ancorarea. Stratul de observare testează extracția și statutul. Stratul de împământare testează identitatea, timpul, modalitatea, cantitățile și autoritatea. Stratul de formalizare testează echivalența semantică și valabilitatea în limba țintă. Stratul de execuție testează ieșirile și certificatele de rezolvare. Stratul de rezultat testează compoziția verdictului și incertitudinea. Stratul de realizare testează fidelitatea răspunsurilor în limba naturală. Fiecare exemplu ar trebui să identifice cel mai timpuriu strat la care ar trebui detectată o defecțiune injectată.

Cazurile de referință ar trebui să fie suficient de mici pentru a fi validate manual și composabile în documente mai lungi. Exemplele mici oferă un adevăr de încredere pentru logica subtilă. Compozitele lungi testează recuperarea, indexarea, continuitatea identității și scalabilitatea. Aceeași regulă de bază poate apărea în mai multe forme lingvistice și structuri de documente. Familiile parafrazelor testează invariația; contrastul stabilește schimbarea unei caracteristici semantice și sensibilitatea la test. Indicele de referință trebuie să evite eșecul comun în care un sistem funcționează bine prin memorarea modelelor de suprafață, ignorând în același timp distincția dorită.

Testarea mutantă ar trebui să fie centrală. Mutațiile pot schimba un cuantificator, pot nega o condiție, pot muta o excepție, pot modifica o limită de dată, pot fuziona identitățile, pot șterge probele, pot schimba forța de schimbare, schimbă rolul de actor și obiect, pot duplica o cerință sau modificați un prag de circuit. O suită robustă ar trebui să detecteze schimbarea preconizată și să lase rezultatele fără legătură stabilă. Scorul de mutație nu este o măsură completă de calitate, dar dezvăluie teste care doar confirmă o cale fericită. Pentru nllAgent, mutațiile executabile ar aborda direct unul dintre lacunele ZXC00000QXZ recunoscute ale depozitului.

Evaluarea trebuie să includă injecție promptă și adversari semantici. Documentele pot conține instrucțiuni îndreptate spre model, titluri înșelătoare, text politic citat, Unicode contradictorial, conținut ascuns sau declarații care seamănă cu declarațiile de ontologie. Spectrul sursă trebuie să le păstreze ca date. Producătorii semantic ar trebui să opereze sub o autoritate de sistem fix. Indicele de referință ar trebui să testeze dacă textul documentului poate modifica circuitul, poate suprima constatările sau poate provoca apeluri de instrumente privilegiate. Rezultatele de securitate ar trebui înregistrate separat de acuratețea semantică.

Evaluarea de tip live-model trebuie să fie suficient de reproductibilă pentru a susține afirmațiile științifice. Fiecare alergare ar trebui să dea dovadă, identificatorul modelului, să solicite schema, temperatura, semințele atunci când sunt acceptate, construcția de context și politica de rejudecare. Intrările și ieșirile trebuie să fie vizate de constrângerile legate de confidențialitate. Studiile repetate estimează variația. Dublurile deterministe rămân utile pentru testele software, dar nu pot stabili calitatea traducerii. Cel puțin două familii model ar trebui să fie testate acolo unde este posibil să distingă efectele arhitecturale de comportamentul unui furnizor.

Evaluarea umană este necesară, dar trebuie proiectată în mod restrâns. Experții nu ar trebui să fie întrebați dacă un răspuns „arată bine”. Acestea ar trebui să judece obligații specifice: dacă formalizarea păstrează sursa, dacă identitățile sunt justificate, dacă un martor este fezabil, dacă explicația prevede toate ipotezele materiale și dacă un necunoscut este returnat în mod corespunzător. Dezacordul inter-rater este în sine o dovadă a ambiguității semantice și nu ar trebui să fie forțată într-o singură etichetă de aur fără analiză.

Valorile cantitative de bază ar trebui să includă acuratețea sursei-ancoră; precizia observării și rechemarea prin statut; ontologie și validitate de tip; precizie de identitate-link, rechemare și

calibrare; precizia temporală și morală; acoperirea setului candidaților; soneria de observare în raport cu interpretările adnotate; formalizarea sincrapența și echivalența semantică; succesul de rezolvare; validitatea martorilor; succes de verificare a dovezilor; rata de asigurare falsă; abție adecvată; acoperire; Un singur scor agregat ar ascunde mecanismul și ar trebui să fie secundar.

Asigurarea falsă merită un accent deosebit. Acesta măsoară cazurile în care sistemul prezintă o concluzie verificată determinată, deși sursa, fundamentul sau setul de interpretare nu o justifică. În aplicațiile de mare presiune, reducerea asigurării false poate conta mai mult decât maximizarea întrebărilor la răspuns. Sistemul ar trebui să primească credit pentru o stare oprită precisă atunci când un LLM direct inventează cu încredere un pod. Adecvarea necunoscută este un rezultat pozitiv.

Dezechilibrele ar trebui să includă răspunsul direct LLM, îndemnul lanțului de gândire sau raționamentul echivalent, execuția augmentată LLM fără artefacte formale persistente, traducerea unui singur formalism, traducerea setului de candidați fără interpretare abstractă, execuția simbolică fără rafinament semantic și arhitectura completă. Pentru studiul de caz nllAgent, eliberarea fundației de bandă actualizată curentă poate servi ca o bază de bază pusă în aplicare, în timp ce identitatea propusă, semantica abstractă și extensiile multi-formalism sunt evaluate din ce în ce mai mult. Acest lucru evită compararea unei viziuni complete de cercetare numai împotriva sistemelor externe mai slabe.

Programul de publicare poate fi organizat în jur de cinci contribuții, mai degrabă decât o afirmație supradimensionată. Prima lucrare ar trebui să fie o lucrare de poziție și arhitectură care să definească limbajul natural executabil, limita de încredere, decalajul de formalizare, plicul de sonorizare și designul multiformalism. Contribuția sa este claritatea conceptuală și o arhitectură de referință, susținută de un studiu de sinteză sistematic și de un mic demonstrativ de lucru.

Cea de-a doua lucrare ar trebui să oficializeze pivotul semantic purtător de proveniență și soliditatea condiționată. Acesta definește obiectele sursă, observațiile de tip, interpretările candidaților, pot/trebuie semantica și compatibilitatea. Dovedește soliditatea calculului abstract în raport cu un set de interpretare comestibilă finită și raportează experimente pe identitate, timp, modalitate și acoperire. Acesta este nucleul de programare-limbă și formal-semantică.

Cea de-a treia lucrare ar trebui să introducă o rafinare semantică ghidată de contraexplicații. Acesta definește cartografierea de la conflictele de rezolvare la angajamentele semantice legate de sursă și compară rafinamentul cu auto-reflecție și feedback-ul de execuție generică. Cel mai puternic rezultat ar fi o asigurare falsă mai mică, reparații mai mici, o mai bună conservare a regresiei și reducerea efortului uman. Chiar și un rezultat negativ ar fi valoros dacă identifică ce feedback de rezolvare este prea scăzut pentru a ghida repararea semantică.

Cea de-a patra lucrare ar trebui să prezinte portofoliul formalismului și contractele de motoare tipizate. Evaluează adaptoarele relaționale, Datalog, decizionale, SMT, temporale și simbolice de execuție pe un model comun de artefacte. Lucrarea ar trebui să sublinieze conservarea semantică și certificatele, mai degrabă decât numărul de integrări. Un set mic de motoare cu adevărat diferite este mai convingător decât multe ambalaje superficiale.

A cincea lucrare ar trebui să fie sistemul nllAgent și lucrarea de evaluare. Ar trebui să distingă în mod clar eliberarea implementată de timpul de rulare mai larg. Acesta poate documenta revizuirile surselor imuabile, LongTextJS, CircuitJS, observațiile bazate pe cerere, compatibilitate, acoperire, execuție restrânsă, guvernanta învățării și publicării, verificarea

reluării, auditul canonic și produsele planului și procesul de probleme grave. Evaluarea ar trebui să includă mutații executabile, înregistrările de modele live, cazurile de identitate, guvernanta în lumea actuală, reperele de planificare și un verificator independent sau implementat separat, acolo unde este posibil.

O a șasea lucrare ulterioară ar putea studia realizarea constrângere a dovezilor. Ar compara generarea directă, redarea șablonului, redarea LLM constrânsă și verificarea post-hoc. Contribuția centrală ar fi o reprezentare de autorizare a mențiunilor și o dovadă că limbajul lizibil poate fi generat fără a schimba modalitatea, incertitudinea sau conținutul faptic. Această lucrare conectează execuția formală la nevoia practică de comunicare umană accesibilă.

Proiectul ar trebui să definească criterii explicite de eșec. Abordarea setului de candidați ar trebui reconsiderată dacă produce atât de multe alternative care pot / trebuie să fie neinformative, iar rafinamentul nu poate recupera precizia. Portofoliul de formalism ar trebui redus dacă rutarea și bug-urile încrucișate în formatactice depășesc beneficiile specializării. Oficializarea asistată LLM ar trebui limitată la a crea sprijin în cazul în care echivalența semantică rămâne slabă în conformitate cu documentele realiste. Realizarea dovedită constrânsă ar trebui să revină la șabloane dacă proza generată de model își depășește în mod repetat licența. Raționamentul din lumea actuală ar trebui să rămână în afara nucleului de încredere dacă nu pot fi aplicate controalele de sursă, timpul, jurisdicția și conflictul.

Onestitatea științifică necesită, de asemenea, o comparație cu alternativele mai simple. Multe verificări ale documentelor pot fi rezolvate prin parsiere deterministe, expresii regulate, validarea schemei, interogări de baze de date sau reguli de autor ale omului. Un LLM este justificat atunci când variația limbii, cartografierea ontologică sau propunerea semantică reduce semnificativ costul de autor sau crește acoperirea. Un nucleu simbolic este justificat atunci când starea exactă, reluarea, martorii sau materia de verificare independentă. O lucrare de cercetare ar trebui să raporteze atunci când un sistem mai simplu funcționează la fel de bine. Limbajul natural executabil nu este un motiv pentru a formaliza fiecare preferință editorială sau pentru a introduce un rezolvator în sarcini care au nevoie doar de un parsier de încredere.

Rezultatul practic probabil nu este o mașină care execută tot limbajul natural. Este o familie de compilatori semantic guvernați pentru domeniile document și agent, împărțirea sursei, împământarea, punerea la cale, forțarea și infrastructura de certificate. Fiecare compilator are un plic semantic declarat. LLMs face acei compilatori mai ușor de autoare, adaptați și explicați. Metodele formale fac ca rezultatele lor acceptate să fie reproductibile și contestabile. Combinația este utilă tocmai pentru că niciuna dintre componente nu este rugată să ofere ceea ce nu poate oferi în mod fiabil singur.

Prin urmare, programul de cercetare este atât mai modest, cât și mai consecvent decât sloganul „limbajul natural ca cod”. Limbajul natural rămâne un mediu deschis, social, dependent de context. Durata de funcționare creează proiecții executabile ale acestora pentru întrebările declarate. Aceste proiecții pot fi suficient de riguroase pentru a susține auditurile reale, planurile, protocoalele și verificările științifice, cu condiția ca sistemul să înregistreze distanța dintre sursă și obiect formal în loc să pretindă că distanța a dispărut.

Concluzie: de la formă probabilistică la consecință simbolică

Recenta renaștere a neuro-simbolului AI a produs un model arhitectural recunoscut. Modelele lingvistice mari sunt folosite acolo unde spațiul de căutare este lingvistic, eterogen sau dificil de enumerat. Sistemele formale sunt utilizate acolo unde starea exactă, consecvența, căutarea treptată limitată, martorii sau dovezile contează. Întrebarea interesantă nu mai este dacă metodele neuronale și simbolice pot fi combinate. Ei pot în mod clar. Întrebarea importantă este unde se află autoritatea și ce este verificată în mod independent.

Sondajul arată că cele mai credibile sisteme nu acordă modelului lingvistic autoritate finală asupra rezultatului. Limbile controlate traduc o suprafață restricționată în semantică formală. Pășitorii semantici generează interogări sau programe executabile. PAL delegează calculul la Python. SatLM delegează constrângerile unui rezolvator. Logic-LM și LINC delegează deducerea motoarelor logice. NL2TL și nl2spec traduc cerințele în specificațiile temporale. Sistemele de planificare utilizează modele formale și instrumente de verificare pentru a menține consistența de orizont lung. SymGPT compilează regulile în limba naturală ERC în constrângeri și utilizează execuția simbolică pentru a construi căi de contract concrete. SAIL utilizează LLMs pentru a căuta transformatoare abstracte, dar admite doar candidați care îndeplinesc obligațiile mecanice. Aceste sisteme diferă foarte mult, dar converg asupra principiului pe care îl propune generarea și un mecanism independent decide.

Această convergență nu trebuie supraestimată. Utilizarea de rezolvare nu verifică înțelegerea. Decalajul de formalizare rămâne problema centrală. Un demonstrator al teoreemului poate obține o concluzie validă de la predicate care denaturează textul. Un program poate returna răspunsul așteptat printr-o cale falsă. Un model SMT poate satisface constrângerile care omid cerința decisivă a utilizatorului. Un executor simbolic poate expune o cale software reală pentru o regulă greșit tradusă. Cele mai puternice sisteme actuale reduc erorile deductive și de calcul, reducând în același timp riscul la parsarea semantică, împământarea, ontologia și domeniul de aplicare.

Răspunsul adecvat este să nu abandonăm instrumentele formale, deoarece traducerea este imperfectă. Este pentru a face traducerea un artefact persistent, tastat, care poate fi revizuit. Sursa trebuie să rămână disponibilă. Fiecare observație semantică trebuie să identifice producătorul, dovezile, statutul și ontologia sa. Identitatea, timpul, modalitatea, cantitățile și autoritatea trebuie să fie reprezentate mai degrabă decât deduse din nou în timpul fiecărui răspuns. Interpretările alternative trebuie să rămână alternative atunci când distincția afectează rezultatul. Un motor simbolic ar trebui să execute doar obiecte împământate care îndeplinesc contracte explicite.

Interpretarea abstractă oferă o teorie utilă pentru partea globală a acestei arhitecturi. Acesta întreabă ce poate fi încheiat conservator dintr-un set mare de posibile state. Aplicare limbajului natural, stările concrete nu sunt „sensuri” nedefinite, ci structuri semantice dedelimitate admisibile în cadrul unei gramate animate declarate, ontologie, revizuire a surselor și politica de împământare. O stare abstractă poate reprezenta apoi ceea ce trebuie să dețină, să dețină, să nu poată ține, să nu poată ține, să conteze conflicte sau să rămână necunoscut. Garanția este condiționată de interpretarea stabilită, dar soliditatea condiționată este încă semnificativă atunci când condițiile sunt explicite și testate.

Execuția simbolică oferă complementul local. Atunci când o analiză abstractă raportează o posibilă încălcare, execuția simbolică sau un alt motor de constrângere poate căuta o interpretare și o cale concretă. O căutare de succes returnează un martor pe care un recenzor îl poate inspecta. O căutare eșuată expune o abstracție prea grosolană sau o alegere semantică incompatibilă. Cartografierea acestei defecțiuni înapoi la sursă se întinde creează o perfecționare semantică ghidată de contraexaple: modelul sau uman revizuieste cel mai mic angajament formal relevant, iar analiza este reluată. Aceasta este o utilizare mai disciplinată a feedback-ului LLM decât autocritica generică, deoarece semnalul de eroare este legat de o dependență formală.

Nici o limbă țintă nu este de natură să susțină orice formă utilă de proză executabilă. Literatura de literatură demonstrează deja dependența de alegerea în limbaj intermediar, iar sistemele contemporane reușesc prin utilizarea Python, SQL, logica temporală de prim ordin, logica PDDL, limbajelor de constrângere sau a gramaticii specifice domeniului în funcție de sarcină. Prin urmare, o durată de funcționare practică ar trebui să utilizeze un pivot semantic purtător de proveniență și un portofoliu de motoare formale. Evaluarea regională și Datalog se ocupă de interogările și închiderea; tabelele de decizie se ocupă de politicile finite; SAT/[Hao et al. 2025]/ZX00Q003QXZZ acoperă constrângerile de interacțiune; verificarea modelului gestionează comportamentele temporale; executarea simbolică gestionează martorii căii; modelele automate de mâner comandat; erorile de logică naturală selectate; asistenții dovediți se ocupă de afirmațiile formale de mare valoare.

Pluralitatea își introduce propriile riscuri de inginerie. Fiecare compilator poate distorsiona semantica, fiecare motor extinde baza de încredere, iar rutarea poate eșua. Portofoliul este justificat numai dacă interfețele sunt tastate, simbolurile partajate păstrează identitatea și proveniența, garanțiile motorului sunt explicite, iar rezultatele sunt normalizate într-un produs purtător de certificat canonic. O logică universală poate rămâne de preferat într-un domeniu îngust. Afirmația arhitecturală nu este că pluralitatea este întotdeauna mai bună; este că limbajul executabil larg necesită diferite structuri de calcul și nu ar trebui să le deghizeze în spatele unui singur vag DSL.

Întoarcerea de la simboluri la limbaj face parte din limita de încredere. Un ultim LLM nu trebuie să fie permis să transforme un rezultat condiționat într-o recomandare necondiționată, o posibilă încălcare a unei încălcări confirmate sau dovezi care lipsesc în probe de absență. Rezultatul canonic ar trebui să definească un set finant de revendicări autorizate cu modalitate, cantități, entități, ipoteze și proveniențe. Randamentul poate îmbunătăți lizibilitatea, dar ar trebui să rămână subordonat autorității respective. Realizarea dovedită a filNA pregătită este finalizarea naturală a arhitecturii.

Implementarea publică nllAgent este semnificativă pentru că deja întruchipează mai multe dintre aceste discipline. LongTextJS păstrează un program de documente soluționate în surse; CircuitJS reprezintă teorii reutilizabile; cererea de observare este derivată din circuite; compatibilitatea și acoperirea sunt explicite; modulele restricționate și extensiile revizuite limitează execuția ascunsă; învățarea are loc în stadializare de unică folosință; publicarea este manuală și imuabilă; verificarea reluării controlează constatările canonice; iar lipsa cunoștințelor nu sunt tratate ca conformitate. Aceste decizii fac din proiect o platformă credibilă pentru cercetare, chiar dacă nu stabilesc o înțelegere semantică generală.

Aceleași materiale publice identifică munca neterminată importantă: materializarea identitară generală, contractele primitive complete de tipare, mutația semantică executabilă, evaluarea

independentă a umbrelor, benchmarking-ul de model live, benchmark-ul de planificare și realizare și pachetele de cunoștințe guvernate din lumea actuală. Pentru această ediție nu a fost localizată nicio evaluare independentă evaluată de peer-review a nllAgent. Judecata oferită aici este, prin urmare, una arhitecturală: implementarea a ales limite neobișnuit de solide, dar cele mai puternice revendicări ale sale de apărare rămân analiza structurală și controlată. Valoarea sa va crește dacă amploarea semantică este extinsă numai prin producători, contracte, criterii de referință și dovezi explicite.

Următorul pas propus este de a nu înlocui LongTextJS sau CircuitJS cu un limbaj universal mai mare. LongTextJS ar trebui să se maturizeze ca o lume semantică finită, interogabilă, purtătoare de provenință. CircuitJS ar trebui să rămână o orchestrație restricționată și un strat de contract de dovezi. Motoarele înregistrate specializate ar trebui să efectueze executări relaționale, constrângerea, temporală, fluxul de date, automata sau execuția sensibilă la cale. Semants May/must ar trebui să devină operațional. Contraexemplele ar trebui să conducă la rafinament semantic concentrat. Produsele canonice CNL ar trebui să controleze ieșirea din limba finală.

Un program riguros de cercetare poate testa fiecare parte. Semantica setului de candidați poate fi comparată cu sistemele cu o singură traducere. Rafinamentul derivat din solvent poate fi comparat cu auto-reflecție. Contractele de probe tastate pot fi testate pentru transfer și asigurare falsă. Un portofoliu de formalism poate fi comparat cu o limbă intermediară universală. Domeniile abstracte pot fi verificate în funcție de seturile de interpretare explicite. Martorii simbolici pot fi evaluați pentru descoperirea defectului și utilitatea recenzorului. Realizarea dovedită constrânsă poate fi testată pentru fidelitate. Arhitecturile modulare și end-to-end pot fi comparate pe aceleași sarcini, mai degrabă decât pe diferite familii de referință.

Indicatorul decisiv ar trebui să fie o asigurare falsă, nu doar rata de răspuns. Un sistem care returnează „necunoscut pentru că identitatea nu este rezolvată” poate fi mai valoros decât unul care produce un verdict încrezător prin fuzionarea în tăcere a două persoane. Un sistem care se oprește pentru că dovezile din lumea actuală sunt învechite este mai demn de încredere decât unul care recuperează un fapt nesusținut. Un sistem care expune trei interpretări este mai bun decât cel care alege cel mai fluent. Abținerea adecvată face parte din semantica de execuție.

Prin urmare, limbajul natural executabil ar trebui să fie înțeles ca o relație de compilație guvernată, nu o proprietate intrinsecă a prozei. Textul sursă rămâne mai bogat decât orice proiecție executabilă. Sistemul selectează o sarcină, materializează semantica relevantă, păstrează incertitudinea, fundamentele simbolurilor, compilarea unui formalism adecvat, execută în limite explicite, verifică rezultatul și returnează o explicație limitată. Ceea ce devine executabil nu este limbajul în întregime, ci o proiecție semantică declarată a cărei presupuneri și limite rămân vizibile.

Această concepție mai îngustă este suficientă pentru un impact practic substanțial. Afirmările științifice pot fi verificate pentru provenință și coerență. Cerințele pot fi legate de teste și stările ciclului de viață. Politicile pot fi compilate în structurile de decizie. Planurile pot fi verificate înainte de realizare. Normele protocolului pot fi cartografiate la analizele de stat și de cale. Documentele lungi pot fi tăiate de teorii reutilizabile fără a cere unui model să-și amintească fiecare detaliu. Agenții pot folosi limbajul natural ca strat de autor și coordonare în timp ce delegă consecințele exacte ale timpului de funcționare simbolic.

Principiul central poate fi afirmat fără simbolism formal: lăsați LLM să descopere forma posibilă; lăsați nucleul de împământare să decidă ce simboluri au voie să denote; lăsați motoarele formale să determine ce urmează; și lăsați finala LLM să explice doar ce indică înregistrările simbolice. Acest principiu nu rezolvă orice problemă de limbaj. Se creează o graniță defensivă între interpretare și consecință, care este fundamentul necesar pentru ca limbajul natural să participe în condiții de siguranță în sistemele executabile.

Referințe

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma și Johan Bos. 2019. „Prima sarcină comună privind structura reprezentării discursului și evaluarea”. Actele Sarcinii comune IWCS privind analiza semantică. Asociația pentru Lingvistică Compuțională. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. „LangPro: demonstrator de teoreme pentru limbaj natural”. Actele Conferinței 2017 privind metodele empirice în procesarea limbii naturale: demonstrații de sistem. Asociația pentru Lingvistică Compuțională. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor și Christopher D. Manning. 2014. „NaturalLI: Deducție logică naturală pentru raționamentul de bun simț”. Actele EMNLP 2014. Asociația pentru Lingvistică Compuțională. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. „NaturalLanguageLinterAgent (nllAgent: ZX0003QXZ–CircuitJS, Documentare și probleme serioase”. Depozitare publică și documentație generată, inspectate 2 august 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d’Avila Garcez, Luciano Serafini și Michael Spranger. 2022. „Rețelele tensorilor Logici”. Inteligența 303: 103649 în domeniul artificială. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D’Elia, Camil Demetrescu și Irene Finocchi. 2018. „Un studiu de sinteză al tehnicilor de execuție simbolică”. ACM Computing Surveys 51(3): 1–39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O’Donoghue și Rosemary Monahan. 2025. „Un scurt sondaj privind formalizarea cerințelor software folosind modele lingvistice mari”. arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander și David Penz. 2025. „Să Faci LLMs Motiv? Problema limbajului intermediar în abordările neurosimbolice”. arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. „Analiză semantică cu o temperatură largă cu boxer”. Actele Conferinței 2008 privind Semantica în Prelucrarea Textului. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios și Shalom Lappin. 2026. „Neuro-Simbolul NLP: Taxonomie, Evaluare și direcții”. Frontiere în domeniul inteligenței 9 artificiale. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo și Yasutaka Kamei. 2026. „Execuție simbolică în regia descoperirii Vulnerabilității: o abordare LLM ghidată în KLEEZ”. arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang și Chuchu Fan. 2023. „NL2TL: Transformarea limbilor naturale în logica temporală folosind modele lingvistice mari”. Actele Conferinței 2023 privind metodele empirice în procesarea limbajului natural. Asociația pentru Lingvistică Compuțională. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu și Helmut Veith. 2000. „Rafinamentul de Abstractizare Condiționată Contraexemplă”. Verificarea asistată de calculator, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt și Caroline Trippel. 2023. „nl2spec: Traducerea interactivă a limbajului natural nestructurat în logică temporală cu modele lingvistice mari”. Verificarea asistată de calculator, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick și Radhia Cousot. 1977. „Interpretarea abstractă: un model de întârziere unificată pentru analiza statică a programelor prin construcție sau aproximarea punctelor de fixare”. Actele POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molletta și Gianluca Moro. 2025. „Inteligența artificială neuro-simbolică: un studiu de sinteză regizat de sarcină în era modelelor negre-Box”. Actele IJCAI 2025, Urmărirea sondajului, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai și Yifan Wang. 2026. „De la interpretare la compilație: Executarea bazată pe compilație a operatorilor semantici”. arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>
- Ferrari, Alessio și Paola Spoletini. 2025. „Inginerie formală de cerințe și modele lingvistice mari: o foaie de parcurs cu două sensuri”. Tehnologia informației și software 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>
- Fuchs, Norbert E. și Rolf Schwitter. 1996. „Atempto în limba engleză controlată (ACE)”. Actele primului atelier internațional privind aplicațiile lingvistice controlate. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>

- Fuchs, Norbert E., Kaarel Kaljurand și Tobias Kuhn. 2008. „ATtempto Conștient de engleză pentru cunoștințe”. *Raționament Web*, LNCS 5224, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan și Graham Neubig. 2022. „PAL: Modele lingvistice asistate de program”. *arXiv:2211.10435*. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson și Jonathan Berant. 2018. „Parsă semantică slab supraveghită cu exemple abstracte”. *Actele ACL 2018. Asociația pentru Lingvistică Computațională*. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. „pamaldi la SemEval-2026 Sarcina 11: raționament silogistic neuro-simbolic prin extracție de structură ghidată de LLM și validare deterministă”. *Actele celui de-al 20-lea atelier internațional privind evaluarea semantică. Asociația pentru Lingvistică Computațională*. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhan, Avaljot Singh și Gagandeep Singh. 2026. „SAIL: Interpreți de bază pentru sunet cu LLMsZ”. *Actele ACM privind programele Lingi 10 (PLDI): 1534–1559*. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu și Percy Liang. 2017. „De la limbaj la programe: Informarea învățării în arma de armare și probabilitatea marginală maximă”. *Actele ACL 2017. Asociația pentru Lingvistică Computațională*. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić și Luca Longo. 2024. „Neno-simbolul AI își îndeplinește promisiunile în procesarea limbajului natural? O Revizuire structurată”. *Web 15(4): 1265–1306-ZX003QXZZ*. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying și Dragomir Radev. 2024. „FOLIOZ: Raționament lingvistic natural cu logica de prim ordin”. *Actele EMNLP 2024. Asociația pentru Lingvistică Computațională*. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang și Chuchu Fan. 2025. „Modelele lingvistice mari pot rezolva planificarea imobiliară cu instrumente formale de verificare”. *Actele NAACL 2025, 3434–3483*. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu și Yongmei Liu. 2025. „LTRAG: Îmbunătățirea autonomiei și autofinamentului pentru raționamentul logic cu Gândul Guided RAG”. *Constatările ACL 2025, 2483–2493*. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu și Shih-hsi Alex Liu. 2026. „Interpretarea abstractă parametriză pentru verificarea transformatorului”. *Actele AAAI 2026*. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel și Percy Liang. 2019. „Robustismul certificat pentru înlocuirile cuvântului de cuvinte aversare”. *Actele EMNLP-IJCNLP 2019. Asociația pentru Lingvistică Computațională*. <https://aclanthology.org/D19-1423/>
- Karhikeyan, T, Om Dehlan, Mausam și Manish Gupta. 2025. „LRPLAN: O colaborare multi-agent a modelelor lingvistice mari și raționament pentru planificarea cu constrângeri implicite și explicite”. *Constatările EMNLP 2025, 8280–8310*. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango și Ondřej Dušek. 2026. „UFAL-CUNI la lucrarea SemEval-11Z operativă 11Z: o metodă neuro-simbolică modulară eficientă pentru raționamentul silogilor”. *Actele celui de-al 20-lea atelier internațional privind evaluarea semantică. Asociația pentru Lingvistică Computațională*. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. „Execuția simbolică și testarea programului”. *Comunicații de ACM 19(7): 385–394*. <https://doi.org/10.1145/360248.360252>
- Kirtania, Shashank, Priyanshu Gupta și Arjun Radhakrishna. 2024. „LOGIC-LM++: Rafinament multi-step pentru formule simbolice”. *Actele celui de-al doilea atelier de raționament al limbajului natural și explicații structurate, 56–63*. <https://aclanthology.org/2024.nlrse-1.6/>
- Kuhn, Tobias. 2014. „Un studiu de sinteză și o clasificare a limbilor naturale controlate”. *Lingvistică Prutațională 40(1): 121-170*. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. „Limbaj natural autonom la logica de prim ordin: un studiu de caz în detectarea erorilor logice”. *Constatările IJCNLP-AAACL 2025, 132–147*. <https://aclanthology.org/2025.findings-ijcnlp.8/>

- Lee, Kang-il, Segwang Kim și Kyomin Jung. 2023. „Parsă semantică slab supravegheată cu filtrarea programului de screening pe bază de execuție”. Actele EMNLP 2023, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng și Gregory J. Duck. 2025. „Un model lingvistic mare a fost lansat simbolic”. Actele ACM privind programele de limbaje 9 (OOPSLA2Z), Article 385Z, Article 385Z. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus și Ni Lao. 2017. „Mașini simbolice neuronale: Învățarea parselor semantice pe FreebaseZ cu o supraveghere slabă”. Actele ACL 2017. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen și Mirella Lapata. 2018. „Discurs Reprezentare Structura De Discurse”. Actele ACL 2018. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman și Regina Barzilay. 2018. „Generarea neuronală de expresii regulate de la limba naturală cu cunoștințe minime de domeniu”. Actele EMNLP 2018. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill și Christopher D. Manning. 2007. „Logica naturală pentru inferența textuală”. Actele Atelierului ACL-PASCAL privind componența textuală și parafrazarea. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill și Christopher D. Manning. 2009. „Un model extins de logică naturală”. Actele IWCS 2009. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester și Luc De Raedt. 2018. „DeepProbLog: Programare de logică exclusivistă neuronală”. Progrese în sistemele 31 de prelucrare a informațiilor neuronale. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin și Jiawen Xiong. 2025. „LLM Modelare automată pentru verificarea protocolului de securitate”. Actele celei de-a 47-a Conferințe Internaționale IEEE/ACM de Inginerie Software, 2025. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeonseok Kim, Chenyu Zhou și Chao Wang. 2025. „Poate LLMs raționa în mod oficial ca interpreți abstracți pentru analiza programului?”. arXiv:2503.12686. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li și Zhi Jin. 2017. „Cuplarea după Execuție Distribuită și simbolică pentru interogări de limbă naturală”. Actele ICML 2017. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum și Roger Levy. 2023. „LINC: O abordare neurosubcubică pentru raționamentul logic prin combinarea modelelor lingvistice cu dovezi logistice de prim ordin”. Actele EMNLP 2023. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang și William Yang Wang. 2023. „Logic-LM: Împuternicirea modelelor lingvistice mari cu soluționări simbolice pentru raționamentul logic fidel”. Descoperiri de EMNLP 2023. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong și Percy Liang. 2016. „Deducere a formularelor logistice din denotări”. Actele ACL 2016. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du și Xingnan Jin. 2025. „FoVer: Verificarea logică de prim ordin pentru raționamentul limbajului natural”. Tranzacțiile Asociației pentru Lingvistică Computațională 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>
- Tafjord, Oyvind, Bhavana Dalvi Mishra și Peter Clark. 2021. „ProofWriter: Generarea de implicații, dovezi și declarații adjudecătoare asupra limbajului natural”. Descoperiri de ACL-IJCNLP 2021. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise și Yilun Zhu. 2025. „Modele lingvistice mari ca formalizante de planificare: un studiu de sinteză”. Descoperiri de ACL 2025. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziafas, Georgios și Hamidreza Kasaei. 2026. „Planificarea sarcinilor neuro-simbolice și a obiectelor cu modele lingvistice mari”. Roboți autonomi 50. <https://doi.org/10.1007/s10514-026-10230-6>
- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov și Matthew Richardson. 2018. „Decodarea programului neuronal ghidat de execuție”. Actele primului atelier despre sinteza programului învățat. <https://arxiv.org/abs/1807.03100>

- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng și Bo Li. 2023. „Cuvântul inteligent Robustness îmbrățișează robustețea certificată la înlocuirile Word-ului Aversar”. Descoperiri de ACL 2023. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee și Qingchuan Zhao. 2025. „Un studiu de sinteză contemporan privind analiza programului asistat de modele lingvistice”. Tendințe în domeniul inteligenței artificiale. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang și Fei Wu. 2025. „Către date și cunoștințe-conștiință AI: Un studiu de sinteză privind calculul neuro-simbolic”. Tranzacțiile IEEE privind analiza tiparelor și inteligența mașinilor 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang și Dan Xu. 2026. „YNU-NLP la SemEval-ZX007QXZZĂ operațiunea 11Z: o abordare neuro-simbolică cu un conținut de deranjant al mecanismului de reflexie și raționament formal în modelele lingvistice”. Actele celui de-al 20-lea atelier internațional privind evaluarea semantică. Asociația pentru Lingvistică Computațională. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu și Tiancheng Zhang. 2025. „Autoformalizarea în era modelelor lingvistice mari: un studiu de sinteză”. arXiv:2505.23486. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyang Zhang, Nobuko Yoshida și Linhai Song. 2026. „SymGPT: Auditarea contractelor inteligente prin combinarea execuției simbolice cu modele lingvistice mari”. Actele ACM privind programele de programare limbi 10 (OOPSLA1), Article 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. „Autoformalizarea problemelor de raționament în limbaj natural în logică de ordinul întâi”. Actele EMNLP 2024. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong și Qiang Liu. 2020. „SAFER: O abordare fără structură pentru robustețe certificată la substituțiile cu cuvânt aversială”. Actele ACL 2020. Asociația pentru Lingvistică Computațională. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig și Greg Durrett. 2023. „SatLM: modele lingvistice asistate de satisfiabilitate folosind prompting declarativ”. Progrese în sistemele 36 de prelucrare a informațiilor neuronale. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen și Lewei He. 2025. „PlanningArena: Un reper modular pentru evaluarea multidimensională a planificării și învățării instrumentelor”. Actele ACL 2025, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>